

The Tao of Types

Thorsten Altenkirch

August 26, 2022

Contents

1	Introduction	7
2	Simple Types	9
2.1	Functions	10
2.2	Identity and composition	12
2.3	λ -calculus	15
2.4	Combinatory logic	17
2.5	Products and Sums	19
2.5.1	Sums	20
2.5.2	Products	21
2.5.3	Strong sums?	22
2.5.4	Finite Types	23
2.6	History	25
2.7	Exercises	27
3	Propositional logic	29
3.1	Boolean logic	29
3.2	Propositions as types	32
3.3	Classical principles	36
3.4	The negative translation	38
3.5	History	40
3.6	Exercises	40
4	Datatypes	45
4.1	The natural numbers	45
4.1.1	Recursion	47
4.1.2	Arithmetic operations	47
4.1.3	The Ackermann function	48
4.2	Iterator and recursor	50
4.2.1	Non primitive recursive functions	53
4.3	Inductive Types	54
4.3.1	Lists	54
4.3.2	Syntax trees	56
4.3.3	Rose trees	57

4.3.4	Ordinals	58
4.4	Positivity	61
4.5	Coinductive types	63
4.5.1	Streams	63
4.5.2	Conatural numbers	65
4.5.3	Coiterator and corecursor	66
4.6	History	68
4.7	Exercises	69
5	Dependent types	73
5.1	Vectors and dependent functions	73
5.2	The family of finite sets	76
5.3	Π -types and Σ -types	77
5.4	Relating simple and dependent type formers	78
5.5	Arithmetic of types	80
5.6	History	83
5.7	Exercises	84
6	Predicate logic	89
6.1	Predicates and Quantifiers	89
6.2	Some Tautologies	91
6.3	Equality	93
6.3.1	Propositional vs definitional equality	96
6.4	Proving properties of addition	97
6.5	Extending the negative translation	101
6.6	History	102
6.7	Exercises	103
7	Programs and proofs	107
7.1	Decidability of equality	108
7.1.1	Church's thesis	110
7.1.2	Uniqueness of equality proofs	111
7.2	Inductively defined relations: less or equal	111
7.3	An alternative definition of less or equal	114
7.4	History	116
7.5	Exercises	118
8	Deduction	121
8.1	The rules of natural deduction	121
8.2	Semantics	123
8.3	Equivalence with combinatory logic	125
8.4	Kripke models	129
8.5	Completeness	132
8.6	Normalisation	133
8.7	History	136
8.8	Exercises	136

9	Eliminators and coinduction	141
9.1	The eliminator for natural numbers	141
9.2	Iterator and eliminators for dependent types	143
9.3	Eliminators for equality	146
9.3.1	The recursor	146
9.3.2	J and K	146
9.3.3	Paulin-Mohring's eliminator	149
9.4	Coinduction for streams	151
9.5	History	154
9.6	Exercises	154
10	Universes and paradoxes	157
10.1	The paradox of trees	157

Chapter 1

Introduction

Chapter 2

Simple Types

Before jumping straight into dependent types with all its beautiful complexities we will first have a look at something simpler, appropriately named *simple types*. We start of with the most fundamental type of all, the function type $_ \rightarrow _$ (section 2.1).

One issue with simple types (which is repeated next chapter when we look at propositional logic) is that it is not clear what to start with since we haven't yet got any types to built functions with. For examples I will use types like $\mathbb{N}$ (the type of natural numbers: $0, 1, 2, \dots$). We will also look at constructions which work for any type, called *Set* in Agda. That is we will be using *polymorphic* constructions without introducing a formal account for polymorphism which we will do later once we have dependent types. A particular example of polymorphic constructions are identity *id* and composition $_ \circ _$ (section 2.2) which give us a first taste of category theory.

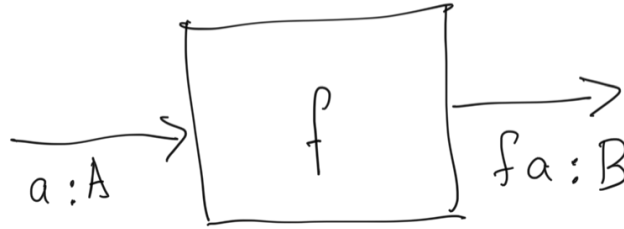
We will have a superficial look at λ -calculus (section 2.3), by which I always mean typed λ -calculus because the untyped one doesn't make any sense to me. At least enough to see that variables are causing a bit of a headache, something which can be avoided by using combinatory logic which features just two combinators (*S* and *K*), but the price we have to pay is the creation of some completely unreadable combinator code. However, I think it is worthwhile to know about them because they are a simple example of a general pattern in Type Theory: the reduction of an intuitively appealing notation into a collection of combinators that are better suited for metatheoretic study. This is reminiscent of the reduction of complex reasoning principles to some basic axioms in set theory. We will also return to combinators later from a more formal perspective.

Functions are not all there is to simple types and hence we are going to look at sums ($_ \uplus _$) and products ($_ \times _$) (section 2.5) which let us encode finite types. Finally we will meet some old acquaintances from high school in the context of type theory.

2.1 Functions

In Type Theory functions are a fundamental concept. This is different from set theory where functions are viewed as a special kind of relations, namely relations that assign to each element of the domain (the input) exactly one element of the codomain (the output). In Type Theory it is the other way around: we start with functions and actually we need functions to say what a relations is, namely a relation between A and B is a function from A and B to the type of propositions. But lets not get ahead of ourselves.

Intuitively, a function between types A and B is a black box where we can input elements of $a : A$ and out come elements of $f a : B$.



You may notice that we don't write $f(a)$ as usual in Mathematics and in many programming languages but instead save brackets and just write $f a$, which is common in functional programming languages like Haskell. We write $A \rightarrow B$ for the type of functions from A to B and hence we write $f : A \rightarrow B$ to express that f is such a function.

I was saying *black box*, even though for reasons of readability I didn't actually draw a black box. The meaning here is rather metaphorical, it means we cannot look into the box. If somebody gives you a function, all you can do it is to feed it elements and observe the output. You have no way to explore the mechanism (this is a sort of digital rights management for functions). This aspect will become important when we discuss extensionality later.

Let's look at some examples! For the purpose of illustration I will use some types like the natural numbers $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ ^{1 2} and the booleans $\text{Bool} = \{\text{true}, \text{false}\}$ even though we will only introduce them later. We will also use some basic operations such as addition $(_ + _)$ on natural numbers.

Let's define a function which adds 2 to its input

```
add2 :  $\mathbb{N} \rightarrow \mathbb{N}$ 
add2 x = x + 2
```

¹In computer science unlike in mathematics we start counting with 0, which enables us to answer questions like *How many elephants are in your fridge?*

²You may think I have succumbed to set theory already by using $\{\dots\}$. But this is not the case, this is just a convenient notation for labelled coproducts which we will introduce soon.

To define `add2` we introduce a parameter `x`. Now we can apply `add2` to a number as in 3 and observe `add2 3` which is (no surprise) 5. Lets do this step by step:

$$\begin{array}{ll} \text{add2 } 3 = 3 + 2 & \text{Use the definition of add2} \\ = 5 & \text{Calculating the sum.} \end{array}$$

In the first step we replace the parameter `x` with the actual argument 2 and then we use our knowledge about addition to conclude. The first step, replacing the parameter with the argument, is called β -reduction.

The definition above combines naming and defining the function. If we just want to write a function without giving it a name we can use λ -abstraction. That is we could have defined the same function as follows:

$$\begin{array}{l} \text{add2} : \mathbb{N} \rightarrow \mathbb{N} \\ \text{add2} = \lambda x \rightarrow x + 2 \end{array}$$

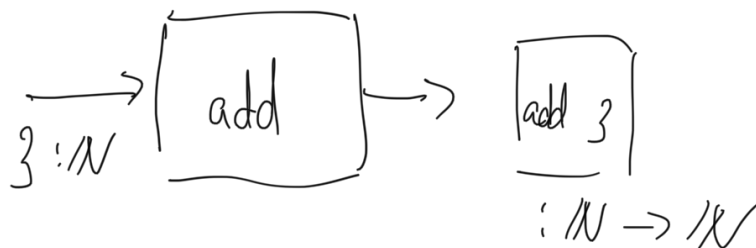
We can view the first definition as an abbreviation for the 2nd. We can now give a more detailed derivation where we differentiate the unfolding of a definition and β -reduction.

$$\begin{array}{ll} \text{add2 } 3 = (\lambda x \rightarrow x + 2) 3 & \text{Unfolding the definition of add2.} \\ = 3 + 2 & \beta\text{-reduction} \\ = 5 & \text{Calculating the sum.} \end{array}$$

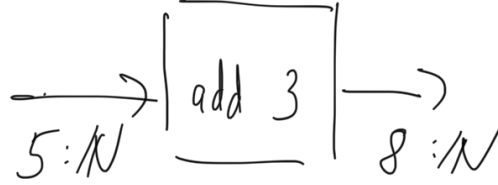
Using λ we can only define a functions with one parameter. To define a function with several parameters we use *currying*, that is a function that returns a function

$$\begin{array}{l} \text{add} : \mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N}) \\ \text{add} = \lambda x \rightarrow (\lambda y \rightarrow x + y) \end{array}$$

So `add 3` : $\mathbb{N} \rightarrow \mathbb{N}$ is the function $\lambda y \rightarrow 3 + y$ that is the function that adds 3, and `(add 3) 5` reduces to $5 + 3$ and hence 8. We can illustrate this – `add` outputs another box `add 3`:



which we can then feed with the 2nd argument to get the final output:



There are some syntactic conventions to simplify the use of currying: $\rightarrow$ is *right associative*, hence we can write $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ for $\mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$ and application is *left associative* hence we can write $g\ 3\ 5$ for $(g\ 3)\ 5$. Moreover we can combine several λ -abstractions and write $\lambda x\ y \rightarrow x + y$ for $\lambda x \rightarrow (\lambda y \rightarrow x + y)$. Consequently the same convention also applies for the explicit function definition:

```
add :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
add x y = x + y
```

Instead of returning a function we can also have functions that have functions as input (these are called *higher order functions*). An example is:

```
k :  $(\mathbb{N} \rightarrow \mathbb{N}) \rightarrow \mathbb{N}$ 
k h = h 2 + h 3
```

Note that in this case we cannot omit the brackets in the function type. k is a function that gets a function on the natural numbers as input and returns a number. What is $k\ add2$? We can reason as follows:

$$\begin{aligned} k\ add2 &= add2\ 2 + add2\ 3 \\ &= (2 + 2) + (3 + 2) \\ &= 9 \end{aligned}$$

2.2 Identity and composition

Some functions work for any type, we call them *polymorphic*. An example is the identity function $id : A \rightarrow A$ which works for every type $A : \text{Set}$. In Agda we can write :

```
id : {A : Set}  $\rightarrow A \rightarrow A$ 
id x = x
```

Writing $\{A : \text{Set}\} \rightarrow \dots$ indicates that the function works for every Set . Agda will automatically instantiate A that is we can just write $id\ 3 : \mathbb{N}$ and Agda infers that $A = \mathbb{N}$ in this case. In some cases it may be necessary to instantiate

the type variable explicitly in this case we can write $\text{id } \{\mathbb{N}\}$ or in case there are many parameters but we only want to instantiate a specific one we write $\text{id } \{A = \mathbb{N}\}$.

We will later explain types like $\{A : \text{Set}\} \rightarrow \dots$ which are actually instances of dependent types. In the moment we view polymorphism as a metamathematical notion, and we will use $A \ B \ C : \text{Set}$ to indicate variable types. In Agda this can be achieved by declaring:

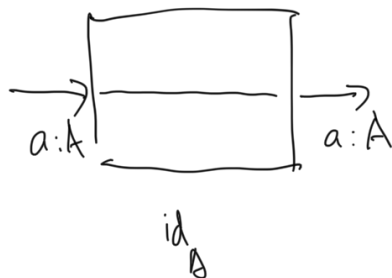
```
variable
  A B C : Set
```

which means that I can write

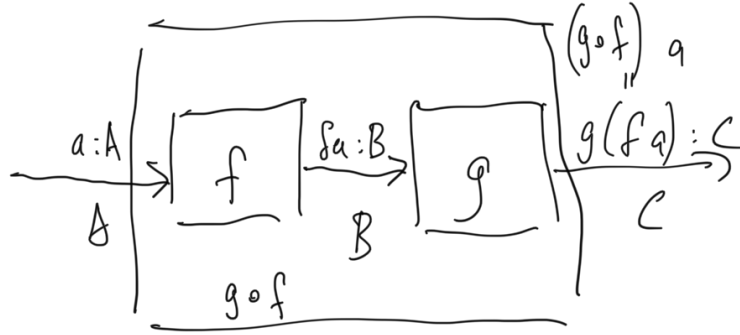
```
id : A → A
id x = x
```

and Agda will automatically translate the type into $\{A : \text{Set}\} \rightarrow A \rightarrow A$.

We can draw a picture for the identity function which just consists of a wire from the input to the output:



Another example is function composition: given $g : A \rightarrow B$ and $f : B \rightarrow C$ we can construct a new function $f \circ g : A \rightarrow C$ which feeds the output of g into the input of f :



Composition can also be defined as a polymorphic function:

$$\begin{aligned} _ \circ _ &: (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C) \\ (g \circ f) \ x &= g \ (f \ x) \end{aligned}$$

In Agda we can define infix functions (and indeed more complicated syntactic schemes) by using `_` to indicate where the arguments go. The price we have to pay for this flexibility is that we have to separate any syntactical component with spaces, e.g. Agda would read `g of` just as one identifier.

Let's do an example. Earlier we define the function `add2 : ℕ → ℕ`, in the same vain we can define a squaring function:

```
square : ℕ → ℕ
square x = x * x
```

Now we can construct `square ∘ add2 : ℕ → ℕ`, what is `(square ∘ add2) 3`? Let's calculate:

$$\begin{aligned} (\text{square} \circ \text{add2}) \ 3 &= \text{square} (\text{add2} \ 3) \\ &= \text{square} (3 + 2) \\ &= \text{square} \ 5 \\ &= 5 * 5 \\ &= 25 \end{aligned}$$

This example can also be used to show that composition is not commutative,

what is $(\text{add2} \circ \text{square})\ 3$?

$$\begin{aligned} (\text{add2} \circ \text{square})\ 3 &= \text{add2}\ (\text{square}\ 3) \\ &= \text{add2}\ (3 * 3) \\ &= \text{add2}\ 9 \\ &= 2 + 9 \\ &= 11 \end{aligned}$$

Hence we can see that $\text{square} \circ \text{add2}$ and $\text{add2} \circ \text{square}$ are different functions. In any case it isn't always possible to turn around composition. Assume as given a function $\text{isEven} : \mathbb{N} \rightarrow \text{Bool}$ that returns `true` if the input is even but `false` otherwise (e.g. $\text{isEven}\ 3 = \text{false}$ but $\text{isEven}\ 6 = \text{true}$). Now we can form $\text{isEven} \circ \text{square} : \mathbb{N} \rightarrow \text{Bool}$ (which actually behaves the same way as isEven) but it doesn't make sense to form $\text{square} \circ \text{isEven}$ because the output of isEven is `Bool` and this doesn't match the input of square which is $\mathbb{N}$.

You may note that there is a strange twist in the order of arguments to $_ \circ _$ which is visible in its type $(B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C)$. Why is it not $(A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C)$? And indeed when we evaluate $f \circ g$ we first evaluate g and then f . However, the reason is that function application is already written *the wrong way around*, that is when we evaluate $\text{square}\ (\text{add2}\ 3)$ we first evaluate $\text{add2}\ 3$ and then $\text{square}\ 5$, which is actually counterintuitive for people who write and read from left to right. Maybe we should write function application postfix $3\ \text{add2}$ instead of $\text{add2}\ 3$. However, it seems to be too late to change this convention. And since $(f \circ g)\ x = f\ (g\ x)$ it seems wiser not to change the order when defining composition.

2.3 λ -calculus

I don't want to give a fully formal introduction to λ -calculus at this point. We will do this later when we have developed enough type theory to do this in Agda itself.

As we have already discussed we can view the explicit definition of a function like

$$\begin{aligned} _ \circ _ &: (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C) \\ (f \circ g)\ x &= f\ (g\ x) \end{aligned}$$

is a shorthand for an explicit definition using λ -terms:

$$\begin{aligned} _ \circ _ &: (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C) \\ _ \circ _ &= \lambda f \rightarrow \lambda g \rightarrow \lambda x \rightarrow f\ (g\ x) \end{aligned}$$

And since unfolding definitions is standard practice in Mathematics and elsewhere we concentrate on what happens to λ -terms.

When we form a λ -term we are using variables. Hence beginners often ask *What is the type of a variable?* The answer is that we assume that we are given a type of any variable, this is often called the (typing-) context.

The other cases are application and λ -abstraction:

variable A variable x has the type we have assumed for it in the context.

application Given a term $M : A \rightarrow B$ and a term $N : A$ we can form a term $M N : B$.

abstraction Given a variable x ³ if from assuming $x : A$ (here we extend the context) we can show that $M : B$ then we can form $\lambda x \rightarrow M : A \rightarrow B$. I need to add that we will only assume one type for a variable, that is by assuming $x : A$ we are ignoring all earlier assumptions regarding x .

This covers the *pure* λ -terms which don't use constructions specific to datatypes like $\mathbb{N}$ or $\mathbf{Bool}$ (but we may assume that the types of standard functions like $_ + _$ or $\mathbf{isEven}$ are in the context).

I have already mentioned β -equality. By β -equality we mean that we can reduce an application of a λ -abstraction to an argument by substituting the variable with the argument, that is

$$(\lambda x \rightarrow M) N = M[x:=N]$$

where $M[x:=N]$ is M with all occurrences of x replaced by N .

Hang on, actually it is a bit more complicated. We certainly do not want to replace bound variables, that is $(\lambda x \rightarrow (\lambda x \rightarrow x)) 3$ should just be $\lambda x \rightarrow x$ and not $\lambda x \rightarrow 3$. We remedy this by saying that $M[x:=N]$ is M with all *free* occurrences of x replaced by N . An occurrence is free if it is not bound.

We are not done yet! Consider $\lambda x \rightarrow (\lambda y \rightarrow x + y)$ and let's say we have a variable y lying around. What is $(\lambda x \rightarrow (\lambda y \rightarrow x + y)) y$? If we mechanically replace x by y we obtain $\lambda y \rightarrow y + y$. This is wrong because the y to refers to the bound variable y and not to our top-level y . This is called variable capture and it has to be avoided.

But then what is $(\lambda x \rightarrow (\lambda y \rightarrow x + y)) y$? Here we introduce another equality, α -equivalence which says that bound variables can be consistently replaced, i.e. $\lambda y \rightarrow x + y = \lambda z \rightarrow x + z$. Using this we can avoid capture by replacing the bound variable y avoiding capture, that is $(\lambda x \rightarrow (\lambda y \rightarrow x + y)) y = \lambda z \rightarrow y + z$. Note that any variable but y would work here.

There is yet another equation, η -equality which is motivated by the idea of extensionality, that is that two functions which are equal when applied to the same argument should be considered equal. In pure λ -calculus this is obtained by adding the following equation: assume $M : A \rightarrow B$ and given a variable $x : A$ which does not appear free in M , then

$$\lambda x \rightarrow M x = M$$

³Note that here x is not the specific variable called x but a metavariable standing for any concrete variable.

Reading it from right to left it means that if you want to show that two functions M and N are equal it is enough to show that $M\ x = N\ x$ where x is a variable which does not appear in M or N because

$$\begin{aligned} M &= \lambda x \rightarrow M\ x \\ &= \lambda x \rightarrow N\ x \\ &= N \end{aligned}$$

As an example we use η -equality to show that $f \circ \text{id} = f$:

$$\begin{aligned} f \circ \text{id} &= \lambda x \rightarrow f(\text{id}\ x) \\ &= \lambda x \rightarrow f\ x \\ &= f \end{aligned}$$

2.4 Combinatory logic

The polymorphic functions from section 2.2, id and $_ \circ _$ are also called combinators. We are going to introduce two combinators in this section, called S and K , which are *functionally complete*. This means that every function which we can write in (pure) λ -calculus can be written just using these two combinators. This applies only to pure functions which do not refer to other datatypes (like $\mathbb{N}$ or Bool), which would require additional combinators. We are going to show this by providing a translation from λ -terms into terms only using combinators, which means that we can eliminate all variables. In a way combinators are a form of functional machine code and they have indeed been used for compilation.

We start with K which introduces constant functions:

$$\begin{aligned} K &: A \rightarrow B \rightarrow A \\ K\ x\ y &= x \end{aligned}$$

So for example $K\ 2 : B \rightarrow \mathbb{N}$ is the function that will always return 2. Note that this function is still polymorphic, it works for any type B .

The other combinator is S which generalizes composition, given types $A\ B\ C$:

$$\begin{aligned} S &: (A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C) \\ S\ f\ g\ x &= f\ x\ (g\ x) \end{aligned}$$

S is similar to $_ \circ _$ but the first functions gets to see not only the result of the 2nd but also the original input. So for example: $S\ (\lambda x\ y \rightarrow x + y)\ (\lambda x \rightarrow 2 * x)$ is the same as $\lambda x \rightarrow x + 2 * x$.⁴

We can derive id (which is usually written I in combinatory logic) from S and K . The idea is that we can derive it from S by replacing the first argument f with K this obtaining $S\ K = \lambda g\ x \rightarrow K\ x\ (g\ x) = \lambda g\ x \rightarrow x$. To get the identity we can substitute g with any function of the appropriate type. The

⁴We need to instantiate A, B, C with $\mathbb{N}$ which is done automatically by `agda`.

simplest choice is K , i.e. $\text{id} = S K K$. When implementing this in Agda we run into a bit of a technical problem, given a type A we would like to define:

```
id : A → A
id = S K K
```

Actually Agda is going to complain because it cannot infer the argument B for the 2nd K – indeed we can use any type here. The simplest choice is A and we need to tell Agda this explicitly:

```
id : A → A
id {A} = S K (K {B = A})
```

We will now show that every pure lambda term can be translated into combinatory logic. I will do this informally we will later use Agda itself to give formal proof of this theorem.

Now let's do composition as an example. We can write $_ \circ _$ as a pure λ -term

```
_∘_ = λ f g x → f (g x)
```

Actually for our purposes it is better to expand the shorthand for repeated λ -abstraction:

```
_∘_ = λ f → λ g → λ x → f (g x)
```

Our strategy is to eliminate the λ s from inside out, that is first we translate $\lambda x \rightarrow f (g x)$ into combinators. This term will still contain the variables f and g . However, next we abstract g and then f to obtain a term using only combinators that contains no variables.

The terms which appear during the translation are terms which have no λ -abstractions but may contain variables, applications and the combinators S and K . We are going to construct $\lambda x \rightarrow M$ by looking at each of the possible cases:

M=x In this case clearly $\lambda x \rightarrow x = \text{id}$ which we have already obtained as $S K K$.

M=y If the variable is different from x we have a function $\lambda x \rightarrow y$ which is a constant function, that is $K y$.

M = M₁ M₂ In this case we use the assumption that we know already how to translate $\lambda x \rightarrow M_1$ and $\lambda x \rightarrow M_2$ Now we have

$$\begin{aligned} \lambda x \rightarrow M &= \lambda x \rightarrow M_1 M_2 \\ &= S (\lambda x \rightarrow M_1) (\lambda x \rightarrow M_2) \end{aligned}$$

which explains the need for S .

M = K Clearly $\lambda x \rightarrow K$ is just a constant function returning K , hence $K K$.

M = **S** As above $\lambda x \rightarrow S$ is just $K S$.

Ok, let's apply the translation to our example $\lambda f \rightarrow \lambda g \rightarrow \lambda x \rightarrow f(g x)$. We start with

$$\begin{aligned} \lambda x \rightarrow f(g x) &= S (\lambda x \rightarrow f) (\lambda x \rightarrow g x) \\ &= S (K f) (S (\lambda x \rightarrow g) (\lambda x \rightarrow x)) \\ &= S (K f) (S (K g) id) \end{aligned}$$

Before proceeding let's introduce some optimisations to avoid truly gigantic terms. So for example we can see that $\lambda x \rightarrow g x$ is just g using η . Another optimisation is that if the variable x does not appear in M then $\lambda x \rightarrow M$ is just $K M$ and there is no need to go through M . Using these optimisations we translate $_o_$ in one go

$$\begin{aligned} \lambda f \rightarrow \lambda g \rightarrow \lambda x \rightarrow f(g x) &= \lambda f \rightarrow \lambda g \rightarrow S (\lambda x \rightarrow f) (\lambda x \rightarrow g x) \\ &= \lambda f \rightarrow \lambda g \rightarrow S (K f) g \\ &= \lambda f \rightarrow S (K f) \\ &= S (\lambda f \rightarrow S) (\lambda f \rightarrow K f) \\ &= S (K S) K \end{aligned}$$

While this term is comparatively small it is still hard to understand. The point of combinator is rather to show that we can avoid all the complications involving variables and just get away with S and K instead.

2.5 Products and Sums

So far we have only encountered the function type. In this section we will get to know some more basic type formers:

products Written $A \times B$, and also called cartesian product. In basic Mathematics we use products to represent coordinate systems, like

$$(1,1) : \mathbb{R} \times \mathbb{R}.$$

sums In Agda we write $A \uplus B$, I prefer $A + B$ but this clashes with $+$ on numbers. They are also called disjoint unions and coproducts.

Actually lets start with sums which are less common in conventional Mathematics but more common in Type Theory and functional programming.

2.5.1 Sums

Sums are necessary to represent alternatives. We have one thing or another. To define sums we use the **data** constructor:

```
data _ $\uplus$ _ (A B : Set) : Set where
  inj1 : A → A  $\uplus$  B
  inj2 : B → A  $\uplus$  B
```

This says that an element of $A \uplus B$ is either $\text{inj}_1 a$ where $a : A$ or it is $\text{inj}_2 b$ where $b : B$.

A simple example is that if in a form you can either fill in your order number (`OrdNum`) or your customer reference (`CustRef`) we can represent this as

`OrdNum  $\uplus$  CustRef.`

Note that even if we use the same number, lets say 1704 as an order number a customer reference we can always sure which is meant because we will use inj_1 1704 if it is an order number and inj_2 1704 if it is a customer reference.

The symbol for sums in Agda is a bit strange: it is a combination of the symbol for set theoretic union $\cup$ and $+$, this is related to the name for sums in set theory *disjoint union*. I prefer to just use $+$ but this clashes with the use for numbers.

How are $\cup$ and $\uplus$ related? Indeed, there is no operation $\cup$ on types. The reason is that $\cup$ in an *intensional* operator whose meaning depends on the choice of representation. For example let $A = \{a, b\}$ and $B = \{0, 1, 2\}$ then

$$\begin{aligned} A \cup B &= \{a, b, 0, 1, 2\} \\ A \uplus B &= \{\text{inj}_1 a, \text{inj}_1 b, \text{inj}_2 0, \text{inj}_2 1, \text{inj}_2 2\} \end{aligned}$$

However, we can view A just as the one possible representation of a type with 2 elements. Another choice would be $A' = \{0, 1\}$. What happens now with $\cup$ and $\uplus$?

$$\begin{aligned} A' \cup B &= \{0, 1, 2\} \\ A' \uplus B &= \{\text{inj}_1 0, \text{inj}_1 1, \text{inj}_2 0, \text{inj}_2 1, \text{inj}_2 2\} \end{aligned}$$

$A' \cup B$ now has only 3 elements, while $A' \uplus B$ again has 5 elements, hence is just another representation of a type with 5 elements. Thus $\uplus$ is insensitive to the representation of a type, this is a property which holds for all operations definable in Type Theory (and hence in Agda).

What can we do with $\uplus$? A useful general combinator is **case** that performs case analysis. we can define it using *pattern matching*:

```
case : (A → C) → (B → C) → A  $\uplus$  B → C
case f g (inj1 a) = f a
case f g (inj2 b) = g b
```

`case` chooses between two functions, `f` knows how to handle `A` and `g` know how to handle `B`. Going back to our example with the order form (`OrdNum` $\uplus$ `CustRef`): if we have a function to lookup an order using the order number

$$\text{lookupOrdNum} : \text{OrdNum} \rightarrow \text{Order}$$

and a function

$$\text{lookupCustRef} : \text{CustRef} \rightarrow \text{Order}$$

we can use `case` to combine them

$$\text{case lookupOrdNum lookupCustRef} : \text{OrdNum} \uplus \text{CustRef} \rightarrow \text{Order}$$

2.5.2 Products

We define products as a special case of a *record type*:

```
record _ $\times$ _ (A B : Set) : Set where
  field
    proj1 : A
    proj2 : B
```

that is we specify the projections, which are functions with the types:

$$\begin{aligned} \text{proj}_1 &: A \times B \rightarrow A \\ \text{proj}_2 &: A \times B \rightarrow B \end{aligned}$$

We can construct elements of $A \times B$ by specifying what the result is when applying projections. This is called *copatternmatching*. For example we can use this to define the pairing constructor which in Agda is just the `_ , _`, that is one doesn't have to put brackets around the tuple.

$$\begin{aligned} _,_ &: A \rightarrow B \rightarrow A \times B \\ \text{proj}_1 (a, b) &= a \\ \text{proj}_2 (a, b) &= b \end{aligned}$$

We can derive the constructor automatically by adding the constructor keyword to the record definition:

```
record _ $\times$ _ (A B : Set) : Set where
  constructor _ , _
  field
    proj1 : A
    proj2 : B
```

What can we do with products? We derive a function `curry` that turns a function from products into a curried function:

$$\begin{aligned} \text{curry} &: (A \times B \rightarrow C) \rightarrow (A \rightarrow B \rightarrow C) \\ \text{curry } f &= \lambda a \, b \rightarrow f (a, b) \end{aligned}$$

We can also do the reverse and translate a curried function back into one on products:

$$\begin{aligned}\text{uncurry} &: (A \rightarrow B \rightarrow C) \rightarrow (A \times B \rightarrow C) \\ \text{uncurry } g &= \lambda x \rightarrow g (\text{proj}_1 x) (\text{proj}_2 x)\end{aligned}$$

Indeed, the two functions are inverse to each other, that is $\text{curry} (\text{uncurry } g) = g$ and $\text{uncurry} (\text{curry } f) = f$. The first equation follows from what we already know:

$$\begin{aligned}\text{curry} (\text{uncurry } g) & \\ &= \lambda a \, b \rightarrow \text{uncurry } g (a, b) && \text{Unfolding curry} \\ &= \lambda a \, b \rightarrow g (\text{proj}_1 (a, b)) (\text{proj}_2 (a, b)) && \text{Unfolding uncurry} \\ &= \lambda a \, b \rightarrow g a \, b && \text{Evaluating projections} \\ &= g && \text{Use } \eta \text{ twice.}\end{aligned}$$

For the 2nd equation we need an η -equality for products which says that $(\text{proj}_1 x, \text{proj}_2 x) = x$. This is also called *surjective pairing*:

$$\begin{aligned}\text{uncurry} (\text{curry } f) & \\ &= \lambda x \rightarrow \text{curry } f (\text{proj}_1 x) (\text{proj}_2 x) && \text{Unfolding uncurry} \\ &= \lambda x \rightarrow f (\text{proj}_1 x, \text{proj}_2 x) && \text{Unfolding curry} \\ &= \lambda x \rightarrow f x && \text{Surjective pairing} \\ &= f && \eta \text{ for functions.}\end{aligned}$$

This shows that the type of functions over products and curried functions are equivalent. Most functional programmers prefer to work with curried functions.

2.5.3 Strong sums?

We can try to play a similar game for coproducts. First of all using a product we can uncurry `case`:

$$\begin{aligned}\text{case-c} &: (A \rightarrow C) \times (B \rightarrow C) \rightarrow A \uplus B \rightarrow C \\ \text{case-c} &= \text{uncurry case}\end{aligned}$$

and hence the inverse of `case` is:

$$\begin{aligned}\text{uncase} &: (A \uplus B \rightarrow C) \rightarrow (A \rightarrow C) \times (B \rightarrow C) \\ \text{uncase } h &= (\lambda a \rightarrow h (\text{inj}_1 a)), (\lambda b \rightarrow h (\text{inj}_2 b))\end{aligned}$$

we would hope that the functions are inverse to each other, that is that $\text{uncase } (\text{case-c } f) = f$ and $\text{case-c } (\text{uncase } g) = g$. The first one is we can show:

$$\begin{aligned}
 & \text{uncase } (\text{case-c } f) \\
 &= (\lambda a \rightarrow \text{case-c } f (\text{inj}_1 a)) , (\lambda b \rightarrow \text{case-c } f (\text{inj}_2 b)) && \text{Unfold uncase} \\
 &= (\lambda a \rightarrow \text{proj}_1 f a , \lambda b \rightarrow \text{proj}_2 f b) && \text{Unfold case-c and case.} \\
 &= (\text{proj}_1 f , \text{proj}_2 f) && \eta \text{ twice.} \\
 &= f && \text{Surjective pairing}
 \end{aligned}$$

However, the 2nd equality doesn't hold in Agda. Actually we can prove it using propositional equality $_ \equiv _$ we are going to introduce later. The equality $_ = _$ we have been considering here when talking *about* ⁵ λ -terms is called definitional equality.

Indeed the η -laws for functions and surjective pairing is already extending the notion of *definitional* equality, they are mainly introduced for convenience. Indeed, the corresponding laws for other types, eg. the natural numbers, would destroy decidability and hence not be very useful for a language like Agda. It is possible, even though not entirely straightforward, to add η -laws for $_ + _$ but this is usually avoided, mainly because it would quickly lead to an exponential blow up of cases and hence be computationally too costly.

2.5.4 Finite Types

The operation $_ \times _$ we have introduced handles the binary case of products. We can use it several times to handle more components, as in $A \times B \times C$. In practice it is better to use records which have more meaningful field names than proj_1 etc.

However, one interesting case not covered by binary products is the nullary product or unit type. In Agda this is denoted as $\top$ and can be defined as a record with no fields:

```
record  $\top$  : Set where
  constructor tt
```

It has one element $\text{tt} : \top$.

We can play the same game with sums and derive the empty sum, that is a sum with no injections:

```
data  $\perp$  : Set where
```

The empty sum is the empty type, it is a type with no elements. A version of **case** for the empty type is quite useful:

```
case $\perp$  : {A : Set}  $\rightarrow \perp \rightarrow A$ 
case $\perp$  ()
```

⁵Within Type Theory we cannot talk about definitional equality $_ = _$ is part of the language itself.

We don't need any cases for $\perp$ – Agda marks this by $()$ which basically means this case has been intentionally left empty. We observe that we get a function from the empty type into any type. This is ok, because we will never be able to run this function since there isn't an element in the empty type as the name suggests.

We can use the unit type $\top$ and sums $\uplus$ to construct some finite types. For example `Two : Set` is a type with 2 elements:

```
Two : Set
Two =  $\top \uplus \top$ 
```

Here are the elements:

```
zero2 : Two
zero2 = inj1 tt
one2 : Two
one2 = inj2 tt
```

We can add one more element to `Two` to construct `Three` a type with 3 elements:

```
Three : Set
Three =  $\top \uplus \text{Two}$ 
zero3 : Three
zero3 = inj1 tt
one3 : Three
one3 = inj2 zero2
two3 : Three
two3 = inj2 one2
```

However, as already in the case of products in practice it is usually preferable to use **data** to define labelled sums: ⁶

```
data Two : Set where
  zero2 one2 : Two
data Three : Set where
  zero3 one3 two3 : Three
```

How many elements are in `Two  $\uplus$  Three`? An example is `inj1 one2`. Ok there is the choice between `inj1` and `inj2`, if we choose `inj1` we then have a choice between 2 elements and if we choose `inj2` we have a choice of 3 elements, and hence we can choose out of $2 + 3 = 5$ elements.

<code>inj₁ zero₂</code>	<code>inj₁ one₂</code>	<code>inj₂ zero₃</code>	<code>inj₂ one₃</code>	<code>inj₂ two₃</code>
---	--	---	--	--

In general if A has n elements and B has m elements then $A \uplus B$ has $m + n$ elements.

⁶Note that we can combine constructors with the same type in one line.

How many elements are in $\text{Two} \times \text{Three}$? A typical example is $\text{one}_2, \text{two}_3$. Here we construct a pair and we have 2 independent choices, for the first components we have 2 options and for the second we have 3 and since these choices are independent we have $2 \times 3 = 6$ elements.

$\text{zero}_2, \text{zero}_3$	$\text{zero}_2, \text{one}_3$	$\text{zero}_2, \text{two}_3$
$\text{one}_2, \text{zero}_3$	$\text{one}_2, \text{one}_3$	$\text{one}_2, \text{two}_3$

In general if A has n elements and B has m elements then $A \times B$ has $m \times n$ elements.

What about the function type? How many elements are in $\text{Two} \rightarrow \text{Three}$? Here is a typical example:

```
foo : Two → Three
foo zero2 = two3
foo one2  = one3
```

We can define any such function by pattern matching: we have 3 choices in each case hence $3 \times 3 = 3^2 = 9$ elements. Now unlike products and sums function types are not symmetric. How many elements are in $\text{Three} \rightarrow \text{Two}$? Here again a typical example:

```
bar : Three → Two
bar zero3 = zero2
bar one3  = one2
bar two3  = zero2
```

There are 3 cases because there are 3 elements in **three** and in each case we have a choice of 2, hence we have $2 \times 2 \times 2 = 2^3 = 8$ elements. From these examples we can see that in general if A has n elements and B has m elements then $A \rightarrow B$ has m^n elements.

Indeed, in Mathematics the function type is often written as an exponential, that is $A \rightarrow B$ is written as B^A .

Previously, we have shown that types $A \times B \rightarrow C$ and $A \rightarrow B \rightarrow C$ are equivalent (via `curry` and `uncurry`). Writing this in exponential notation we find an old friend from high school:

$$C^{A \times B} = (C^B)^A$$

Similarly, with strong coproducts we would have that $(A \rightarrow C) \times (B \rightarrow C)$ and $A \uplus B \rightarrow C$ are equivalent (via `case-c` and `uncase`). If we write $+$ for $\uplus$ we arrive at another well known equation:

$$C^A \times C^B = C^{A+B}$$

2.6 History

λ -calculus was invented by Alonzo Church who used it in papers about formal logic in the 1930ies, e.g. [Chu32]. The story goes that the use of λ arose by

accident: Church used a graphical notation where he connected the binding and the usage of a variable. I imagine he wrote something like this:



But the typesetter (this was not only before latex but before computers) went back and said, *Sorry Professor, I can't do that. But this looks a bit like a λ. Is it ok if I use just this?*



I don't know whether this is actually true but it is a nice story. Church realised that one of the problems of his calculus was that he could define too many weird functions, including the so called *fixpoint combinator* which allows to compute the fixpoint of any function by general recursion. But what is the fixpoint of negation? To avoid this sort of problems Church also invented the typed λ -calculus.⁷ Church had two primitive types, ι (the greek letter *iota*) standing for individuals and o (the greek letter *omikron* but it is the same as an *o*) standing for propositions, motivated by his intended application to formal logic. In my presentation I am not considering any fixed set of base types but a view of them as type variables.

The untyped λ -calculus is still useful as a universal programming language which is an alternative to Turing machines. Indeed, when Alan Turing visited Church they realised that Turing machines and λ -calculus compute the same set of computable functions. The generalisation of this observation to all computational mechanisms is known as the Church-Turing thesis.

In many papers about typed λ -calculus one starts with the untyped λ -calculus and then introduces a type system on top, as a way of assigning types to untyped terms. I do not follow this view which I think is too syntactic and considers typed terms as primitive. We call this *intrinsic typing*. Yes, it is true if we have to implement λ -terms we will at some point think about untyped terms but we also need to think about strings, characters, bits etc and will not talk about these implementation details.

Combinatory logic was actually invented before λ -calculus by Moses Schön-

⁷A reference here is to a paper from 1940: [Chu40]

finkel in the 1920ies ⁸. Again his interest was in formal logic and he realised that combinators are a nice way to avoid the complexities of variables. This was also the motivation of Haskell Curry who reinvented combinatory logic more or less in the form we know it now. ⁹ Some people say that the idea of *currying* a function with several arguments which is named after Curry, should have been called *schönfinkeln* but this never caught on - I wonder why? A good overview over the history of λ -calculus and combinatory logic is [CH06].

To understand my presentation of products and sums better (also called coproducts) one needs to know about category theory. Category theory is basically an abstract form of algebra which was introduced by Saunders MacLane and others. ¹⁰ Their motivation came mainly from pure Mathematics, algebraic topology, I believe. However, later it was realised that category theory is also extremely useful for theoretical computer science. One important observation was Joachim Lambek's observation that a fundamental notion in category theory, *cartesian closed categories*, corresponds exactly to typed λ -calculus. In the categorical account products are already built-in and function types are characterised by the curry-uncurry-equivalence. One of the beautiful features of category theory is the presence of a mirror (*duality*) and it turns out that the mirror image (the dual) of products are sums or coproducts as we have introduced which are also extremely useful.

At this point you may wonder what the mirror image of functions is? This is a longer story but the short answer is that there isn't one. If you assume that there is one, the theory collapses: all λ -terms are equal.

λ -calculus is important for programming, to be precise in *functional programming*. Indeed the 2nd oldest programming language, LISP (The oldest is FORTRAN) was developed by John McCarthy in the late 1950ies and was inspired by the (untyped) λ -calculus [McC60]. Later types were introduced, famously by Robin Milner in the form of ML which stands for *Metalanguage* as a component of the Edinburgh LCF project (*Logic for computable functions*) [GMM⁺78]. Nowadays languages like Haskell (named after Haskell curry) ¹¹ and CAML are modern instances of typed functional programming and Agda's syntax and mechanisms borrow a lot especially from Haskell.

2.7 Exercises

1. Given

$$\begin{aligned} \text{add3} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{add3 } x &= x + 3 \end{aligned}$$

And for any type A:

⁸If you can read german, you can check out [Sch24].

⁹An early reference is See [Cur30]

¹⁰MacLanes book *Categories for the working Mathematician* is still worth a look. [ML78]

¹¹A good introduction to Haskell is [Hut16]

$$\begin{aligned} \text{tw} &: (A \rightarrow A) \rightarrow A \rightarrow A \\ \text{tw } f \text{ } n &= f (f \text{ } n) \end{aligned}$$

Consider tw tw add3 1 .

- (a) What is its type? Justify your answer.
 - (b) What is its value? Show every step of the evaluation.
2. Derive a term in combinatory logic (that is only using S and K) for tw using

$$\begin{aligned} \text{tw} &: (A \rightarrow A) \rightarrow A \rightarrow A \\ \text{tw} &= \lambda f \rightarrow \lambda n \rightarrow f (f \text{ } n) \end{aligned}$$

3. In section 2.2 we defined identity and composition:

$$\begin{aligned} \text{id} &: A \rightarrow A \\ \text{id } x &= x \\ _ \circ _ &: (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C) \\ (f \circ g) \text{ } x &= f (g \text{ } x) \end{aligned}$$

Using β and η show that the following equations ¹² hold definitionally:

$$\begin{aligned} f \circ \text{id} &= f \\ \text{id} \circ f &= f \\ f \circ (g \circ h) &= (f \circ g) \circ h \end{aligned}$$

4. Define an operation with the following type:

$$A \times : (B \rightarrow C) \rightarrow A \times B \rightarrow A \times C$$

and show that the following equations hold using β , η and surjective pairing:

$$\begin{aligned} A \times \text{id} &= \text{id} \\ A \times (f \circ g) &= (A \times f) \circ (A \times g) \end{aligned}$$

This shows that the operation $A \times _$ on types is (definitionally) a functor.

5. Find elements of the following types using only pure λ -terms and the combinators for sums and products (proj_1 , proj_2 , $_$, $_$, inj_1 , inj_2 , case).

$$\begin{aligned} x_0 &: (A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C) \\ x_1 &: (A \rightarrow B) \rightarrow ((A \rightarrow C) \rightarrow C) \rightarrow ((B \rightarrow C) \rightarrow C) \\ x_2 &: A \times (B \uplus C) \rightarrow (A \times B) \uplus (A \times C) \\ x_3 &: (A \times B) \uplus (A \times C) \rightarrow A \times (B \uplus C) \end{aligned}$$

¹²These are the equations of a category. We are establishing that types and functions from a category, definitionally.

Chapter 3

Propositional logic

When we study logic we usually start with propositional logic, which introduces the basic logical connectives:

conjunction The logical operation **and** which is usually denoted as $_ \wedge _$ or as $_ \& _$ as an operation on `Bool`.

disjunction The logical operation **or** which is denoted as $_ \vee _$ or as $_ | _$ for `Bool`.

negation The logical operation **not** which is denoted as $\neg$ or as $!$ for `Bool`.

implication The logical operation **if ... then ...** which is written as $_ \Rightarrow _$ but has not standard notation as an operation on booleans.

logical equivalence We often say **if, and only if** which is abbreviated as **iff**, this is written as $\Leftrightarrow$. For `Bool` this just corresponds to equality on booleans which we will write $_ == _$.

Propositional logic is usually the first step which is later followed by predicate logic which introduces the quantifiers **for all** $\forall$ and **exists** $\exists$. While propositional logic is simpler it is somehow abstract because we cannot say anything interesting because we cannot form interesting propositions yet. Hence we use logical variables to form propositions and study logical tautologies, that is propositions that are true for every assignment of truth values.

Before we explain the type theoretic approach using propositions as types, we review the classical semantics using `Bool`.

3.1 Boolean logic

What is a truth value or a proposition? The classical approach is to equate propositions with `Bool` which is a type with exactly two values `true` and `false`. We could use the operations on types we have introduced in the last section to

define `Bool` as $\top \sqcup \top$ but we will just use the mechanism of datatypes and define:

```
data Bool : Set where
  true  : Bool
  false : Bool
```

Now to define `_&_` we use pattern matching:

```
_&_ : Bool → Bool → Bool
true & y = y
false & y = false
```

The idea is that if the first parameter is `true` then the result of `true & y` is just `y`, while if the first parameter is `false` then `false & y == false` for all `y`. We could have also used the 2nd parameter since the operation is commutative. Actually this is a tautology: $x \& y == y \& x$ that is this expression is `true` for all possible values of $x, y : \text{Bool}$. We can show this by drawing a truth table:

x	y	x & y	y & x	x & y == y & x
false	false	false	false	true
false	true	false	false	true
true	false	false	false	true
true	true	true	true	true

Actually I haven't yet defined the function `_==_` but I leave this as an exercise.

Next we define the operation boolean or:

```
_|_ : Bool → Bool → Bool
true | y = true
false | y = y
```

The idea is similar as for `&`: if the first parameter is `true` we already know that `true | y == true` but if the first parameter is `false` then `false | y == y`. We leave the verification that `_|_` is commutative as an exercise.

Already with `_&_` and `_|_` we can explore a number of laws, which are the laws of *boolean algebra*. As in arithmetic we have the law of distributivity $x * (y + z) == x * y + x * z$ which now becomes $x \& (y | z) == x \& y | x \& z$. When writing the law we already exploited the convention that `_&_` binds stronger than `_|_`.

As above we can verify the law by drawing the truth table:

x	y	z	$x \& (y \mid z)$	$x \& y \mid x \& z$	$x \& (y \mid z) == x \& y \mid x \& z$
false	false	false	false	false	true
false	false	true	false	false	true
false	true	false	false	false	true
false	true	true	false	false	true
true	false	false	false	false	true
true	false	true	true	true	true
true	true	false	true	true	true
true	true	true	true	true	true

Actually we don't have to look at the whole truth table but we can show directly that $x \& (y \mid z)$ and $x \& y \mid x \& z$ are equal. We only look at x we have two case $x = \text{true}$ and $x = \text{false}$. In the case $x = \text{true}$ we can reason for the left hand side:

$$\begin{aligned} x \& (y \mid z) &= \text{true} \& (y \mid z) && \text{using the definition of } _ \& _ \\ &= y \mid z \end{aligned}$$

and for the right hand side:

$$\begin{aligned} (x \mid y) \& (x \mid z) &= (\text{true} \& y) \mid (\text{true} \& z) && \text{using the definition of } _ \& _ \text{ twice} \\ &= y \& z \end{aligned}$$

Hence both sides are identical. In the case $x = \text{false}$ we can also show that both sides are identical:

$$\begin{aligned} x \& (y \mid z) &= \text{false} \& (y \mid z) && \text{using the definition of } _ \& _ \\ &= \text{false} \end{aligned}$$

and for the right hand side:

$$\begin{aligned} (x \mid y) \& (x \mid z) &= (\text{false} \& y) \mid (\text{false} \& z) && \text{using the definition of } \& \text{ twice} \\ &= \text{false} \mid \text{false} && \text{using the definition of } \mid \\ &= \text{false} \end{aligned}$$

Other important equations are the de Morgan laws, which are basically the observation that the truth tables of $_ \& _$ and $_ \mid _$ can be obtained from each other by switching **true** and **false**. To express this we need negation:

$$\begin{aligned} !_ &: \text{Bool} \rightarrow \text{Bool} \\ ! \text{true} &= \text{false} \\ ! \text{false} &= \text{true} \end{aligned}$$

The de Morgan laws now are:

x	y	$!(x \mid y)$	$!x \& !y$	$!(x \mid y) == !x \& !y$
false	false	true	true	true
false	true	true	true	true
true	false	true	true	true
true	true	false	false	true

x	y	$!(x \& y)$	$!x \mid !y$	$!(x \& y) == !x \mid !y$
false	false	true	true	true
false	true	false	false	true
true	false	false	false	true
true	true	false	false	true

Let us verify the 2nd one again by equational reasoning, again we only need to consider x .

x=true

$$\begin{aligned} !(x \& y) &= !(\text{true} \& y) \\ &= !y \end{aligned}$$

$$\begin{aligned} !x \mid !y &= !\text{true} \mid !y \\ &= \text{false} \mid !y \\ &= !y \end{aligned}$$

x=false

$$\begin{aligned} !(x \& y) &= !(\text{false} \& y) \\ &= !\text{false} \\ &= \text{true} \end{aligned}$$

$$\begin{aligned} !x \mid !y &= !\text{false} \mid !y \\ &= \text{true} \mid !y \\ &= \text{true} \end{aligned}$$

We will soon see how paper proofs like this can be done in Agda itself (see section 6.7 exercise 3).

3.2 Propositions as types

While the boolean semantics is quite standard it has some serious shortcomings. Can we really identify truth values with `Bool`? There are many propositions of which we don't know whether they are true or false hence we cannot assign a boolean value to them. The problem becomes obvious once we move on to predicate logic: a predicate over the natural number would be just a function $P : \mathbb{N} \rightarrow \text{Bool}$ but what is the truth value of the statement that P is true for all natural numbers $\forall x : \mathbb{N} \rightarrow P\ x : \text{Bool}$? We would have to check P for infinitely many inputs which is clearly not possible.

In Type Theory we use another definition of truth values, instead of saying that a proposition is either true or false, we say that a proposition is something

we can have evidence for. A proposition holds if we can find evidence. It isn't always clear whether we can find evidence for a proposition hence this is different from the boolean explanation. Indeed this interpretation is associated with *intuitionistic logic*, which has different laws than boolean algebra (it is called Heyting algebra).

To be precise we identify propositions with `Set` that is we assign to any proposition the type of evidence for it.¹

$$\text{prop} = \text{Set}$$

We will use `P Q R` to stand for arbitrary propositions (i.e. elements of `prop`). What is evidence for `P and Q` that is `P ∧ Q`? Clearly it is a pair `p , q` of evidence `p : P` and evidence `q : Q`. That is:

$$\begin{aligned} _ \wedge _ &: \text{prop} \rightarrow \text{prop} \rightarrow \text{prop} \\ P \wedge Q &= P \times Q \end{aligned}$$

Similarly, evidence for `P or Q` is the sum of evidence for `P` and `Q`:

$$\begin{aligned} _ \vee _ &: \text{prop} \rightarrow \text{prop} \rightarrow \text{prop} \\ P \vee Q &= P \uplus Q \end{aligned}$$

Implication which doesn't play a central role in boolean logic is central in intuitionistic logic. What is evidence for `P ⇒ Q`? It is a function that transforms evidence for `P` to evidence for `Q`. Hence:

$$\begin{aligned} _ \Rightarrow _ &: \text{prop} \rightarrow \text{prop} \rightarrow \text{prop} \\ P \Rightarrow Q &= P \rightarrow Q \end{aligned}$$

We also need to define the logical constants: `True` has a trivial proof, while `False` is the empty type, there is no evidence for it.

$$\begin{aligned} \text{True} &: \text{prop} \\ \text{True} &= \top \\ \text{False} &: \text{prop} \\ \text{False} &= \perp \end{aligned}$$

Having all these we can define negation by saying that `¬ P` holds if `P` entails impossible:

$$\begin{aligned} \neg &: \text{prop} \rightarrow \text{prop} \\ \neg P &= P \Rightarrow \text{False} \end{aligned}$$

We also define logical equivalence as implications going in both direction, which is somehow obvious from the symbol used:

¹Don't confuse this with Agda's proof-irrelevant `Prop`, which identifies all elements of a proposition. We will discuss proof-irrelevance later.

$$\begin{aligned} _ \Leftrightarrow _ &: \text{prop} \rightarrow \text{prop} \rightarrow \text{prop} \\ P \Leftrightarrow Q &= (P \Rightarrow Q) \wedge (Q \Rightarrow P) \end{aligned}$$

In the boolean setting we were able to use equality $_ \equiv _$ for logical equivalence. This is not possible in the propositions as types explanation presented here. ²

We use the usual assumptions how to read logical formulas:

- $_ \wedge _$ binds stronger than $_ \vee _$.
- $_ \vee _$ binds stronger than $_ \Rightarrow _$.
- $_ \Rightarrow _$ binds stronger than $_ \Leftrightarrow _$.
- $_ \Rightarrow _$ is right associative (like $_ \rightarrow _$).

The following table summarizes our definitions of logical connectives:

<i>latin</i>	<i>english</i>	<i>logic</i>	<i>types / definitions</i>
Conjunction	And	$P \wedge Q$	$P \times Q$
Disjunction	Or	$P \vee Q$	$P \uplus Q$
Implication	if-then	$P \Rightarrow Q$	$P \rightarrow Q$
Verum	true	True	$\top$
Falsum	false	False	$\perp$
Negation	not	$\neg P$	$P \Rightarrow \text{False}$
Equivalence	if-and-only-if	$P \Leftrightarrow Q$	$(P \Rightarrow Q) \wedge (Q \Rightarrow P)$

Let's revisit the tautologies we have shown using the boolean interpretation earlier. Certainly $_ \wedge _$ is commutative, because $_ \times _$ is. We only need to show one direction because the other one follows by symmetry. We can show it easily by copatternmatching:

$$\begin{aligned} \wedge\text{-comm}\rightarrow &: P \wedge Q \Rightarrow Q \wedge P \\ \text{proj}_1 (\wedge\text{-comm}\rightarrow x) &= \text{proj}_2 x \\ \text{proj}_2 (\wedge\text{-comm}\rightarrow x) &= \text{proj}_1 x \\ \wedge\text{-comm} &: P \wedge Q \Leftrightarrow Q \wedge P \\ \text{proj}_1 \wedge\text{-comm} &= \wedge\text{-comm}\rightarrow \\ \text{proj}_2 \wedge\text{-comm} &= \wedge\text{-comm}\rightarrow \end{aligned}$$

Since we have declared a constructor for $_ \times _$, namely $_, _$ we could have proven $\wedge\text{-comm}\rightarrow$ also by pattern matching:

$$\begin{aligned} \wedge\text{-comm}\rightarrow &: P \wedge Q \Rightarrow Q \wedge P \\ \wedge\text{-comm}\rightarrow (p, q) &= q, p \end{aligned}$$

which is sometimes more convenient as the next example shows.

Let's prove that $_ \wedge _$ distributes over $_ \vee _$, that is:

²However, we will recover this identification when we introduce Homotopy Type Theory later, which also uses a small modification of the translation presented here.

$$\wedge\vee\text{-distr} : P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$$

Let's use the Agda system to construct the proof / program interactively. We start with

$$\wedge\vee\text{-distr} = ?$$

Our first step is to use copattern matching to split $_ \Leftrightarrow _$ which is defined as a conjunction in its two parts:

$$\begin{aligned} \text{proj}_1 \wedge\vee\text{-distr} &= ? \\ \text{proj}_2 \wedge\vee\text{-distr} &= ? \end{aligned}$$

We first focus on the $_ \Rightarrow _$ direction. This is a function hence let us introduce an assumption / parameter:

$$\text{proj}_1 \wedge\vee\text{-distr } x = ?$$

Our goal is to prove $P \wedge Q \vee P \wedge R$ and we can use $x : P \wedge (Q \vee R)$. Our next step is to analyze our assumption using pattern matching:

$$\text{proj}_1 \wedge\vee\text{-distr } (p, qr) = ?$$

We now have $p : P$ and $qr : Q \vee R$. We analyze qr further which means we get two cases:

$$\begin{aligned} \text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_1 q) &= ? \\ \text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_2 r) &= ? \end{aligned}$$

Now we are cooking with gas. In the first case we have $q : Q$ hence we can prove $P \wedge Q \vee P \wedge R$. But first of all we have to decide whether we use inj_1 or inj_2 . No process for guessing which one:

$$\text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_1 q) = \text{inj}_1 ?$$

Now we are left to prove $P \wedge Q$ which is no problem with the assumptions lying around:

$$\text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_1 q) = \text{inj}_1 (p, q)$$

The next line works the same way:

$$\text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_2 r) = \text{inj}_2 (p, r)$$

At this point I am getting bored and just complete the program:

$$\begin{aligned} \text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_1 q) &= \text{inj}_1 (p, q) \\ \text{proj}_1 \wedge\vee\text{-distr } (p, \text{inj}_2 r) &= \text{inj}_2 (p, r) \\ \text{proj}_2 \wedge\vee\text{-distr } (\text{inj}_1 (p, q)) &= p, \text{inj}_1 q \\ \text{proj}_2 \wedge\vee\text{-distr } (\text{inj}_2 (p, r)) &= p, \text{inj}_2 r \end{aligned}$$

I hope you have been getting bored too. Indeed it shouldn't be difficult to write this little program without much interaction but I wanted to demonstrate how to do it step by step because in some cases this may be necessary.

In this instance the two functions we define to show a logical equivalence are actually inverse to each other, but this is not always the case. So for example:

$$\begin{aligned} \wedge\text{-dbl} &: P \Leftrightarrow P \wedge P \\ \text{proj}_1 \wedge\text{-dbl } p &= p, p \\ \text{proj}_2 \wedge\text{-dbl } (p, -) &= p \end{aligned}$$

These two functions are not inverse, e.g. if we start with 2, 3 and go back and forth we end up with 2, 2.

Next we revisit the de Morgan laws, the first one is no problem:

$$\begin{aligned} \text{dm}_1 &: \neg (P \vee Q) \Leftrightarrow \neg P \wedge \neg Q \\ \text{proj}_1 \text{dm}_1 f &= (\lambda p \rightarrow f(\text{inj}_1 p)), \lambda p \rightarrow f(\text{inj}_2 p) \\ \text{proj}_2 \text{dm}_1 (g, h) (\text{inj}_1 p) &= g p \\ \text{proj}_2 \text{dm}_1 (g, h) (\text{inj}_2 q) &= h q \end{aligned}$$

But for the 2nd one we run into a problem in one direction:

$$\begin{aligned} \text{dm}_2 &: \neg (P \wedge Q) \Leftrightarrow \neg P \vee \neg Q \\ \text{proj}_1 \text{dm}_2 f &= ? \\ \text{proj}_2 \text{dm}_2 (\text{inj}_1 f) (p, q) &= f p \\ \text{proj}_2 \text{dm}_2 (\text{inj}_2 g) (p, q) &= g q \end{aligned}$$

We cannot show $\neg (P \wedge Q) \Rightarrow \neg P \vee \neg Q$. When we attempt to show this we assume $f : \neg (P \wedge Q)$, and now we need to establish $\neg P \vee \neg Q$. The problem is that we have no information which case we should choose. E.g. if we choose to show $\neg P$ we assume $p : P$ and we need to derive **False**. The only possibility is to use f but this requires $P \wedge Q$ as the input and we only have P .

After all this should not be surprising: We may know that not both propositions can be false but we don't know which one it is. E.g. if somebody tells us that it is not true that both it rains and we go to the zoo, then we still don't know whether it doesn't rain or we don't go to the zoo or both.

So indeed while the 2nd de Morgan law holds in the boolean semantics it doesn't hold intuitionistically (it is not a law of Heyting algebra).

3.3 Classical principles

So what is the exact difference between the boolean logic which is also called *classical logic*, and the evidence based logic which is called intuitionistic logic. It seems that certain propositions, or actually propositional schemes since they involved propositional variables, which are provable classically are not provably intuitionistically. But what exactly is the difference?

It turns out that the extra power of classical logic is exactly the assumption that every proposition is either true or false. This is expressed as the following

scheme which is called the *law of the excluded middle* or in latin *tertium non datur* (the third is not given). We define

$$\begin{aligned} \text{TND} &: \text{prop} \rightarrow \text{prop} \\ \text{TND } P &= P \vee \neg P \end{aligned}$$

This means that we write $\text{TND } P$ to express that excluded middle holds for P . Classical logic corresponds to assuming that TND holds for all propositions.

Let's go back to the instance of the de Morgan law which we couldn't prove. It turns out that we can prove it using TND :

$$\begin{aligned} \text{dm}_{21}\text{tnd} &: \text{TND } P \rightarrow \neg(P \wedge Q) \Rightarrow \neg P \vee \neg Q \\ \text{dm}_{21}\text{tnd} (\text{inj}_1 p) f &= \text{inj}_2 (\lambda q \rightarrow f(p, q)) \\ \text{dm}_{21}\text{tnd} (\text{inj}_2 np) f &= \text{inj}_1 np \end{aligned}$$

The idea is that If P holds then we can prove $\neg Q$ because from assuming Q we can derive a contradiction using the premise $\neg(P \wedge Q)$. On the other hand if $\neg P$ holds we are done using just this.

We can see that using TND we can simulate truth tables: this proof corresponds to the algebraic proof for booleans we have done in section 3.1.

There is an alternative principle we can use, which we will see is equivalent to TND . This is the *principle of indirect proof*, also called *reductio ad absurdo* (reduction to the absurd) in latin. This you see in many mathematical proofs: to prove a proposition P we assume $\neg P$ and derive a contradiction. Since deriving a contradiction is the same as negation this just means that $\neg(\neg P)$ implies P :

$$\begin{aligned} \text{RAA} &: \text{prop} \rightarrow \text{prop} \\ \text{RAA } P &= \neg(\neg P) \rightarrow P \end{aligned}$$

We can show that we can derive RAA from TND using the same technique we have used for the de Morgan law above:

$$\begin{aligned} \text{tnd} \rightarrow \text{raa} &: \text{TND } P \rightarrow \text{RAA } P \\ \text{tnd} \rightarrow \text{raa} (\text{inj}_1 p) \text{ nnp} &= p \\ \text{tnd} \rightarrow \text{raa} (\text{inj}_2 np) \text{ nnp} &= \text{case}\bot (\text{nnp } np) \end{aligned}$$

We assume $\text{nnp} : \neg(\neg P)$. Now we analyze P using tnd : if it is $p : P$ we are done. On the other hand if we have $np : \neg P$ we can derive a contradiction $\text{nnp } np : \text{False}$ and then use $\text{case}\bot$ to conclude.

The other direction is somehow more interesting: while we cannot prove $P \vee \neg P$ in general, we can show that it cannot be false. That is we can prove $\neg(\neg(P \vee \neg P))$ ³

$$\begin{aligned} \text{nntnd} &: \neg(\neg(P \vee \neg P)) \\ \text{nntnd } \text{npnp} &= \text{npnp} (\text{inj}_2 (\lambda p \rightarrow \text{npnp} (\text{inj}_1 p))) \end{aligned}$$

³This should be called the principle of the excluded middle: it states that it is impossible that a proposition is neither true nor false.

It is worthwhile to step through this proof. Ok we assume **npnp** : $\neg (P \vee \neg P)$ to derive **False**. The only option now is just to use **npnp** which means we need to show $P \vee \neg P$. There is no chance that we could prove P hence lets try $\neg P$. Ok we assume **p** : P and have to derive **False** again and again there is no choice but to use **npnp** and what dejavu we are back proving $P \vee \neg P$. But this time we actually know **p** : P hence a we have to do is to use this assumption.

Now we can just use **RAA** ($P \vee \neg P$) to derive **TND** P from **ntnd**:

$$\begin{aligned} \text{raa} \rightarrow \text{tnd} & : \text{RAA } (P \vee \neg P) \rightarrow \text{TND } P \\ \text{raa} \rightarrow \text{tnd } \text{raa} & = \text{raa } \text{ntnd} \end{aligned}$$

So if we assume that **RAA** P for all propositions P then we can also show **TND** Q for all propositions, even though it is not the same one. In this sense the two principles are equivalent.

3.4 The negative translation

We may wonder when exactly is a classical tautology provable intuitionistically, i.e. with the propositions as types interpretation, and when not. We have seen that one of the de Morgan rules $\neg (P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$ is provable intuitionistically but for the other one only one direction $\neg P \vee \neg Q \Rightarrow \neg (P \wedge Q)$ is provable but the other direction $\neg (P \wedge Q) \Rightarrow \neg P \vee \neg Q$ required classical reasoning.

The difference is evidence for an intuitionistic disjunction $P \vee Q$ contains information, namely which of the P and Q actually holds. However, in the 2nd de Morgan rule $\neg (P \wedge Q)$ merely states that it cannot be that both P and Q hold while $\neg P \vee \neg Q$ tells us which of the two actually doesn't hold and we cannot create this information from nothing.

We call propositions that contain information *positive* and the ones that do not *negative*. We will see that we can understand classical logic as the logic of negative propositions and that negative propositions are the one for which **RAA** holds.

First of all we show that any negated proposition is negative

$$\text{neg-is-neg} : \text{RAA } (\neg P)$$

which expands to showing that $\neg (\neg (\neg P)) \rightarrow \neg P$ which may be slightly surprising because in general we don't have $\neg (\neg P) \rightarrow P$ in general. But it makes now sense because both sides are negative, while in the later case we can replace P by a positive proposition.

We can show this directly, but maybe it is more instructive to do this a bit more slowly. We can show that every proposition implies its double negation:

$$\begin{aligned} \text{nn} : P & \rightarrow \neg (\neg P) \\ \text{nn } p \text{ np} & = \text{np } p \end{aligned}$$

Indeed this is just simple application. We can also show the following principle:

$$\text{neg-impl} : (P \rightarrow Q) \rightarrow \neg Q \rightarrow \neg P$$

It is always good to do a little intuitive check: If we know *If the sun shines we go to the zoo* then *If we don't go to the zoo the sun doesn't shine*. Here is the proof:

$$\text{neg-impl } f \text{ nq } p = \text{nq } (f \text{ } p)$$

Now we can show `neg-is-neg` by just combining the two results:

$$\text{neg-is-neg} = \text{neg-impl } \text{nn}$$

We can also show that conjunction and implication preserve negativity (for implication we only need that the conclusion is negative):

$$\begin{aligned} \text{and-neg} &: \text{RAA } P \rightarrow \text{RAA } Q \rightarrow \text{RAA } (P \wedge Q) \\ \text{impl-neg} &: \text{RAA } Q \rightarrow \text{RAA } (P \rightarrow Q) \end{aligned}$$

I leave this as an exercise.

The only connective that is not negative is disjunction. Classically we can show that $P \vee Q$ is equivalent to the negative $\neg(\neg P \wedge \neg Q)$ because $P \vee Q \Leftrightarrow \neg(\neg(P \vee Q))$ and using de Morgan $\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$. Hence we can define a classical disjunction:

$$\begin{aligned} _ \vee^c _ &: \text{prop} \rightarrow \text{prop} \rightarrow \text{prop} \\ P \vee^c Q &= \neg(\neg P \wedge \neg Q) \end{aligned}$$

What can we do with $_ \vee^c _$? We can actually prove the injections easily:

$$\begin{aligned} \text{inj}_1^c &: P \rightarrow P \vee^c Q \\ \text{inj}_2^c &: Q \rightarrow P \vee^c Q \end{aligned}$$

But what about $\text{case}^c : (P \rightarrow R) \rightarrow (Q \rightarrow R) \rightarrow P \vee^c Q \rightarrow R$? It turns out that we can derive it, if we assume that R is negative:

$$\text{case}^c : \text{RAA } R \rightarrow (P \rightarrow R) \rightarrow (Q \rightarrow R) \rightarrow P \vee^c Q \rightarrow R$$

Now using classical disjunction we can actually prove excluded middle:

$$\text{tnd}^c : P \vee^c \neg P$$

Having inj_1 , inj_2 , case^c and tnd^c together with all the standard principles for the remaining negative connectives we can simulate any classical proof. Any propositional variable quantifies only over negative propositions, that is when we assume $P : \text{prop}$ we need to add $\text{raap} : \text{RAA } P$. So basically the negative translation provides an alternative to the boolean explanation of classical logic. However, it has a big advantage, because as we will see it also works for predicate logic where the boolean explanation fails.

Hence classical logic can be understood as the logic of negative propositions. Hence we can summarise that *a classical logician is somebody who just can't say anything positive*.

3.5 History

Boolean algebra was invented by George Boole in the 19th century who also gave his name to the type of booleans or short `Bool`. However, the laws of boolean algebra don't play a big role in our presentation. Truth tables were already used in the 19th century first by Peirce and they were popularized by Wittgenstein and Post in the early 20th century.

Intuitionistic logic was introduced by Brouwer in the 1920ies who also had a big fight with Hilbert about the use of the excluded middle. This was called the *Grundlagenstreit* (Argument about foundations).⁴ Hilbert complained:

Taking the principle of excluded middle from the mathematician would be the same, say, as proscribing the telescope to the astronomer or to the boxer the use of his fists. To prohibit existence statements and the principle of excluded middle is tantamount to relinquishing the science of mathematics altogether.

It is fair to say that Hilbert won, at least as far as mainstream Mathematics is concerned. But don't forget this was before the invention of computers.

It was Brouwer's student Heyting who made the formal rules of intuitionistic logic precise and who also came up with a constructive semantics for it, the Brouwer-Heyting-Kolmogorov interpretation which is closely related to the propositions as types explanation but it uses untyped *realizers*.

The propositions as types explanation is also called the *Curry-Howard Isomorphism*:⁵ they observed an analogy between the rules of logic and the rules of typed λ -calculus. I prefer calling it the propositions as types explanation because where does logic come from in the first place? I think we should use the idea that propositions can be identified with the type of their evidence to justify the rules of logic. In this way the semantics comes first and the formal rules are just a reflection of the semantics. While this is not historically correct, I think this is a better way to look at this.

The negative translation which is also called the double negation translation goes back to Gödel and Gentzen. In the context of functional programming it is also known as the continuation passing style transformation (CPS transform) which provides a translation for control operators like `call-cc` (*call with current continuation*).

3.6 Exercises

1. Define the operations

$$\begin{aligned} \text{implb} &: \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \\ _ == _ &: \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool} \end{aligned}$$

⁴I recommend van Dalen's biography of Brouwer which also includes lots of details about this famous disagreement. [VD99]

⁵It was first observed by Curry in a letter in 1934 and later refined by Howard in 1969 but this was only published in 1980 [How80].

Here `implb` is the boolean version of implication. There are given by the following truth tables:

P	Q	implb P Q	P == Q
false	false	true	true
false	true	true	false
true	false	false	false
true	true	true	true

2. `if_then_else_` is a basic combinator on `Bool`. It is defined as follows

```
if_then_else_ : Bool → A → A → A
if true then y else z = y
if false then y else z = z
```

Define all the logical operations

```
_&_ : Bool → Bool → Bool
_|_ : Bool → Bool → Bool
!_ : Bool → Bool
implb : Bool → Bool → Bool
_==_ : Bool → Bool → Bool
```

in agda using only `if_then_else_` (i.e without using pattern matching).

3. Prove the following equivalences. It is helpful to use some auxilliary statements to be able to use pattern matching more easily.

All propositions in this part are provable.

```
e1 : (P → Q ∧ R) ⇔ ((P → Q) ∧ (P → R))
e2 : (P ∨ Q → R) ⇔ ((P → R) ∧ (Q → R))
```

4. We are playing *logic poker* today.

Try to prove the following propositions using Agda.

```
P1 = {P Q : prop} → (P ⇒ Q) ⇒ ¬ P ⇒ ¬ Q
P2 = {P Q : prop} → (P ⇒ Q) ⇒ ¬ Q ⇒ ¬ P
P3 = {P Q : prop} → (P ⇒ Q) ⇒ ¬ P ∨ Q
P4 = {P Q : prop} → ¬ P ∨ Q ⇒ (P ⇒ Q)
P5 = {P : prop} → ¬ (P ⇔ ¬ P)
P6 = {P : prop} → ¬ (P ∨ ¬ P)
P7 = {P : prop} → ¬ (¬ (¬ P)) ⇒ ¬ P
P8 = {P Q : prop} → ((P ⇒ Q) ⇒ P) ⇒ P
P9 = {P : prop} → (¬ (¬ P) ⇒ P) ⇒ P ∨ ¬ P
P10 = {P : prop} → P ∨ ¬ P ⇒ (¬ (¬ P) ⇒ P)
```

Consider the following cases:

- (a) You can prove it directly (intuitionistically). E.g. you have

$$Pa = \{P : \text{prop}\} \rightarrow P \rightarrow P$$

in this case you just provide a proof:

$$\begin{aligned} pa &: Pa \\ pa\ x &= x \end{aligned}$$

- (b) You cannot prove it with intuitionistic logic but you can prove it with classical logic.

E.g. if I ask you to prove

$$Pb = \{P\ Q : \text{prop}\} \rightarrow \neg(P \wedge Q) \rightarrow \neg P \vee \neg Q$$

and you realize that you cannot prove this intuitionistically but you can prove it with classical logic that is using $RAA\ P$ for some proposition $P : \text{prop}$. I define

$$CLASS = \{P : \text{prop}\} \rightarrow RAA\ P$$

and you prove

$$pb : CLASS \rightarrow Pb$$

When you prove pa you get an extra parameter raa which you can use in the proof:

$$\begin{aligned} pb\ raa\ h &= \\ &raa\ (\lambda x \rightarrow x\ (inj_1\ (\lambda p \rightarrow x\ (inj_2\ (\lambda q \rightarrow h\ (p, q)))))) \end{aligned}$$

If you need $TND\ P$ you can use $raa \rightarrow tnd$.

- (c) It isn't even true in classical logic. In this case you should find a counterexample. That is you should be able to instantiate the propositional variables with $\top$ and $\perp$ and prove the negation.

$$Pc = \{P\ Q : \text{prop}\} \rightarrow P \rightarrow Q$$

but you realize that this is false by instantiating $P = \top$ and $Q = \perp$. Ok, in this case you prove

$$\begin{aligned} pc &: Pc \rightarrow \perp \\ pc\ h &= h\ \{\top\}\ \{\perp\}\ tt \end{aligned}$$

5. Complete the missing proofs from the negative translation (section 3.4):

$$\begin{aligned} \text{and-neg} &: RAA\ P \rightarrow RAA\ Q \rightarrow RAA\ (P \wedge Q) \\ \text{impl-neg} &: RAA\ Q \rightarrow RAA\ (P \rightarrow Q) \end{aligned}$$

$\text{inj}_1^c : P \rightarrow P \vee^c Q$
 $\text{inj}_2^c : Q \rightarrow P \vee^c Q$
 $\text{case}^c : \text{RAA } R \rightarrow (P \rightarrow R) \rightarrow (Q \rightarrow R) \rightarrow P \vee^c Q \rightarrow R$
 $\text{tnd}^c : P \vee^c \neg P$

Hint: It is useful to prove that double negation is monotone:

$$(P \rightarrow Q) \rightarrow \neg(\neg P) \rightarrow \neg(\neg Q)$$

Chapter 4

Datatypes

The types we have seen so far, that is functions ($_ \rightarrow _$), products ($_ \times _, \top$) and sums ($_ + _, \perp$) are finite or preserve finiteness. In computation but also in Mathematics we often use infinite types, such as the natural numbers ($\mathbb{N}$), which I have already used for illustration. In this chapter we will look at such *datatypes*, starting with the natural numbers as the canonical example. We discuss primitive recursion first in a schematic way and then given by combinators: the recursor which can be reduced to the iterator (also called fold). We then look at various examples of inductive types such as lists, syntax trees and infinite trees representing ordinal notations. We investigate the limits of what constitutes a reasonable inductive type and what not — this is related to the notion of positivity. Using the notion of categorical mirror we also investigate the dual of inductive types: coinductive types. Examples here are streams and coinductive natural numbers, which can also be infinite.

4.1 The natural numbers

One of the most versatile datatypes we consider is the type of natural numbers, i.e. the counting numbers $0, 1, 2, 3, 4, \dots$. Indeed, because we are computer scientists we are starting counting with 0, which is useful when calculating off-sets in datastructures but it is also the answer to the question *How many elephants do you have in your fridge* (I hope).

It was Guiseppe Peano from Torino who wrote down the precise laws of the natural numbers in predicate logic at the end of the 19th century, this is nowadays known as Peano Arithmetic. We aren't going to study these laws in detail but use one important idea: according to Peano a natural number is either zero or it is the successor of another natural number. We can implement this in Agda:

```
data  $\mathbb{N}$  : Set where  
  zero :  $\mathbb{N}$   
  suc  :  $\mathbb{N} \rightarrow \mathbb{N}$ 
```

This is the definition of a sum, but the novelty is that it is recursive, that is we are using $\mathbb{N}$ which is just being defined in the argument of the constructor `suc`.

Given this we can derive the usual notation: ¹

$$\begin{array}{ll}
 0 = \text{zero} & \\
 1 = \text{suc } 0 & = \text{suc zero} \\
 2 = \text{suc } 1 & = \text{suc (suc zero)} \\
 3 = \text{suc } 2 & = \text{suc (suc (suc zero))} \\
 \vdots & \vdots
 \end{array}$$

Let's define a function on the natural numbers, for example the predecessor:

```

pred :  $\mathbb{N} \rightarrow \mathbb{N}$ 
pred zero    = zero
pred (suc n) = n

```

Here we use pattern matching to analyse the constructor as we have seen already. Actually this function is a bit of a cheat because there isn't really a predecessor of 0. A better version uses `Maybe` where `Maybe A` is just a shorthand for $\top \uplus A$:

```

data Maybe (A : Set) : Set where
  nothing : Maybe A
  just    : A  $\rightarrow$  Maybe A

```

Using `Maybe` we can now define a version of `pred` that signals an error on 0 by returning `nothing`:

```

pred :  $\mathbb{N} \rightarrow \text{Maybe } \mathbb{N}$ 
pred zero    = nothing
pred (suc n) = just n

```

This version of `pred` indicates an error on 0 by returning `nothing` but for any other number returns just the predecessor.

This predecessor is the exact inverse of the constructors which can be combined into one function:

```

zerosuc : Maybe  $\mathbb{N} \rightarrow \mathbb{N}$ 
zerosuc nothing = zero
zerosuc (just n) = suc n

```

It is not hard to see that the functions `pred` and `zerosuc` are inverse to each other.

¹Luckily we can tell Agda to use the standard notation for numbers simply by invoking a *pragma*:

```
{-# BUILTIN NATURAL  $\mathbb{N}$  #-}
```

4.1.1 Recursion

To define more interesting functions on the natural numbers we need to use recursion, for example let's define the doubling function:

```
double : ℕ → ℕ
double zero    = zero
double (suc n) = suc (suc (double n))
```

That is the double of 0 is 0 and the double of a number of the form `suc n` is `2 + double n`, which as we haven't yet defined `_+_` is just `suc (suc (double n))`.

For example `double 3 = 6`. Let's go through this step by step:

```
double 3 = double (suc (suc (suc zero)))
          = suc (suc (double (suc (suc zero))))
          = suc (suc (suc (suc (double (suc zero)))))
          = suc (suc (suc (suc (suc (suc (double zero))))))
          = suc (suc (suc (suc (suc (suc zero)))))
          = 6
```

We can justify the use of recursion: all natural numbers are constructed via `zero` and `suc`. To define a function `f` over the natural numbers we just have to give a case for `zero` and one for `suc n`. Since the number `n` has to be constructed *before* we `suc n` we can construct the answer to the function `f` in the same way and compute the answer to `f n` before we compute the answer to `f (suc n)` hence we can use `f n` when constructing `f (suc n)`.

Not all functions are exactly of this form, already the inverse of `double`, that is `half` which forgets the remainder, uses a slightly more general scheme

```
half : ℕ → ℕ
half zero          = zero
half (suc zero)    = zero
half (suc (suc n)) = suc (half n)
```

That is `half` of 0 and 1 is zero and `half` of `2 + n` is `1 + half n`. The recurrence from `suc (suc n)` can be justified in the same way as before because certainly `n` is constructed before `suc (suc n)`.

4.1.2 Arithmetic operations

Let's define the standard functions of arithmetic starting with addition:

```
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)
```

The idea of this definition of addition is that for example $3 + 5$ is the 3rd successor of 5 that $\text{suc} (\text{suc} (\text{suc } 5))$. This means that $0 + n$ is just n and $\text{suc } m + n$ is one more than $m + n$.

Next we define multiplication $_ * _$ which is repeated addition in the same way as $_ + _$ is repeated suc .

$$\begin{aligned} _ * _ &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \\ \text{zero} * n &= 0 \\ \text{suc } m * n &= n + m * n \end{aligned}$$

That is for example $3 * 5$ is $5 + 5 + 5 + 0$. Recursively we express this by saying that $0 * n$ is just 0 and $\text{suc } n * m$ is $n + (m * n)$.

We can go further and define exponentiation as repeated multiplication. In the previous cases it didn't matter over which argument we recursed because they were commutative. But exponentiation certainly isn't, e.g. $2^3 = 8$ but $3^2 = 9$. If we want to mimic the usual order of arguments for exponentiation we need to recur on the 2nd argument:

$$\begin{aligned} _ \uparrow _ &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \\ m \uparrow \text{zero} &= 1 \\ m \uparrow \text{suc } n &= m * m \uparrow n \end{aligned}$$

For example $3 \uparrow 5$ is $3 * 3 * 3 * 3 * 3 * 1$. Hence $m \uparrow 0 = 1$ and $m \uparrow \text{suc } n = m * m \uparrow n$.

4.1.3 The Ackermann function

Each instance of recursion corresponds to using a for-loop in conventional programming. While addition is just defined using one for-loop, multiplication calls addition inside its loop, hence we have a nested for-loop. And exponentiation uses a double nested for-loop. Historically, functions that only use for-loops are called primitive recursive, and all the examples we have seen are examples of primitive recursion. Hilbert, a famous german Mathematician, asked whether all total recursive functions can be defined using primitive recursion. His student Ackermann showed that this is not the case by exhibiting a function that is not primitive recursive because it grows faster than any primitive recursive function. The idea of the Ackermann function is to have one extra parameter which determines the nesting of for loops, that is he defined a function $\text{ack} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ such that

$$\begin{aligned} \text{ack } 0 \ m \ n &= \text{suc } n \\ \text{ack } 1 \ m \ n &= m + n \\ \text{ack } 2 \ m \ n &= m * n \\ \text{ack } 3 \ m \ n &= m \uparrow n \\ &\vdots \end{aligned}$$

The next function, hyperexponentiation that is repeated exponentiation doesn't really have a standard notation anymore. We can define `ack` using recursion over the natural numbers:

```

ack : ℕ → ℕ → ℕ → ℕ
ack zero      m n      = suc n
ack (suc zero) m zero   = m
ack (suc (suc zero)) m zero = 0
ack (suc (suc (suc zero))) m zero = 1
ack (suc (suc (suc (suc l)))) m zero = m
ack (suc l)    m (suc n) = ack l m (ack (suc l) m n)

```

To stay consistent with exponentiation we define the functions by recursion over the 2nd argument. The first line defines `ack 0` to be just the successor of the 2nd argument because then we can obtain addition by repeating it. The next three lines define the 0 cases for addition $m + 0 = m$, multiplication $m * 0 = 0$ and exponentiation $m \uparrow 0 = 1$. The idea for multiplication and exponentiation is to use the neutral element of the previous operation, corresponding to 0 iterations. However, since there is no neutral element for exponentiation, that is no number such that $x^n = n$, we just use the first argument as for addition. The last line uniformly expresses that the function for `ack (suc l)` is defined by iterating the previous function `ack l n` times.

Actually the definition above is a bit complicated due to all the different cases for 0, hence often a simplified version of the Ackermann function is used where the levels don't exactly correspond to the usual definitions of the arithmetic operations but have the same nesting of for loops:

```

acks : ℕ → ℕ → ℕ → ℕ
acks zero  m n      = suc n
acks (suc l) m zero   = m
acks (suc l) m (suc n) = acks l m (acks (suc l) m n)

```

Using either `ack` or `acks` we can define the function `fast`:

```

fast : ℕ → ℕ
fast n = ack n n n

```

We can only evaluate the first few instances of `fast`

```

fast 1 = 1 + 1           = 2
fast 2 = 2 * 2           = 4
fast 3 = 3 ↑ 3           = 27
fast 4 = 4 ↑ (4 ↑ (4 ↑ 4)) = ?

```

Due to the fact that our implementation of numbers using a unary representation we can't even compute `fast 4` in a reasonable time. But even using a very efficient implementation of numbers will not save us much, the incredible fast

growing behaviour of `fast` will spoil any attempt to compute many more results. And indeed, `fast` grows faster than any primitive recursive function that is any function definable only with for-loops.

4.2 Iterator and recursor

We intuitively explained and justified recursive definitions over the natural numbers but now we shall be more precise and say exactly what recursive definitions are permissible.

In general we define a function $f : \mathbb{N} \rightarrow M$ ² by recursion over the natural numbers

$$\begin{aligned} f\ 0 &= z \\ f\ (\text{suc } n) &= s\ (f\ n) \end{aligned}$$

where

$$\begin{aligned} z &: M \\ s &: M \rightarrow M \end{aligned}$$

We can turn this into a higher order function, the iterator³:

$$\begin{aligned} \text{It}\mathbb{N} &: M \rightarrow (M \rightarrow M) \rightarrow \mathbb{N} \rightarrow M \\ \text{It}\mathbb{N}\ z\ s\ \text{zero} &= z \\ \text{It}\mathbb{N}\ z\ s\ (\text{suc } n) &= s\ (\text{It}\mathbb{N}\ z\ s\ n) \end{aligned}$$

⁴ We call M the *motive* and z, s methods. Our claim is that we can reduce recursion over the natural number to the iterator. Ok, we haven't actually made precise what we mean by recursion over the natural numbers and introducing the iterator is exactly one way to do this.

We can derive `dbl` using only the iterator:

$$\begin{aligned} \text{double-it} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{double-it} &= \text{It}\mathbb{N}\ 0\ (\lambda\ \text{double-n} \rightarrow \text{suc } (\text{suc } \text{double-n})) \end{aligned}$$

The instances for the methods z and s can be read off the recursive definition of `double`:

$$\begin{aligned} \text{double} &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{double zero} &= \text{zero} \\ \text{double } (\text{suc } n) &= \text{suc } (\text{suc } (\text{double } n)) \end{aligned}$$

The reduction of `half` is less obvious - here again the recursive definition:

²In the following I use $A\ B\ M : \text{Set}$ as variables as in chapter 2.

³In category theory $\text{It}\mathbb{N}\ z\ s : \mathbb{N} \rightarrow M$ is the universal morphism from the initial algebra. It is also called a *fold* in functional programming, or a *catamorphism*.

⁴Remember that M is one of the names we have declared to be used as implicit parameters for `Set`.

```

half : ℕ → ℕ
half zero      = zero
half (suc zero) = zero
half (suc (suc n)) = suc (half n)

```

`half` doesn't follow exactly the pattern we have stated before but it is reducible to it. The idea is that we define an auxiliary function `half-aux` which computes the results of `half n` and `half (suc n)` simultaneously using a product.

```

half-aux : ℕ → ℕ × ℕ
half-aux zero = zero , zero
half-aux (suc n) = proj2 (half-aux n) , suc (proj1 (half-aux n))

```

⁵ The first line is justified by the observation that `half 0` , `half 1` = `0` , `0`. The recursive call follows from the fact that we have already `half (suc n)` as the 2nd component of `half-aux n` and by the recursive definition `half (suc (suc n))` = `half n` which explains the 2nd component. Now `half-aux` is an instance of our scheme and we can define

```

half-aux-it : ℕ → ℕ × ℕ
half-aux-it = ltℕ (zero , zero) (λ p → proj2 p , suc (proj1 p))
half-it : ℕ → ℕ
half-it n = proj1 (half-aux-it n)

```

The definition of `_+_` is straightforward, here the recursive version:

```

_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)

```

We can transform it into an instance of our scheme using higher order functions:

```

zero  +_ = λ n → n
suc m +_ = λ n → suc (m + n)

```

And hence we define

```

_+it_ : ℕ → ℕ → ℕ
_+it_ = ltℕ (λ n → n) (λ m+ n → suc (m+ n))

```

⁵The 2nd line can be more nicely written using a local definition:

```

half-aux (suc n) = proj2 hn , suc (proj1 hn)
  where hn = half-aux n

```

or even better using **with** which enables pattern matching on an intermediate result:

```

half-aux (suc n) with half-aux n
half-aux (suc n) | half-n , half-sn = half-sn , (suc half-n)

```

The same technique works for multiplication:

$$\begin{aligned} _ \text{-it} _ &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \\ _ \text{-it} _ &= \text{ltN} (\lambda n \rightarrow 0) \lambda m * n \rightarrow n + (m * n) \end{aligned}$$

A more interesting case is a function which doesn't only use the recursive result but also the input. An example is the factorial function which can be recursively defined we follows:

$$\begin{aligned} _ ! &: \mathbb{N} \rightarrow \mathbb{N} \\ \text{zero} ! &= 1 \\ \text{suc } n ! &= \text{suc } n * n ! \end{aligned}$$

$n !$ is the number of ways we can put n people on n chairs. If there are no people and no chairs we have to do nothing hence the answer is 1 (this seems to confuse some people). Otherwise in the case $(\text{suc } n) !$ there are $\text{suc } n$ ways to place the first person and then $n !$ to distribute n people on n chairs.

The factorial function is an instance of a slightly modified scheme, to define $f : \mathbb{N} \rightarrow A$ we use:

$$\begin{aligned} f 0 &= z \\ f (\text{suc } n) &= s n (f n) \end{aligned}$$

where

$$\begin{aligned} z &: A \\ s &: \mathbb{N} \rightarrow A \rightarrow A \end{aligned}$$

As above we can turn this into a general higher order function, called the recursor ⁶:

$$\begin{aligned} \text{RN} &: M \rightarrow (\mathbb{N} \rightarrow M \rightarrow M) \rightarrow \mathbb{N} \rightarrow M \\ \text{RN } z \text{ s } \text{zero} &= z \\ \text{RN } z \text{ s } (\text{suc } n) &= s n (\text{RN } z \text{ s } n) \end{aligned}$$

We can define the factorial function using only the recursor:

$$\begin{aligned} _ !\text{-r} &: \mathbb{N} \rightarrow \mathbb{N} \\ _ !\text{-r} &= \text{RN } 1 (\lambda n n! \rightarrow \text{suc } n * n!) \end{aligned}$$

We can actually derive RN from ltN by (re-)computing the input and the output together:

$$\begin{aligned} \text{RN-it} &: M \rightarrow (\mathbb{N} \rightarrow M \rightarrow M) \rightarrow \mathbb{N} \rightarrow M \\ \text{RN-it } \{M\} z \text{ s } n &= \text{proj}_2 (\text{ltN } z' \text{ s' } n) \\ \text{where} \\ z' &: \mathbb{N} \times M \end{aligned}$$

⁶Called *paramorphism* by Meertens.

$$\begin{aligned}
z' &= 0, z \\
s' : \mathbb{N} \times \mathbb{M} &\rightarrow \mathbb{N} \times \mathbb{M} \\
s'(n, m) &= \text{suc } n, s \ n \ m
\end{aligned}$$

That is we use $\mathbb{N} \times \mathbb{M}$ as the motive, the first component is identical to the input. Hence in the 0-case we use $0, z$ and in the successor case given n, m we compute $\text{suc } n, s \ n \ m$, hence we are able to supply the extra argument. In the end we have only to extract the 2nd component to obtain the result.

Finally let's have a look at the simplified Ackermann function:

$$\begin{aligned}
\text{acks} : \mathbb{N} &\rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \\
\text{acks zero } m \ n &= \text{suc } n \\
\text{acks (suc l) m zero} &= m \\
\text{acks (suc l) m (suc n)} &= \text{acks l m (acks (suc l) m n)}
\end{aligned}$$

Can we compute the Ackermann function using the iterator or the recursor? It seems that our scheme exactly captures primitive recursion. However, the traditional notion of primitive recursion doesn't include the use of higher order functions which leads to a much more powerful scheme. And indeed the Ackermann function is definable:

$$\begin{aligned}
\text{acks-it} : \mathbb{N} &\rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \\
\text{acks-it} &= \text{lt}\mathbb{N} (\lambda m \ n \rightarrow \text{suc } n) \\
&(\lambda \text{ack-l } m \rightarrow \text{lt}\mathbb{N} \ m \ \lambda \text{ack-sl-m-n} \rightarrow \text{ack-l } m \ \text{ack-sl-m-n})
\end{aligned}$$

The main idea here is to translate the pattern using higher order functions as in the case for addition and multiplication, and then use the iterator again to translate the nested pattern.

Indeed, the power of primitive recursion with higher order function is much more than ordinary primitive recursion. We can define all recursive functions that can be proven total in Peano Arithmetic. Indeed a lambda calculus with a type of natural numbers and the recursor is called *System T* and was introduced by Gödel as a functional system corresponding to Peano Arithmetic.

4.2.1 Non primitive recursive functions

There are recursive functions over the natural numbers that cannot be defined directly using primitive recursion. An example is the computation for the *greatest common divisor* using subtraction. First we define *cut-off subtraction* which stops at 0:

$$\begin{aligned}
_ - _ : \mathbb{N} &\rightarrow \mathbb{N} \rightarrow \mathbb{N} \\
\text{zero} - n &= \text{zero} \\
\text{suc } m - \text{zero} &= \text{suc } m \\
\text{suc } m - \text{suc } n &= m - n
\end{aligned}$$

That is $3 - 2 = 1$ while $1 - 2 = 0$. Another ingredient is a comparison function `comp` which gets two numbers m, n and returns different values depending on whether $m < n$, $m = n$ or $m > n$:

```

comp : ℕ → ℕ → A → A → A → A
comp zero zero m<n m=n m>n      = m=n
comp zero (suc n) m<n m=n m>n    = m<n
comp (suc m) zero m<n m=n m>n    = m>n
comp (suc m) (suc n) m<n m=n m>n = comp m n m<n m=n m>n

```

Now we can define `gcd` where we always subtract the smaller number from the larger one until they are equal:

```

gcd : ℕ → ℕ → ℕ
gcd m n = comp m n (gcd m (n - m)) m (gcd (m - n) m)

```

e.g. `gcd 56 84 = 27`.

However, Agda complains *Termination checking failed* because it cannot see that the arguments were reduced during recursion.⁷ And indeed, there is no obvious way to translate this function into one using the iterator or the recursor.

In this case we can save the day by noticing that the recursion is bounded by the sum of the numbers and define:

```

gcd-aux : ℕ → ℕ → ℕ → ℕ
gcd-aux zero m n = zero
gcd-aux (suc fuel) m n =
  comp m n (gcd-aux fuel m (n - m)) m (gcd-aux fuel (m - n) m)
gcd' : ℕ → ℕ → ℕ
gcd' m n = gcd-aux (m + n) m n

```

The first argument is often called *fuel* because it acts like the fuel for a car, we have to pay one unit for each recursive call. However, we have set up our definition so that we can never run out of fuel. Hence the first line in the definition of `gcd-aux` will never be executed and we could have put in any value instead of 0. There are better ways to deal with this situation, especially once we have dependent types.

However, there are recursive and terminating functions which we cannot define using higher order primitive recursion.

4.3 Inductive Types

4.3.1 Lists

The type of natural numbers

```

data ℕ : Set where
  zero : ℕ
  suc  : ℕ → ℕ

```

⁷We can override this warning by using the pragma

```
{-# TERMINATING #-}
```

before the declaration of the function.

of an *inductive datatype* which we can view as a recursively defined sum. Another example of an *inductive datatype* is the type of lists, representing finite sequences of a given type:

```
data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A
```

Constructors in this case are the empty list `[]` (pronounced *nil*) and given `a : A` and another list `l : List A` we can construct a new list `a :: l` using the constructor `_::_`⁸ (pronounced *cons*). All lists are made from `[]` and `_::_`, for example `1 :: 2 :: 3 :: [] : List ℕ`.

Recursive definitions on lists follow the same idea as for natural numbers. Examples are the length function:

```
length : List A → ℕ
length [] = 0
length (x :: l) = 1 + length l
```

which computes the length of a list; a function that sums the numbers in a list:

```
sumList : List ℕ → ℕ
sumList [] = 0
sumList (n :: ns) = n + sumList ns
```

and the append function which concatenates two lists:

```
_++_ : List A → List A → List A
[] ++ ys = ys
(x :: xs) ++ ys = x :: (xs ++ ys)
```

So for example

```
length (1 :: 2 :: 3 :: []) = 3
sumList (1 :: 2 :: 3 :: []) = 6
(1 :: 2 :: 3 :: []) ++ (4 :: 5 :: []) = 1 :: 2 :: 3 :: 4 :: 5 :: []
```

Append is very similar to addition, indeed addition can be viewed as a special case of append if we identify $\mathbb{N}$ with `List  $\top$` . However, unlike `_+_`, `_++_` is not commutative in general, since the order of elements matter.

Lists are a functor, that is an operation that can not only applied to elements but also to functions. This is witnessed by the `map`-function, that applies a function to every element of a list, producing a new list:

⁸In the language Haskell a single `:` is used for cons, and a double `::` for typing. Type Theoreticians put more emphasis on typing hence they prefer to have the shorter notation for this.

```

map : (A → B) → List A → List B
map f [] = []
map f (x :: xs) = f x :: map f xs

```

So for example

```
map suc (1 :: 2 :: 3 :: []) = 2 :: 3 :: 4 :: []
```

As for natural numbers recursive definitions over lists can be reduced to one combinator, which is usually called fold but which we consequently call `ltList`:

```

ltList : M → (A → M → M) → List A → M
ltList n c [] = n
ltList n c (a :: l) = c a (ltList n c l)

```

All the recursive functions over lists we have defined can be reduced to `ltList` using the ideas we have established in the previous section:

```

length-it : List A → ℕ
length-it = ltList 0 (λ a length-l → suc length-l)
sumList-it : List ℕ → ℕ
sumList-it = ltList 0 _+_
_++it_ : {A : Set} → List A → List A → List A
_++it_ = ltList (λ l → l) (λ a m++l → a :: (m++l))
map-it : {A B : Set} → (A → B) → List A → List B
map-it f = ltList [] (λ a map-f-l → f a :: map-f-l)

```

We can also derive a recursor for lists and other inductive datatypes which allows to use the input list in the recursive call but we leave the details as an exercise.

4.3.2 Syntax trees

A common application of inductive datatypes is the representation of syntax. An expression like $2 * (3 + 4)$ can be viewed as a tree generated by constructors for constants and for each of the operators:

```

data Expr : Set where
  const : ℕ → Expr
  _[_+_]_ : Expr → Expr → Expr
  _[_*_]_ : Expr → Expr → Expr

```

So our example expression can be represented as follows:

```
const 2 [_*_] (const 3 [_+] const 4) : Expr
```

An obvious function we want to define is the evaluator which computes the numeric value of an expression:

```

eval : Expr → ℕ
eval (const x) = x
eval (e1 [+] e2) = eval e1 * eval e2
eval (e1 [*] e2) = eval e1 + eval e2

```

So for example

```
eval (const 2 [*] (const 3 [+] const 4)) = 14
```

As for other inductive types we introduce an iterator for `Expr`:

```

ltExpr : (ℕ → M) → (M → M → M)
        → (M → M → M)
        → Expr → M
ltExpr c p m (const x) = c x
ltExpr c p m (e1 [+] e2) = p (ltExpr c p m e1) (ltExpr c p m e2)
ltExpr c p m (e1 [*] e2) = m (ltExpr c p m e1) (ltExpr c p m e2)

```

I hope that the general scheme for deriving iterators is now evident. The methods correspond to the constructors where every occurrence of the datatype is replaced with the motive `M`. The function which is computed by the iterator recursively replaces each constructor with the method.

We can define the evaluator using the iterator:

```

eval-it : Expr → ℕ
eval-it = ltExpr
        (λ n → n)
        (λ eval-e1 eval-e2 → eval-e1 + eval-e2)
        (λ eval-e1 eval-e2 → eval-e1 * eval-e2)

```

4.3.3 Rose trees

The syntax trees from the previous section were binary trees, the nodes created by `_[+]_` and `_[*]_` each have two subtrees. Using lists of subtrees we can define trees with an arbitrary number of subtrees which are called *rose trees*.

```

data RoseTree : Set where
  node : List RoseTree → RoseTree

```

We don't need a special constructor for leaves, because we can just have a node with no subtrees `node []`.

As an example we define the following rose tree:

```

rt : RoseTree
rt = node (node []
           :: node (node (node [] :: []) :: node [] :: []))

```

```

:: node (node [] :: [])
:: []

```

As an example for a recursive function on rose trees we count the number of nodes:

```

countNodes : RoseTree → ℕ
countNodes (node ts) = suc (countNodesList ts)
  where countNodesList : List RoseTree → ℕ
        countNodesList [] = 0
        countNodesList (t :: ts) = countNodes t + countNodesList ts

```

Here we use an auxiliary function which counts the nodes in a list of rose trees. We use **where** to do this.⁹

So for example `countNodes rt` evaluates to 8.

What is the iterator for Rose Trees? We just apply the scheme and obtain the type of methods by replacing `RoseTree` with the motive. We end up with

```

ItRoseTree : ((List M) → M) → RoseTree → M

```

But how can we recursively apply the iterator to the arguments which are inside a list. A good way to address this is to use `map`:

```

ItRoseTree m (node ts) = m (map (ItRoseTree m) ts)

```

However, Agda’s termination checker isn’t clever enough to see that recursion through `map` will terminate and we have to use the following more elaborate version where we implement an instance of `map` locally to the iterator:

```

ItRoseTree {M} m (node ts) = m (mapIt ts)
  where mapIt : List RoseTree → List M
        mapIt [] = []
        mapIt (t :: ts) = ItRoseTree m t :: mapIt ts

```

Now we can derive our count function from the iterator. The method is the function `sumList` which, as we have seen, can be defined using only the iterator for lists:

```

countNodes-it : RoseTree → ℕ
countNodes-it = ItRoseTree (λ ns → suc (sumList ns))

```

4.3.4 Ordinals

All the trees we have seen so far were *finitely branching* that is any node only had finitely many subtrees. We can also consider infinitely branching trees and a nice application of those is a notation for ordinal numbers.

⁹Agda has both **let** and **where**, but for reasons I don’t understand only **where** supports the definition of functions by pattern matching.

The idea of ordinal numbers is that we also consider infinite numbers. That is we have the sequence

$$0, 1, 2, 3, 4, \dots$$

but now we add an element on the top which we call ω :

$$0, 1, 2, 3, 4, \dots \omega$$

The idea is that ω is greater than any of the numbers in the sequence $0, 1, 2, 3, 4, \dots$ and it is the smallest such ordinal. We can continue this and create a new infinite sequence:

$$0, 1, 2, 3, 4, \dots \omega, \omega + 1, \omega + 2, \omega + 3, \omega + 4, \dots$$

And again we have an element on the top:

$$0, 1, 2, 3, 4, \dots \omega, \omega + 1, \omega + 2, \omega + 3, \omega + 4, \dots \omega + \omega$$

We can continue this game infinitely and produce yet another ordinal $\omega * \omega$ and so on. The main idea is that every ordinal number is associated with the corresponding $_<_$ relation. In each case this relation is well-founded, that is you can only go down a finite number of steps before reaching 0. Note that this also holds for ω , in the first step we can go down to any finite number and after this it is clear that we will end up at 0. However, there is no bound to the length of this sequence.

Here is the definition of our notation for ordinals. It starts like the definition for natural numbers but then we add another constructor `lim` which allows us to construct ordinals from infinite sequences:

```
data Ord : Set where
  zero : Ord
  suc  : Ord → Ord
  lim  : (ℕ → Ord) → Ord
```

We can embed the natural numbers into the ordinals:

```
emb : ℕ → Ord
emb zero = zero
emb (suc x) = suc (emb x)
```

Using `emb` we can derive ω :

```
ω : Ord
ω = lim emb
```

We can lift the arithmetic operations to ordinals to derive ordinals like $\omega + \omega$ and $\omega * \omega$. Before we define addition it is a good idea to note that ordinal addition is not commutative, that is $\alpha + \beta$ is not the same as $\beta + \alpha$. We can understand addition by just putting the ordinals next to each other. So for example $1 + \omega$ looks like this:

$$*, 0, 1, 2, 3, 4, \dots$$

Here we write $*$ for the 0 in the ordinal 1. On the other hand $\omega + 1$ looks like this

$$0, 1, 2, 3, 4, \dots *$$

In the first case we have just shifted the ordinals by 1, but the order we obtain is equivalent to the one of ω , that is in the sense of orders the ordinals $1 + \omega$ and ω are equivalent. This is not the case for $\omega + 1$ because this time there is an infinite gap between the finite elements and the new one, which is not equivalent to ω .

While when defining $_{+}$ for natural numbers it didn't matter over which argument we did the recursion for the ordinals it does. And the correct choice is to do recursion over the 2nd argument:

$$\begin{aligned} \text{m} +_{\text{ord}} \text{zero} &= \text{m} \\ \text{m} +_{\text{ord}} \text{suc } n &= \text{suc } (\text{m} +_{\text{ord}} n) \\ \text{m} +_{\text{ord}} \text{lim } f &= \text{lim } (\lambda i \rightarrow \text{m} +_{\text{ord}} f i) \end{aligned}$$

We define addition for a limit ordinal, that is an ordinal of the form $\text{lim } f$ by adding the same element to each element of the sequence and then taking the limit of the sequence. So for example $\omega + \omega$ is obtained by taking the limit of the sequence $\omega, \omega + 1, \omega + 2, \dots$ matching our intuitive explanation.

Ordinals can be used to quantify the recursive strength of a recursion scheme but also the strength of a logic. A famous example is related to the fight of Hercules against the Hydra. The Hydra is a monsters with many heads which we view as a rose tree. Hercules can chop of one head of the Hydra (that is remove a subtree of the form `node []`) but as a response the Hydra can grow an arbitrary number of heads at lower levels. However, Hercules will always win after a finite number of steps and be able to chop off all the heads.

The fact that Hercules will win is an instance of a particular ordinal called ϵ_0 which we can obtain by defining multiplication and exponentiation and then take the limit of ω -towers of the form

$$\omega, \omega^\omega, \omega^{(\omega^\omega)}, \dots$$

Any Hydra configuration can be viewed as an ordinal in this sequence and any move by Hercules chooses a smaller ordinal. However, one can show that the ordinal ϵ_0 corresponds to the logical strength of Arithmetic and hence we cannot prove that the game terminate using only induction and natural numbers.

Before we finish let's derive the iterator for `Ord`:

$$\begin{aligned} \text{ltOrd} : \{M : \text{Set}\} &\rightarrow M \rightarrow (M \rightarrow M) \rightarrow ((\mathbb{N} \rightarrow M) \rightarrow M) \\ &\rightarrow \text{Ord} \rightarrow M \\ \text{ltOrd } z \text{ s l } \text{zero} &= z \\ \text{ltOrd } z \text{ s l } (\text{suc } x) &= \text{s } (\text{ltOrd } z \text{ s l } x) \\ \text{ltOrd } z \text{ s l } (\text{lim } f) &= \text{l } (\lambda i \rightarrow \text{ltOrd } z \text{ s l } (f i)) \end{aligned}$$

and implement ordinal addition as an instance:

```

_+ord-it_ : Ord → Ord → Ord
α+ord-it β = ltOrd α suc lim β

```

Note that we explicitly do recursion over the 2nd argument.

4.4 Positivity

Introductions to λ -calculus often start with the untyped λ -calculus which presents the syntax of the λ -calculus without types. We can try to simulate this idea by defining a strange datatype Λ :

```

data  $\Lambda$  : Set where
  lam : ( $\Lambda \rightarrow \Lambda$ )  $\rightarrow$   $\Lambda$ 

```

Agda won't accept this definition (for good reasons) but for the moment we can switch this off by saying `NO_POSITIVITY_CHECK`. The idea is that any function $\Lambda \rightarrow \Lambda$ gives rise to another untyped λ -term which corresponds to its λ -abstraction. The simplest example is the identity function $\lambda x \rightarrow x$ which is represented as `lam ( $\lambda x \rightarrow x$ ) :  $\Lambda$` . To be able to form more interesting terms we need to use application. However, this is easy to define using pattern matching:

```

app :  $\Lambda \rightarrow \Lambda \rightarrow \Lambda$ 
app (lam f) x = f x

```

Since every untyped λ -term is produced by an abstraction, we can just use this to obtain a function and hence we have application. We can obtain some interesting terms such as self-apply which is

```

 $\lambda x \rightarrow x x$ 

```

Hence using Λ we define:

```

self-apply :  $\Lambda$ 
self-apply = lam ( $\lambda x \rightarrow$  app x x)

```

What happens if we apply `self-apply` to itself? This term is called Ω :

```

 $\Omega = (\lambda x \rightarrow x x) (\lambda x \rightarrow x x)$ 

```

we translate this to

```

 $\Omega$  :  $\Lambda$ 
 $\Omega$  = app self-apply self-apply

```

Now the strange thing about Ω is that when we β -reduce it we get back to Ω :

$$\begin{aligned}
 \Omega &= (\lambda x \rightarrow x x) (\lambda x \rightarrow x x) \\
 &= (x x) [x \mapsto (\lambda x \rightarrow x x)] \\
 &= (\lambda x \rightarrow x x) (\lambda x \rightarrow x x) \\
 &= \Omega
 \end{aligned}$$

This means that we have constructed an element of Λ which is not generated by `lam`. And indeed if we try to evaluate Ω Agda will hang.

It is the negative occurrence of the type we are defining which causes this. Another example of a type which only has a negative occurrence is the following (and again we have to switch off the positivity checker).

```
data Weird1 : Set where
  foo : (Weird1 → ⊥) → Weird1
```

It is hard to explain `Weird1` but that is the point. We can show that `Weird1` is empty using pattern matching. The idea is that if there is an element of `Weird1` then we have a function `Weird1 → ⊥` and hence we can derive $\perp$:

```
¬weird1 : Weird1 → ⊥
¬weird1 (foo x) = x (foo x)
```

However, now using `¬weird1` we can actually construct an element of `Weird1`, namely `foo ¬weird1`. But since we have shown that `Weird1` is empty we can derive an element of the empty type:

```
bad : ⊥
bad = ¬weird1 (foo ¬weird1)
```

This is certainly bad, because the empty type was supposed to be empty.

These examples illustrate that we need to have some restrictions on the types of constructors when defining an inductive datatype. The question to answer first is what do we actually mean by an inductive datatype? All the elements of an inductive datatype are generated from the constructors. In the case that an argument of the constructor is the type we are defining means that this should be an element which we know already. That is we create the elements in stages, e.g. with the natural numbers in the first step we can only create `zero` : $\mathbb{N}$, or for the expressions we can only create the constants `const 1` : `Expr`. But in the next step we can create `suc n` : $\mathbb{N}$ or in the case of `Expr` we can create for example `const 1` `[+]` `const 2` and so on. This also justifies the recursion principles expressed via the iterators. This line of thought explains inductive datatypes where the type appears as an argument to a constructor.

This also works for rose trees but here we have to create a whole list of trees before we can go on creating a new tree.

An interesting example is

```
data T1 : Set where
  foo : T1 → T1
```

What are the elements of this type? Since we have no leaves we can't even get started to create elements. Hence the type is empty. We can actually show this by defining a function into the empty type by recursion:

```
¬t1 : T1 → ⊥
¬t1 (foo x) = ¬t1 x
```

Understanding infinitely branching trees like `Ord` is a bit more involved. We first create all the trees we can build in finitely many stages. These include some of the `lim`-trees but only the one which only use the trees we have constructed before, hence which have only a finite number of different subtrees. Once we have created all of them we have reached a stage ω . Now we can continue to create trees using all the infinitely many we have created up to ω . And so on, we can repeat this process to get more and more trees.

Hence we can allow constructor arguments which are of the form $A \rightarrow X$ where X is the type we are defining just now and A is a type we already know. This is called a *strictly positive type* and this is the condition Agda is using to decide whether a datatype declaration is acceptable.

But what about the following type?

```
data Weird2 : Set where
  foo : ((Weird2 → Bool) → Bool) → Weird2
```

In this example the occurrence of `Weird2` in the argument is not strictly positive but still positive, i.e. double negative.

Should we allow such types? Agda doesn't think so, but why? First of all our semantic explanation is out of the window, we cannot generate `Weird2` in stages. But is it inconsistent? This doesn't seem to be the case even though the proof needs quite strong logical principles. However, this type is inconsistent with classical logic.

If we assume TND or RAA then the type `Bool` is the type of propositions and `Weird2` constructs a fixpoint of the double power set and we can use well known constructions (Cantor's diagonalisation) to derive a contradiction.

Hence, while positive datatypes on their own seem to be consistent, they rule out certain assumptions like the assumption of classical logic. Moreover they are not justified semantically. These seem to be good reasons to rule them out.

4.5 Coinductive types

One of the nice features of category theory is the availability of a mirror: for any construction X there is a dual one, which is usually called $\text{co-}X$. For example from the point of category products are dual to sums and indeed sums are sometimes called coproducts. We have seen that recursive sums aka inductive types are very useful, hence now we wonder what can we do with recursive products or coinductive types.

4.5.1 Streams

An example of a coinductive type is the type of streams, or infinite sequences. Given a stream over A we can obtain its `head` which is an element of A and its `tail` which is another stream. Hence we define:

```

record Stream (A : Set) : Set where
  coinductive
  field
    head : A
    tail : Stream A

```

Unlike in the case for sums we have to indicate that we want to define a recursive record by invoking the keyword `coinductive`.¹⁰

To construct a stream we just have to present a head and a tail. We can derive a constructor for streams using copattern matching which we have already seen for products:

```

_::_ : A → Stream A → Stream A
head (a :: as) = a
tail (a :: as) = as

```

Indeed we can automatically derive the constructor by saying:

```

record Stream (A : Set) : Set where
  constructor _::_
  coinductive
  field
    head : A
    tail : Stream A

```

We can use `copatternmatching` which we have already seen for products to define the function

```

from : ℕ → Stream ℕ

```

which for any number `n` returns the stream of numbers starting with `n` that is `n, n+1, n+2, ...`.

```

head (from n) = n
tail (from n) = from (suc n)

```

Clearly the `head` of `from n` is just `n`, while the `tail` is the stream starting with the next number, hence `from (suc n)`.

While we understood inductive types by construction, that is all elements are constructed using the constructors, we understand coinductive types by the way they are destructed. E.g. a stream is anything of which we can take a head and a tail.

The recursive definition of `from n` is reasonable, because we can always compute the `head` and the `tail` of the result of `from n`, we never get stuck.

Clearly we cannot define `from` for lists

¹⁰The reason is that Agda uses the η -law for records which is undecidable for recursive records. Since agda doesn't implement the η -law for sums there is no need to flag recursive sums.

```

fromList : ℕ → List ℕ
fromList n = n :: fromList (suc n)

```

because the recursive definition is not justified by descending along an inductively defined type eg $\mathbb{N}$.

On the other hand while we can define a function that filters out elements from a list given a boolean function:

```

filterL : {A : Set} → (A → Bool) → List A → List A
filterL f [] = []
filterL f (x :: xs) with f x
filterL f (x :: xs) | false = filterL f xs
filterL f (x :: xs) | true  = x :: filterL f xs

```

However, Agda will complain if we try to do the same for streams:

```

filterS : {A : Set} → (A → Bool) → Stream A → Stream A
filterS f xs with f (head xs)
filterS f xs | false = filterS f (tail xs)
filterS f xs | true  = (head xs) :: filterS f (tail xs)

```

Indeed, we cannot answer the question what is the head of `filterS f xs`. And indeed there may be none, we can define a stream of `false`:

```

falses : Stream Bool
head falses = false
tail falses  = falses

```

but what is `head (filterS ( $\lambda x \rightarrow x$ ) falses) : Bool?`

4.5.2 Conatural numbers

Actually streams are not the dual of lists because they do not allow empty streams. We can define a type of colists which contain both finite and infinite sequences but instead we are going to study the dual of natural numbers, the conatural numbers. As we have observed previously the destructor for natural numbers is `pred : ℕ → Maybe ℕ`. Hence we define conatural numbers as objects that have a predecessor:

```

record ℕ∞ : Set where
  coinductive
  field
    pred∞ : Maybe ℕ∞

```

As we have derived the predecessor for the natural numbers we can now derive `0` and `suc` for conatural numbers:

```

zero∞ : ℕ∞
pred∞ zero∞ = nothing

```

```

suc $\infty$  :  $\mathbb{N}\infty \rightarrow \mathbb{N}\infty$ 
pred $\infty$  (suc $\infty$  n) = just n

```

The rationale behind these definitions should be clear: `zero $\infty$` is the conatural number on which `pred $\infty$` returns an error, while the successor of a conatural number is the one whose predecessor is `just` that number.

Apart from those finite numbers we can also create an infinite number:

```

 $\infty$  :  $\mathbb{N}\infty$ 
pred $\infty$   $\infty$  = just  $\infty$ 

```

The definition is straightforward: what is the predecessor of ∞ ? It is certainly not 0 but it is `just  $\infty$` .

Let's define addition for conatural numbers. It is a good idea to have the definition of addition of natural numbers in mind:

```

_+_ :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
zero + n = n
suc m + n = suc (m + n)

```

In contrast to define `_+ $\infty$ _` we have to determine what is the predecessor of `m + $\infty$  n`?

```

_+ $\infty$ _ :  $\mathbb{N}\infty \rightarrow \mathbb{N}\infty \rightarrow \mathbb{N}\infty$ 
pred $\infty$  (m + $\infty$  n) with pred $\infty$  m
pred $\infty$  (m + $\infty$  n) | nothing = pred $\infty$  n
pred $\infty$  (m + $\infty$  n) | just m' = just (m' + $\infty$  n)

```

To find this out we query the first argument. If it was 0, that is `pred $\infty$  m` returns `nothing` then it is the predecessor of the 2nd argument. However, if the predecessor is `just m'` then the result is `just (m' + $\infty$  n)`.

`_+ $\infty$ _` is the infinite extension of `_+_`. To make this statement a bit more precise, we can define an embedding

```

 $\mathbb{N} \rightarrow \mathbb{N}\infty$  :  $\mathbb{N} \rightarrow \mathbb{N}\infty$ 
 $\mathbb{N} \rightarrow \mathbb{N}\infty$  zero = zero $\infty$ 
 $\mathbb{N} \rightarrow \mathbb{N}\infty$  (suc n) = suc $\infty$  ( $\mathbb{N} \rightarrow \mathbb{N}\infty$  n)

```

just using recursion. Now the defining property of `_+ $\infty$ _` wrt `_+_` is

```

 $\mathbb{N} \rightarrow \mathbb{N}\infty$  (m + n)  $\equiv$  ( $\mathbb{N} \rightarrow \infty$  m) + $\infty$  ( $\mathbb{N} \rightarrow \infty$  n)

```

We haven't yet developed the machinery to prove this and we haven't explained when two coinductive numbers are equal.

4.5.3 Coiterator and corecursor

As for recursion over an inductive type we can derive a general combinator to do corecursion over a coinductive type. To define a function `f : W  $\rightarrow$  Stream A` corecursively we are using copattern patching:

```

head (f x) = h x
tail (f x) = f (t x)

```

where

```

h : A → W
t : W → W

```

This scheme of *coiteration* can be expressed as one higher order function:

```

ColtStream : {A W : Set} → (W → A) → (W → W)
              → W → Stream A
head (ColtStream h t x) = h x
tail (ColtStream h t x) = ColtStream h t (t x)

```

As an example the from function can be defined using only the coiterator:

```

from-coit : ℕ → Stream ℕ
from-coit = ColtStream (λ n → n) (λ n → suc n)

```

Similarly in the case of the conatural numbers we can define:

```

Coltℕ∞ : {W : Set} → (W → Maybe W) → W → ℕ∞
pred∞ (Coltℕ∞ p x) with p x
pred∞ (Coltℕ∞ p x) | nothing = nothing
pred∞ (Coltℕ∞ p x) | just x' = just (Coltℕ∞ p x')

```

As an example we can derive

```

∞-coit : ℕ∞
∞-coit = Coltℕ∞ just tt

```

However, it turns deriving `_+∞_` is a bit more difficult. Here is the code we have written using `copatternmatching`:

```

_+∞_ : ℕ∞ → ℕ∞ → ℕ∞
pred∞ (m +∞ n) with pred∞ m
pred∞ (m +∞ n) | nothing = pred∞ n
pred∞ (m +∞ n) | just m' = just (m' +∞ n)

```

We clearly do corecursion on the first argument. However, if the first argument is 0, that is of `pred∞ m` is `nothing` we immediately return `pred∞ n`. This sort of behaviour cannot be directly achieved with the coiterator, but requires a *corecursion*. The corecursor can stop corecursion and just return an answer, hence the method returns a sum:

```

CoRℕ∞ : {W : Set} → (W → ℕ∞ ⊔ Maybe W) → W → ℕ∞
pred∞ (CoRℕ∞ p x) with p x
pred∞ (CoRℕ∞ p x) | inj1 n = pred∞ n

```

```

pred $\infty$  (CoRN $\infty$  p x) | inj2 nothing = nothing
pred $\infty$  (CoRN $\infty$  p x) | inj2 (just x') = just (CoRN $\infty$  p x')

```

Using the corecursor we can define $_+\infty_$. We are also using a case combinator for Maybe

```

caseMaybe : {A M : Set} → M → (A → M) → Maybe A → M
caseMaybe n j nothing = n
caseMaybe n j (just x) = j x

```

```

 $\_+\infty\text{-corec}\_ : \mathbb{N}\infty \rightarrow \mathbb{N}\infty \rightarrow \mathbb{N}\infty$ 
m  $\_+\infty\text{-corec}$  n =
  CoRN $\infty$  ( $\lambda$  x → caseMaybe (inj1 n) ( $\lambda$  m' → inj2 (just m')) (pred $\infty$  x)) m

```

The corecursor is dual to the recursor, which can be also formulated using a product but this is curried in the usual formulation. And as for the recursor, the corecursor can be derived from the coiterator.

4.6 History

Guiseppe Peano lived in Turino in the late 19th century. He wrote down the rules of reasoning for the natural numbers which is now in his honour called *Peano Arithmetic*. A proper account of Peano Arithmetic uses predicate logic which we haven't yet introduced. Peano only used addition and multiplication in his presentation but it can be shown that all primitive recursive functions can be derived. Interestingly, this ceases to be the case if you leave out multiplication which leads to something called Pressburger Arithmetic.

By primitive recursive functions one means traditionally functions which can be derived using no function types, that is with $\mathbb{RN}$ but only products and $\mathbb{N}$ can be used. This intuitively corresponds to for-loops in programming. Ackermann formulated his function to show that there are computable functions which cannot be defined with primitive recursion.

It was Gödel who formulated a system which can express all functions that are definable in Peano arithmetic. This is called System T¹¹ and it is basically what we have presented, i.e. you have $\mathbb{RN}$ and function types. There are still functions which are not definable in this system, corresponding to the problem of Archilles and the hydra which we have mentioned already in the section on ordinals.

Our presentation of inductive types goes back to algebraic datatypes and pattern matching invented by my PhD supervisor Rod Burstall which were first implemented in a language called Hope [BMS80]. Subsequently algebraic datatypes and pattern matching was used in many functional programming languages including Miranda and Haskell. Coinductive types were investigated by Coquand [Coq93] who also formulated the guarded corecursion principle we

¹¹Gödel introduced System T in [Göd58] (in german).

are using. Copatternmatching was introduced much later in the context of Type Theory by Andreas Abel and Brigitte Pientka [AP13].

We only briefly touched upon the untyped λ -calculus which, together with untyped combinatory logic, is a basic computational mechanism indeed an alternative to Turing machines. A good overview over the untyped λ -calculus is Barendregt's book [B+84].

4.7 Exercises

1. Define the following functions using pattern matching and structural recursion on the natural numbers.

- (a) Define a function `even?` that determines whether its input is even.

`even? :  $\mathbb{N} \rightarrow \text{Bool}$`

Examples :

<code>even? 3</code>	<code>= false</code>
<code>even? 6</code>	<code>= true</code>

- (b) Define a function `sum` that sums the numbers from 0 until `n-1`.

`sum :  $\mathbb{N} \rightarrow \mathbb{N}$`

Examples:

<code>sum 2</code>	<code>= 3</code>
<code>sum 10</code>	<code>= 55</code>

- (c) Define a function `max` that calculates the maximum of 2 numbers.

`max :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$`

Examples:

<code>max 2 3</code>	<code>= 3</code>
<code>max 10 1</code>	<code>= 10</code>

- (d) Define a function `fib` which calculates the n th item of the Fibonacci sequence: 1, 1, 2, 3, 5, 8, 13, ... (each number is the sum of the two previous ones).

`fib :  $\mathbb{N} \rightarrow \mathbb{N}$`

Examples:

<code>fib 0</code>	<code>= 1</code>
<code>fib 6</code>	<code>= 13</code>

- (e) Define a function `eq` that determines whether two numbers are equal.

$$\text{eq?} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool}$$

Examples:

$$\begin{array}{ll} \text{eq? } 4 \ 3 & = \text{false} \\ \text{eq? } 6 \ 6 & = \text{true} \end{array}$$

- (f) Define a function `rem` such that `rem m n` returns the remainder when dividing `m` by `suc n` (this way we avoid division by 0).

$$\text{rem} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$$

Examples:

$$\begin{array}{ll} \text{rem } 5 \ 1 & = 1 \\ \text{rem } 11 \ 2 & = 2 \end{array}$$

- (g) Define a function `div` such that `div m n` returns the result of dividing `m` by `suc n` (ignoring the remainder).

$$\text{div} : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$$

Examples:

$$\begin{array}{ll} \text{div } 5 \ 1 & = 2 \\ \text{div } 11 \ 2 & = 3 \end{array}$$

2. Define all the functions of the previous question but this time only use the iterator `ltN` and/or the recursor `RN`.

Use the following naming convention: if the function was called `f` then call it `f-i` if you only use the iterator, but `f-r` if you use the recursor (and possibly the iterator).

3. In this exercise we implement `tree-sort` to sort a list of natural numbers.

We define the type of trees with labels on the nodes.

```
data Tree (A : Set) : Set where
  leaf : Tree A
  node : Tree A → A → Tree A → Tree A
```

Here is an example tree:

```
node (node leaf 1 (node leaf 2 leaf)) 5 leaf
```

Define a function `tree2list` which collects all the leaves from left to right in a list.

$$\text{tree2list} : \{A : \text{Set}\} \rightarrow \text{Tree } A \rightarrow \text{List } A$$

Here is a simple unit test

$$\text{tree2list } t = 1 :: 2 :: 5 :: []$$

We are going to produce sorted trees. A sorted tree is one where for every node the leaves on the left have smaller values and the leaves on the right have larger values.

I give you a comparison function on $\mathbb{N}$.

$$\begin{aligned} _ \leq? _ &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Bool} \\ \text{zero} \leq? m &= \text{true} \\ \text{suc } n \leq? \text{zero} &= \text{false} \\ \text{suc } n \leq? \text{suc } m &= n \leq? m \end{aligned}$$

Write a function that transforms a list of natural number into a sorted tree.

$$\text{list2tree} : \text{List } \mathbb{N} \rightarrow \text{Tree } \mathbb{N}$$

Here is a test:

$$\begin{aligned} \text{list2tree } (10 :: 2 :: 1 :: 5 :: []) \\ = \text{node } (\text{node leaf } 1 \text{ (node leaf } 2 \text{ leaf)}) 5 \text{ (node leaf } 10 \text{ leaf)} \end{aligned}$$

Hint: it may be a good idea to write a function

$$\text{insert} : \mathbb{N} \rightarrow \text{Tree } \mathbb{N} \rightarrow \text{Tree } \mathbb{N}$$

that inserts one value into a sorted tree maintaining its sortedness.

Using the previous function it is now easy to define `tree-sort` a function that sorts lists.

$$\text{tree-sort} : \text{List } \mathbb{N} \rightarrow \text{List } \mathbb{N}$$

Here is a unit test for `treesort`.

$$\text{tree-sort } (10 :: 2 :: 1 :: 5 :: []) = 1 :: 2 :: 5 :: 10 :: []$$

4. Reimplement `tree-sort` using only the iterators for lists and trees (also called folds).

The functions should **not** use pattern matching on lists or trees or any recursion but only use the combinators defined here:

```

foldList : {A : Set} {M : Set} →
  M → (A → M → M) → List A → M
foldList mnil mcons [] = mnil
foldList mnil mcons (x :: xs) = mcons x (foldList mnil mcons xs)

foldTree : {A : Set} {M : Set} →
  M → (M → A → M → M) → Tree A → M
foldTree mleaf mnode leaf = mleaf
foldTree mleaf mnode (node l x r) =
  mnode (foldTree mleaf mnode l) x (foldTree mleaf mnode r)

```

If the original function is called x then the function with folds only is called $x\text{-}f$.

5. We have already defined addition for conatural numbers:

```

_+∞_ : ℕ∞ → ℕ∞ → ℕ∞
prd∞ (m +∞ n) with prd∞ m
prd∞ (m +∞ n) | just m' = just (m' +∞ n)
prd∞ (m +∞ n) | nothing = prd∞ n

```

Our goal is now to define multiplication:

```

_*∞_ : ℕ∞ → ℕ∞ → ℕ∞

```

This is not so easy because we need to make sure that the recursion is syntactically guarded, i.e. the recursive call has to appear directly at the right hand side not inside another function.

Hint: Define an auxilliary function first which solves a more general problem.

Now here is a simple unit test:

$$\mathbb{N}_{\infty} \rightarrow \mathbb{N} \ (\mathbb{N} \rightarrow \mathbb{N}_{\infty} \ 3 \ *_{\infty} \ \mathbb{N} \rightarrow \mathbb{N}_{\infty} \ 5) = 15$$

6. Define the map operations for all the examples of functors:

```

T-List-map : (A → B) → T-List C A → T-List C B
T-Expr-map : (A → B) → T-Expr A → T-Expr B
T-RoseTree-map : (A → B) → T-RoseTree A → T-RoseTree B
T-Ord-map : (A → B) → T-Ord A → T-Ord B

```

Chapter 5

Dependent types

We are now getting to one of the main innovations of Type Theory: dependent types. In general a dependent type is a function with codomain `Set`. Examples are vectors $\text{Vec } A : \mathbb{N} \rightarrow \text{Set}$ where $\text{Vec } A \ n$ is a sequence elements of A of length n , and $\text{Fin} : \mathbb{N} \rightarrow \text{Set}$ where $\text{Fin } n$ corresponds to numbers less than n .

Actually we have already seen dependent types. For example $\text{List} : \text{Set} \rightarrow \text{Set}$ is a dependent type indexed by `Set`. Hence *Polymorphism* is actually subsumed by dependent types! Dependent types can be indexed by several indices, and indeed $\text{Vec} : \text{Set} \rightarrow \mathbb{N} \rightarrow \text{Set}$ if we spell out all the dependencies.

We will discuss Π -types which generalize function types and Σ -types which generalize products. There are a number of interesting relations between dependent type formers and simply typed ones and we will also extend the *type arithmetic* from 2.5.4 to dependent types.

5.1 Vectors and dependent functions

We are going to look at some examples for dependent functions using vectors. I will postpone the actual definition of vectors until I have explained some syntactic conventions. $\text{Vec } A \ n$ is the type of sequences of elements of A of length n , it is a refinement of the type of lists and we use the same constructors. For example $1 :: 2 :: 3 :: [] : \text{Vec } \mathbb{N} \ 3$.

We can define a function that produces a list of 0s of length n a list:

```
zeroes-l :  $\mathbb{N} \rightarrow \text{List } \mathbb{N}$ 
zeroes-l zero = []
zeroes-l (suc n) = 0 :: zeroes-l n
```

E.g. $\text{zeroes-l } 3 = 0 :: 0 :: 0 :: []$. Using vectors we can define an analogous function with a more precise type because we know that $\text{zeroes-l } n$ will produce a list of length n . So instead we output a vector of the length given by the input.

```
zeroes-v :  $(n : \mathbb{N}) \rightarrow \text{Vec } \mathbb{N} \ n$ 
```

```

zeroes-v zero    = []
zeroes-v (suc n) = 0 :: (zeroes-v n)

```

We see that now the function type is *dependent* and the function type constructor actually introduces a variable.

In this example we can also see that we can often turn non-dependent functions into dependent ones with a more precise type. Let's look at another example: `append`. We have already seen the `append` function on lists which was defined infix but for the purpose of the current discussion I am going to redefine it in prefix form:

```

append-l : List A → List A → List A
append-l [] ys = ys
append-l (x :: xs) ys = x :: (append-l xs ys)

```

We can define a very similar operation on vectors, which we call `append-v` which appends two vectors and records in its type that the length of the result is the sum of its inputs:

```

append-v : (i j : ℕ) → Vec A i → Vec A j → Vec A (i + j)
append-v zero j [] w = w
append-v (suc i) j (x :: v) w = x :: (append-v i j v w)

```

Here the type $(i\ j : \mathbb{N}) \rightarrow \dots$ is just a shorthand for $(i : \mathbb{N}) \rightarrow (j : \mathbb{N}) \rightarrow \dots$. Let's define some vectors

```

v1 : Vec ℕ 2
v1 = 1 :: 2 :: []
v2 : Vec ℕ 3
v2 = 3 :: 4 :: 5 :: []

```

and append them using `append-v`:

```
append-v 2 3 v1 v2 : Vec ℕ 5
```

using that $2 + 3 = 5$. And this term will evaluate to

```
1 :: 2 :: 3 :: 4 :: 5 :: [] : Vec ℕ 5
```

Already at this point we note that always having to supply the indices for an operation like `append-v` will clutter up the code and potentially make it unreadable. However, it is also completely unnecessary because we can deduce them from the subsequent arguments.

To avoid this clutter dependently typed languages have introduced the notion of *implicit arguments* which are arguments which are filled in automatically by the type checker. To make an argument implicit we replace $(x : A) \rightarrow \dots$ by $\{x : A\} \rightarrow \dots$. Hence we can write

```
append-v : {i j : ℕ} → Vec A i → Vec A j → Vec A (i + j)
```

and when applying `append-v` we just write

```
append-v v1 v2 : Vec ℕ 5
```

and the type checker will insert the hidden arguments 2 and 3 which it can infer from the later arguments. We can also make implicit arguments explicit again by using curly brackets again:

```
append-v {2} {3} v1 v2 : Vec ℕ 5
```

Actually using implicit parameters we can also switch to the more readable infix definition of `append` for vectors resembling our original definition for lists:

```
_++v_ : {i j : ℕ} → Vec A i → Vec A j → Vec A (i + j)
[]      ++v w = w
(x :: v) ++v w = x :: (v ++v w)
```

We have already seen implicit arguments when looking at polymorphic functions like `length : List A → ℕ` which exploited our convention that `A` is a set parameter. Agda will expand this to

```
length : {A : Set} → List A → ℕ
```

and in an application like `length (1 :: 2 :: 3 :: [])` the implicit parameter `ℕ` is inserted, i.e. this expression *elaborates* to `length {ℕ} (1 :: 2 :: 3 :: [])`.

We see that not only are datatypes like `List : Set → Set` are actually dependent types but also the mystery of polymorphic functions can be subsumed by dependent function types without much notational overhead.

And indeed the constructor `_::_` for lists already has an implicit parameter namely:

```
_::_ : {A : Set} → A → List A → List A
```

and hiding it enables us to use the customary infix notation.

After having explained this syntactic convention, I can reveal the actual definition of vectors:

```
data Vec (A : Set) : ℕ → Set where
  [] : Vec A 0
  _::_ : {n : ℕ} → A → Vec A n → Vec A (suc n)
```

This is an inductive definition, very similar to the inductive definition of lists. As before `A : Set` is a parameter but we define not a `Set` but a dependent type `ℕ → Set`. The constructors create elements for different instances of `Vec A`.

We can also see that the overloaded ¹ `_::_` for vectors has actually two implicit arguments:

¹Agda allows the overloading of constructors since it can determine which actual constructor is meant from the type.

$$_::_ : \{A : \text{Set}\} \{n : \mathbb{N}\} \rightarrow A \rightarrow \text{Vec } A \ n \rightarrow \text{Vec } A \ (\text{suc } n)$$

You may notice that the dependency on `Set` and on the $\mathbb{N}$ in the declaration of `Vec` looks different. This is partly convenience since `A` is the same in the whole declaration but in this case it also has an impact on keeping `Vec A n` in `Set` but this is something which we will discuss later in more detail.

5.2 The family of finite sets

Using dependent types we can avoid run-time errors such as the typical *index out of range error* we get when accessing an array. As a simple example lets look at the indexing operation for lists. We would like to implement an operation:

$$_!!_ : \text{List } A \rightarrow \mathbb{N} \rightarrow A$$

but while we can fill in the cases where the list is a cons we get stuck on the case where it is empty:

$$\begin{aligned} _!!_ &: \text{List } A \rightarrow \mathbb{N} \rightarrow A \\ [] !! n &= ? \\ (x :: xs) !! \text{zero} &= x \\ (x :: xs) !! \text{suc } n &= xs !! n \end{aligned}$$

Indeed there is no way to complete this program since `A` could be empty.

We can handle the error explicitly by using `Maybe`:

$$\begin{aligned} _!!_ &: \{A : \text{Set}\} \rightarrow \text{List } A \rightarrow \mathbb{N} \rightarrow \text{Maybe } A \\ [] !! n &= \text{nothing} \\ (x :: as) !! \text{zero} &= \text{just } x \\ (x :: as) !! \text{suc } n &= as !! n \end{aligned}$$

This is ok and we can use the fact that `Maybe` is a monad to write programs with error handling. However, we still may get an error anywhere in the middle of a computation.

Wouldn't it better, if we could show statically that no such error can occur? And indeed we can use vectors for this purpose. The other ingredient is `Fin` which provides us with the allowed index-range of a vector. As `Vec` was a refinement of `List`, `Fin` is a refinement of $\mathbb{N}$:

```
data Fin :  $\mathbb{N} \rightarrow \text{Set}$  where
  zero :  $\{n : \mathbb{N}\} \rightarrow \text{Fin } (\text{suc } n)$ 
  suc  :  $\{n : \mathbb{N}\} \rightarrow \text{Fin } n \rightarrow \text{Fin } (\text{suc } n)$ 
```

The idea is that every non-empty finite type has a `zero` and that the elements of `Fin n` get embedded into `Fin (suc n)` using `suc` for `Fin`. That is the first levels

of `Fin` look like this

```

Fin 0 = {}
Fin 1 = {zero {0}}
Fin 2 = {zero {1}, suc {1} (zero {0})}
Fin 3 = {zero {2}, suc {2} (zero {1}), suc {2} (suc {1} (zero {0}))}
      ⋮

```

Here it is actually helpful to use the convention to make the implicit arguments explicit.

Now we can use `Vec` and `Fin` to write a safe version of the lookup function:

```

_!!v_ : {A : Set} {n : ℕ} → Vec A n → Fin n → A
(x :: v) !!v zero = x
(x :: v) !!v suc i = v !!v i

```

Using `_!!v_` we can access

```
v1 !!v (suc zero) = 2
```

but

```
v1 !!v (suc (suc (suc zero)))
```

isn't well-typed.

5.3 Π -types and Σ -types

In the previous section we have encountered dependent function types which are also called Π -types.² In general we have a type $A : \text{Set}$ and a dependent type $B : A \rightarrow \text{Set}$ and we can form $(x : A) \rightarrow B x : \text{Set}$. We can understand the simple function type $A \rightarrow B$ as a special case where the 2nd type doesn't actually depend on the first and hence we omit the binding.

$$A \rightarrow B = (_ : A) \rightarrow B$$

In all other aspects dependent function types behave very much like simple function types. The application rule now reads that given $f : (x : A) \rightarrow B$ and $a : A$ we obtain $f a : B a$. We can define dependent function either by an explicit definition or using λ -abstraction.

Previously, after introducing `_→_` we defined products `_×_`. And indeed as Π types generalize $\rightarrow$ we also have a dependent generalisation of $\times$, the Σ -type. As for `_×_` this is just an instance of a record type:

²I will explain later in this chapter where the Π comes from.

```

record  $\Sigma$  (A : Set) (B : A  $\rightarrow$  Set) : Set where
  constructor _,_
  field
    proj1 : A
    proj2 : B proj1

```

Elements of $\Sigma A B$ are pairs a, b where the 1st component is $a : A$ and the 2nd component is $b : B a$, that is the type of the 2nd component depends on the 1st. Correspondingly we have the projections

```

proj1 :  $\Sigma A B \rightarrow A$ 
proj2 : (a :  $\Sigma A B$ )  $\rightarrow B$  (proj1 a)

```

Note that the type of the second projection is a dependent function type. To improve readability I am introducing a special syntax for Σ -types and write $\Sigma[x \in A] P$ for $\Sigma A (\lambda x \rightarrow P)$.³

A simple example would be a type of flexible vectors:

```

FlexVec : Set  $\rightarrow$  Set
FlexVec A =  $\Sigma[n \in \mathbb{N}] \text{Vec } A n$ 

```

An element of $\text{FlexVec } A$ is a pair (n, v) where $n : \mathbb{N}$ and $v : \text{Vec } A n$. It isn't hard to see that $\text{FlexVec } A$ is equivalent to $\text{List } A$ and indeed we can implement the standard functions on lists:

```

[]fv : FlexVec A
[]fv = 0, []
_::fv_ : A  $\rightarrow$  FlexVec A  $\rightarrow$  FlexVec A
a::fv (n, v) = suc n, (a::v)

```

I leave it as an exercise to derive ltList for FlexVec .

As before for function types products are a special case of Σ -types:

```

A  $\times$  B =  $\Sigma[_ \in A] B$ 

```

However, there is also a different relation we shall explore.

5.4 Relating simple and dependent type formers

You may have noticed that we haven't got a dependent version of sums $_ \uplus _$. However, doesn't the use of the symbol Σ indicate that Σ -types should be related to sums and not products?

And indeed we can derive a binary sums from Σ -types and if_then_else_ .⁴

³I am using $\in$ here because Agda doesn't allow me to use $:$ in abbreviations.

⁴ if_then_else_ was defined in section 3.6, but only for Set . Here I need a *level polymorphic* one to return sets. I will explain this later.

$$\begin{aligned} _ \uplus' _ &: \text{Set} \rightarrow \text{Set} \rightarrow \text{Set} \\ A \uplus' B &= \Sigma[x \in \text{Bool}] \text{ if } x \text{ then } A \text{ else } B \end{aligned}$$

The idea is that an element of $A \uplus' B$ is a pair (b, x) consisting of a boolean b representing the choice between the two alternatives and then depending on this choice an element of either A or B . Given this idea it is straightforward to derive the injections:

$$\begin{aligned} \text{inj}_1' &: A \rightarrow A \uplus' B \\ \text{inj}_1' a &= \text{true}, a \\ \text{inj}_2' &: B \rightarrow A \uplus' B \\ \text{inj}_2' b &= \text{false}, b \end{aligned}$$

We can also derive `case` using pattern matching:

$$\begin{aligned} \text{case}' &: (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow A \uplus' B \rightarrow C \\ \text{case}' f g (\text{false}, b) &= g b \\ \text{case}' f g (\text{true}, a) &= f a \end{aligned}$$

In the same vain we can derive $_ \times _$ from Π :

$$\begin{aligned} _ \times' _ &: \text{Set} \rightarrow \text{Set} \rightarrow \text{Set} \\ A \times' B &= (x : \text{Bool}) \rightarrow \text{if } x \text{ then } A \text{ else } B \end{aligned}$$

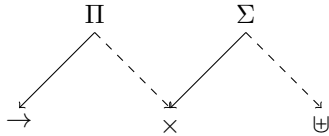
An element of $A \times' B$ is simply a dependent function from booleans to either A or B depending on the input. Hence, the projections are realised by application:

$$\begin{aligned} \text{proj}_1' &: A \times' B \rightarrow A \\ \text{proj}_1' f &= f \text{ true} \\ \text{proj}_2' &: A \times' B \rightarrow B \\ \text{proj}_2' f &= f \text{ false} \end{aligned}$$

The pairing constructor introduces a dependent function by pattern matching:

$$\begin{aligned} _ ,' _ &: A \rightarrow B \rightarrow A \times' B \\ (a ,' b) \text{ false} &= b \\ (a ,' b) \text{ true} &= a \end{aligned}$$

We can relate all the basic type formers in one picture:



Here $\rightarrow$ goes from the dependent to the non-dependent version while $\multimap$ goes from the infinite version to the binary one as discussed above.

You may notice that there are two ways to derive $\times$: either is the non-dependent version of Σ or as the binary case of Π . This corresponds to the

fact that $\times$ can be viewed inductively – if derived from Σ – or coinductively, if derived from Π .

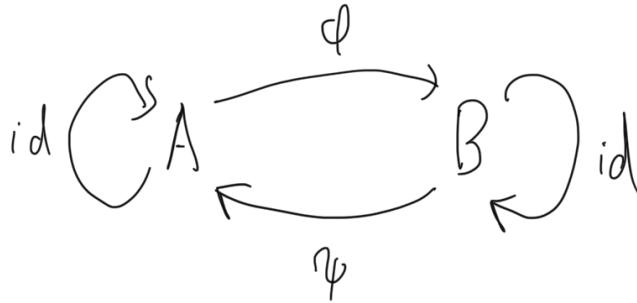
This also explains the use of symbols: Π -types are really infinite products and Σ -types are infinite sums.

5.5 Arithmetic of types

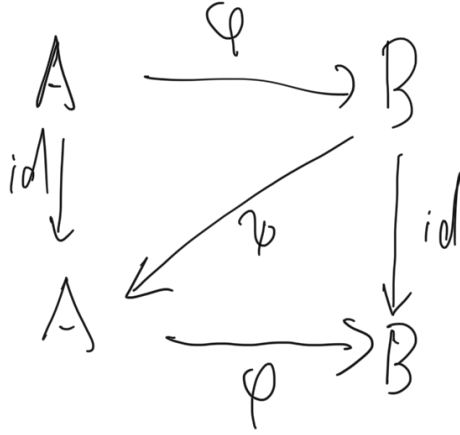
In section 2.5.4 we observed that there is a nice connection between operation on finite types and operations on numbers. Using `Fin` we can make this relation more precise:

$$\begin{aligned}\text{Fin } m \uplus \text{Fin } n &\cong \text{Fin } (m + n) \\ \text{Fin } m \times \text{Fin } n &\cong \text{Fin } (m * n) \\ \text{Fin } m \rightarrow \text{Fin } n &\cong \text{Fin } (n \uparrow m)\end{aligned}$$

By $A \cong B$ we mean *A and B have the same number of elements*. This can be made precise by saying that we have functions $\phi : A \rightarrow B$ and $\psi : B \rightarrow A$ which are inverse to each other, that is $\phi \circ \psi = \text{id}$ and $\psi \circ \phi = \text{id}$. I usually draw the following categorical diagram to depict this situation:



But I realized that it is not clear what compositions should commute. A better diagram is:



In any case since I still haven't introduced equality types ⁵ we will limit ourselves to just talking about the functions in both direction. Hence I define:

```

record _ ~ _ (A B : Set) : Set where
  field
    phi : A -> B
    psi : B -> A
  
```

As an example we can give translations for $\text{Bool} \sim \text{Fin } 2$.

```

bool~2 : Bool ~ Fin 2
phi bool~2 false = zero
phi bool~2 true  = suc zero
psi bool~2 zero  = false
psi bool~2 (suc zero) = true
  
```

It should be clear that these two functions are indeed inverse to each other.

As an example I construct the first equivalence:

```

sum-eq : {m n : ℕ} -> (Fin m ⊔ Fin n) ~ Fin (m + n)
  
```

The idea is that we implement the operations for $\text{Fin } m \uplus \text{Fin } n$ on $\text{Fin } (m + n)$ and then use this for the translation.

The first injection is an embedding which keeps the value the same but changes the index:

```

inj1f : {m n : ℕ} -> Fin m -> Fin (m + n)
inj1f zero = zero
inj1f (suc i) = suc (inj1f i)
  
```

⁵They are coming soon, but the equivalences involving functions really rely on functional extensionality which is only provable in cubical type theory.

The second injection has to shift the elements over the first m that is it applies m successors:

$$\begin{aligned} \text{inj}_2 f &: \{m\ n : \mathbb{N}\} \rightarrow \text{Fin } n \rightarrow \text{Fin } (m + n) \\ \text{inj}_2 f \{ \text{zero} \} i &= i \\ \text{inj}_2 f \{ \text{suc } m \} x &= \text{suc } (\text{inj}_2 f \{ m \} x) \end{aligned}$$

This is also an example where we make a hidden parameter explicit. We didn't really have to supply $\{m\}$ in the recursive call but writing it here actually increases readability.

We can now use the defined injections as the translations of the actual injections and hence implement the first translation:

$$\begin{aligned} \phi \text{ sum-eq } (\text{inj}_1 x) &= \text{inj}_1 f x \\ \phi \text{ sum-eq } (\text{inj}_2 y) &= \text{inj}_2 f y \\ \psi \text{ sum-eq} &= ? \end{aligned}$$

To implement the inverse direction we need to implement `case` on $\text{Fin } (m + n)$ and then use the original injections depending on the interval we are in.

$$\begin{aligned} \text{casef} &: \{m\ n : \mathbb{N}\} \{C : \text{Set}\} \rightarrow (\text{Fin } m \rightarrow C) \rightarrow (\text{Fin } n \rightarrow C) \\ &\rightarrow \text{Fin } (m + n) \rightarrow C \\ \text{casef } \{ \text{zero} \} f\ g\ x &= g\ x \\ \text{casef } \{ \text{suc } m \} f\ g\ \text{zero} &= f\ \text{zero} \\ \text{casef } \{ \text{suc } m \} f\ g\ (\text{suc } i) &= \\ &\text{casef } \{ m \} (\lambda j \rightarrow f (\text{suc } j))\ g\ i \end{aligned}$$

To achieve this we recur both over the first index m and the input $x : \text{Fin } (m + n)$. If $m = 0$ we know that we are in the 2nd case and we can just apply g . Otherwise $\text{suc } m$ we check the input: if it is `zero` we are certainly in the first case and we just use $f\ \text{zero}$. Otherwise we recur on m and i but we need to adjust f by composing it with the successor.

Using `casef` we can complete our translation:

$$\begin{aligned} \phi \text{ sum-eq } (\text{inj}_1 x) &= \text{inj}_1 f x \\ \phi \text{ sum-eq } (\text{inj}_2 y) &= \text{inj}_2 f y \\ \psi \text{ sum-eq} &= \text{casef } \text{inj}_1\ \text{inj}_2 \end{aligned}$$

We can now extend the equivalence to Π - and Σ -types. In Mathematics we use them to represent products and sums over a finite series (actually also infinite but we won't cover this here):

$$\begin{aligned} \Sigma_{i=0}^{i < n} a_i &= a_0 + a_1 + \dots + a_{n-1} \\ \Pi_{i=0}^{i < n} a_i &= a_0 * a_1 * \dots * a_{n-1} \end{aligned}$$

Using finite types we can implement these operations. The sequence a is simply represented as a function $\text{Fin } n \rightarrow \mathbb{N}$. We define

$$\begin{aligned} \Sigma\mathbb{N} &: (n : \mathbb{N}) (a : \text{Fin } n \rightarrow \mathbb{N}) \rightarrow \mathbb{N} \\ \Sigma\mathbb{N} \text{ zero vs } &= 0 \\ \Sigma\mathbb{N} (\text{suc } n) a &= a \text{ zero} + \Sigma\mathbb{N} n (\lambda i \rightarrow a (\text{suc } i)) \end{aligned}$$

and using the same pattern:

$$\begin{aligned} \Pi\mathbb{N} &: (n : \mathbb{N}) (a : \text{Fin } n \rightarrow \mathbb{N}) \rightarrow \mathbb{N} \\ \Pi\mathbb{N} \text{ zero } a &= 1 \\ \Pi\mathbb{N} (\text{suc } n) a &= a \text{ zero} * \Pi\mathbb{N} n (\lambda i \rightarrow a (\text{suc } i)) \end{aligned}$$

We now have the following equivalences:

$$\begin{aligned} \Sigma[i \in \text{Fin } n] \text{Fin } (a \ i)) &\sim \text{Fin } (\Sigma\mathbb{N} n \ a) \\ ((i : \text{Fin } n) \rightarrow \text{Fin } (f \ i)) &\sim \text{Fin } (\Pi\mathbb{N} n \ f) \end{aligned}$$

which is one explanation of the use of Π and Σ for the operations on types.

Here is an example to illustrate the equivalence for Σ -types. Let

$$\begin{aligned} i &: \mathbb{N} \\ i &= 3 \\ a &: \text{Fin } i \rightarrow \mathbb{N} \\ a \text{ zero} &= 3 \\ a (\text{suc } \text{zero}) &= 1 \\ a (\text{suc } (\text{suc } n)) &= 2 \end{aligned}$$

Now clearly

$$\begin{aligned} \Sigma\mathbb{N} i \ a &= 3 + 1 + 2 \\ &= 6 \end{aligned}$$

The following table shows the equivalence:

$0 \mapsto (0, 0)$	$1 \mapsto (0, 1)$	$2 \mapsto (0, 2)$
$3 \mapsto (1, 0)$		
$4 \mapsto (2, 0)$	$5 \mapsto (2, 1)$	

I leave it as an exercise to construct the equivalences. The idea is the same as I have shown in the example of for $_ \uplus _$ we just have to implement the basic operations on the type for the finite representation.

5.6 History

Dependent types were the main innovation which Per Martin-Löf introduced when developing his Type Theory in the early 1970ies [ML75]. Hence it is sometimes called *dependent type theory*. One motivation for dependent types is to extend the proposition as types explanation to predicate logic, which we will study in the next chapter.

Martin-Löf already mentioned the possibility of implementing his theory and this challenge was taken up in the 1980ies by Bob Constable and his group who implemented the NuPRL system, where NuPRL is short for *Nearly ultimate Proof Representation Language* [G⁺95].

A comprehensive overview over Martin-Löf’s Type Theory is given in [MLS84] which was coauthored with Giovanni Sambin. This book presents what we now call *extensional type theory* a presentation of *intensional type theory* which is closer to what is implemented in Agda can be found in [NPS90].

My own first encounter with dependent types was Randy Pollack’s LEGO system [LP92] which was based on Thierry Coquand’s calculus of constructions [LP92] which combined dependent types and Girard’s System F [Gir72]. The calculus of constructions was also the basis of the Coq system [dt20] which has been in development since the 1990ies and which is still very popular.

Apart from using it for proof assistants dependent types are very useful for programming. A relatively early language with dependent types is Cayenne by Lennart Augustsson [Aug98]. The Agda system we are using was developed by Ulf Norell [Nor08] at Gothenburg. It was influenced by earlier implementations of dependently typed languages in Gothenburg (like the ALF system [AGNvS94]) and got its name from a language developed by Catarina Coquand. It was also influenced by Conor’s McBride’s Epigram system which famously featured a 2-dimensional syntax [McB04]. Edwin Brady developed Idris emphasising the programming aspect of dependent types [Bra17] using type-driven program development.

5.7 Exercises

1. We implement some basic operations on vectors and matrices of natural numbers.

We define vectors using `Vec`:

```
Vector : ℕ → Set
Vector m = Vec ℕ m
```

Here `Vector n` is an n -dimensional vector. Here are some examples:

```
v1 : Vector 3
v1 = 1 :: 2 :: 3 :: []
v2 : Vector 3
v2 = 2 :: 3 :: 0 :: []
```

Implement the multiplication of a skalar with a vector:

```
_*_v_ : {n : ℕ} → ℕ → Vector n → Vector n
```

Here is an example:

```
2 *_v v1 = 2 :: 4 :: 6 :: []
```

Next we implement vector addition:

$$_ +v _ : \{n : \mathbb{N}\} \rightarrow \text{Vector } n \rightarrow \text{Vector } n \rightarrow \text{Vector } n$$

Here is an example:

$$v1 +v v2 = 3 :: 5 :: 3 :: []$$

Next we introduce matrices. A matrix is simply a `Vec` of row vectors:

$$\begin{aligned} \text{Matrix} &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Set} \\ \text{Matrix } m \ n &= \text{Vec } (\text{Vector } n) \ m \end{aligned}$$

Hence `Matrix m n` is an $m \times n$ -Matrix.

Here are some examples:

```
id3 : Matrix 3 3
id3 = (1 :: 0 :: 0 :: [])
      :: (0 :: 1 :: 0 :: [])
      :: (0 :: 0 :: 1 :: [])
      :: []

inv3 : Matrix 3 3
inv3 = (0 :: 0 :: 1 :: [])
      :: (0 :: 1 :: 0 :: [])
      :: (1 :: 0 :: 0 :: [])
      :: []

m3 : Matrix 3 3
m3 = (1 :: 2 :: 3 :: [])
      :: (4 :: 5 :: 6 :: [])
      :: (7 :: 8 :: 9 :: [])
      :: []

m4 : Matrix 3 2
m4 = (1 :: 2 :: [])
      :: (4 :: 5 :: [])
      :: (7 :: 8 :: [])
      :: []
```

The next task is to define the multiplication of a vector with a matrix:

$$_ *vm _ : \{m \ n : \mathbb{N}\} \rightarrow \text{Vector } m \rightarrow \text{Matrix } m \ n \rightarrow \text{Vector } n$$

The vector is now viewed as a column vector. Here are some examples:

$$\begin{aligned} v1 *vm m4 &= 30 :: 36 :: [] \\ v1 *vm m3 &= 30 :: 36 :: 42 :: [] \end{aligned}$$

If the input vector is empty you have to produce a vector of 0s with the size given by the matrix. This calls for an axilliary function.

Finally, the task is to implement matrix multiplication. It is recommended to make use of the function we have just defined previously.

$_mm_ : \{l\ m\ n : \mathbb{N}\} \rightarrow \text{Matrix } l\ m \rightarrow \text{Matrix } m\ n \rightarrow \text{Matrix } l\ n$

Here are several examples:

```
inv3 *mm m3 = (7 :: 8 :: 9 :: []) ::
              (4 :: 5 :: 6 :: []) ::
              (1 :: 2 :: 3 :: []) :: []
m3 *mm m4 = (30 :: 36 :: []) ::
            (66 :: 81 :: []) ::
            (102 :: 126 :: []) :: []
m3 *mm m3 = (30 :: 36 :: 42 :: []) ::
            (66 :: 81 :: 96 :: []) ::
            (102 :: 126 :: 150 :: []) :: []
```

2. We continue the topic of matrices. The task is now to define the transpose of a matrix:

$\text{transpose} : \{m\ n : \mathbb{N}\} \rightarrow \text{Matrix } m\ n \rightarrow \text{Matrix } n\ m$

There are certainly several ways to solve this exercise but a nice way is to first establish combinators to establish that $F\ A = \text{Vec } A\ n$ is a *applicative* functor. That is we have operations

```
return : A → F A
app : F (A → B) → F A → F B
```

which are subject to some laws which we will ignore just now.

3. Define the following operations on Fin :

```
max : {n : ℕ} → Fin (suc n)
emb : {n : ℕ} → Fin n → Fin (suc n)
rev : {n : ℕ} → Fin n → Fin n
```

where max computes the maximal element of a finite set, e.g. $\text{max } \{2\} = \text{suc } (\text{suc } \text{zero})$; emb embeds $\text{Fin } n$ into $\text{Fin } (\text{suc } n)$ without changing the numerical value, e.g. $\text{emb } \{2\} (\text{suc } \text{zero}) = \text{suc } \text{zero}$; while rev reverts the order, e.g. $\text{rev } \{4\} (\text{suc } \text{zero}) = \text{suc } (\text{suc } \text{zero})$ and vice versa $\text{rev } \{4\} (\text{suc } (\text{suc } \text{zero})) = \text{suc } \text{zero}$.

4. Derive the translations for $_ \times _$ and $_ \rightarrow _$ and their numerical counterparts. That is derive:

```
prod-eq : {m n : ℕ} → (Fin m × Fin n) ~ Fin (m * n)
exp-eq  : {m n : ℕ} → (Fin m → Fin n) ~ Fin (n ↑ m)
```

5. Derive the translations for Σ - and Π -types and their numerical counterparts. That is derive:

$$\begin{aligned} \Sigma\text{-eq} : \{m : \mathbb{N}\} \{f : \text{Fin } m \rightarrow \mathbb{N}\} \\ \rightarrow (\Sigma [i \in \text{Fin } m] \text{Fin } (f i)) \sim \text{Fin } (\Sigma \mathbb{N} m f) \\ \Sigma\text{-eq} : \{m : \mathbb{N}\} \{f : \text{Fin } m \rightarrow \mathbb{N}\} \\ \rightarrow ((i : \text{Fin } m) \rightarrow \text{Fin } (f i)) \sim \text{Fin } (\Pi \mathbb{N} m f) \end{aligned}$$

Chapter 6

Predicate logic

Predicate logic is the language of Mathematics. Unlike propositional logic which on its own isn't good for anything we can express any mathematical concept in predicate logic. This is why predicate logic has been used to by Peano to present the theory of natural numbers and later by Zermelo and Fraenkel to write down the axioms of set theory. However, from the type-theoretic perspective we don't have to present the formalism of predicate logic but as for propositional logic we just need to explain how we can represent the idioms of predicate logic in Type Theory using the propositions as types translation.

In chapter 3 we introduced propositional logic with the connectives $_ \wedge _$ (and), $_ \vee _$ (or), implication $_ \Rightarrow _$, negation $\neg _$, logical equivalence $_ \Leftrightarrow _$ and the logical constants `True` and `False`. Now we are going to extend this to predicate logic and we are going to introduce the quantifiers $\forall$ (for all) and $\exists$ (exists) and the polymorphic relation $_ \equiv _$ (equality).¹

6.1 Predicates and Quantifiers

What is a predicate? Actually predicate is just another word for property. An example is the predicate `Even` : $\mathbb{N} \rightarrow \text{prop}$, a predicate is a function from some domain to propositions. Since we assume `prop` = `Set` a predicate is nothing but a dependent type. So for example `Even 2` : `prop` is the proposition that 2 is even, while `Even 3` : `prop` is the proposition that 3 is even. We expect that the former is non-empty, i.e. has evidence, while the latter should be no evidence for the latter.

Predicates with several inputs are called relations. Using currying we can represent relations without using products ($_ \times _$) so for example

$$_ \leq _ : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{prop}$$

¹I would prefer to use $_ = _$ for equality and but this would use with agda using it for definitional equality (which we cannot talk about from within Type Theory).

is the less or equal relation. This is not limited to binary relations, an example for a ternary one is congruence modulo $_ \equiv _ \text{mod } _$ e.g. $3 \equiv 7 \text{ mod } 2 : \text{prop}$ expresses the (true) proposition that 3 and 7 have the same remainder when divided by 2.

To avoid repetition we just assume as given $A : \text{Set}$ predicates $P Q : A \rightarrow \text{prop}$ and a proposition $R : \text{prop}$.

We need to define the following operations:

$\forall [x \in A] P x$ For all $x : A$, $P x$ holds.

$\exists [x \in A] P x$ There is an $x : A$, such that $P x$ holds.

So for example $\forall [x \in \mathbb{N}] \text{Even } x \vee \text{Odd } x$ expresses that all natural are even or odd, and $\exists [x \in \mathbb{N}] \text{Even } x$ expresses that there is an even natural number.

What is evidence for $\forall [x \in A] P x$? It is a function that assigns to every $x : A$ evidence for $P x$. Hence we use the dependent function type and define

$$\forall [x \in A] P x = (x : A) \rightarrow P x$$

What is evidence for $\exists P x \in A, P x$? It is a pair of an element $a : A$ and evidence for $P a$. Hence we use a dependent pair type or Σ -type to interpret exists:

$$\exists [x \in A] P x = \Sigma [x \in A] P x$$

We can use function application to instantiate a universally quantified assumption. That is given $h : \forall [x \in A] P x$ and $a : A$ we obtain $f a : P a$. To prove a universally quantified proposition $\forall [x \in A] P x$ we assume as given an arbitrary $x : A$ and prove $p : P x$ and now $\lambda x \rightarrow p : \forall [x \in A] P x$.

To prove an existential statement $\exists [x \in A] P x$ we need to find a witness $a : A$ and show that the witness satisfies the property $p : P a$ then $(a, p) : \exists [x \in A] P x$. On the other hand we can show the elimination principle for existential quantification: that is if we assume $h : \exists [x \in A] P x$ and we can show that a given any $x : A$ which that satisfies $p x$ we can prove a proposition R then we know R . This can be expressed as follows:

$$\text{ex-e} : (\exists [x \in A] P x) \rightarrow (\forall [x \in A] (P x \Rightarrow R)) \rightarrow R$$

This is easy to prove using pattern matching:

$$\text{ex-e } (a, p) f = f a p$$

I noticed that students often struggle with this principle because it is a bit hard to verbalize. Here is my attempt using a concrete example” I use A to be the students in my class, I use $P x$ to mean that x is clever and R means that I m happy. Now we know it is true that if any student is clever then I am happy this corresponds to $\forall [x \in A] P x \rightarrow R$.

We can now complete the table showing the propositions as types explanation for all logical connectives:

<i>latin</i>	<i>english</i>	<i>logic</i>	<i>types / definitions</i>
Conjunction	and	$P \wedge Q$	$P \times Q$
Disjunction	or	$P \vee Q$	$P \uplus Q$
Implication	if-then	$P \Rightarrow Q$	$P \rightarrow Q$
Verum	true	True	$\top$
Falsum	false	False	$\perp$
Negation	not	$\neg P$	$P \Rightarrow \text{False}$
Equivalence	if-and-only-if	$P \Leftrightarrow Q$	$(P \Rightarrow Q) \wedge (Q \Rightarrow P)$
Universal quantification	for all	$\forall [x \in A] P x$	$(x : A) \rightarrow P x$
Existential quantification	exists	$\exists [x \in A] P x$	$\Sigma [x \in A] P x$

Abstract explanations like this are either obvious or incomprehensible. Hence the best is to look at some examples which is what we are going to do in the rest of this chapter.

6.2 Some Tautologies

To illustrate what we can do with our definition of quantifiers, we shall prove some tautologies, that is statements in predicate logic which are true in general.

The first tautology I want to prove is that $\forall$ commutes with $\wedge$, that is

$$\text{taut}_1 : (\forall [x \in A] P x \wedge Q x) \Leftrightarrow (\forall [x \in A] P x) \wedge (\forall [x \in A] Q x)$$

I think it is always helpful to think of a concrete example: so let's say A is the set of students in the lecture, $P x$ means that student x has yellow laces, and $Q x$ means that x wears a black shirt. Now it is clearly the same to say *Everybody has yellow laces and wears a black shirt* and *Everybody has yellow laces and everybody wears a black shirt*, and this has nothing to do with students, laces or shirt colors.

Ok two prove the statement we need to prove two implications:

$$\begin{aligned} \text{proj}_1 \text{taut}_1 &= ? \\ \text{proj}_2 \text{taut}_1 &= ? \end{aligned}$$

Each of them is an implication, hence we may as well introduce parameters.

$$\begin{aligned} \text{proj}_1 \text{taut}_1 h &= ? \\ \text{proj}_2 \text{taut}_1 h &= ? \end{aligned}$$

In the first case we have an assumption $h : \forall [x \in A] P x \wedge Q x$, i.e. a dependent function and we need to prove $(\forall [x \in A] P x) \wedge (\forall [x \in A] Q x)$, that is a pair of functions $?, ?$. In the first component we need to prove $\forall [x \in A] P x$ that is a function $\lambda x \rightarrow ?$ where $? : P x$. Now we can derive $h x : P x \wedge Q x$ and hence obtain $\text{proj}_1 (h x) : P x$. Similarly, we use $\lambda x \rightarrow \text{proj}_2 (h x)$ for the 2nd component.

$$\text{proj}_1 \text{taut}_1 h = (\lambda x \rightarrow \text{proj}_1 (h x)), \lambda x \rightarrow \text{proj}_2 (h x)$$

For the other direction

$$\text{proj}_2 \text{taut}_1 h = ?$$

We have that $h : (\forall [x \in A] P x) \wedge (\forall [x \in A] Q x)$ and we need to show $? : \forall [x \in A] P x \wedge Q x$. We first split our assumption h into two

$$\text{proj}_2 \text{taut}_1 (hp, hq) = ?$$

With $hp : \forall [x \in A] P x$ and $hq : \forall [x \in A] Q x$. Now to prove $? : \forall [x \in A] P x \wedge Q x$ we need to derive a function, hence we use $\lambda x \rightarrow ?$ where $? : P x \wedge Q x$. Hence we need to produce a pair $?, ? : P x \wedge Q x$ which we can do using $hp x : P x$ and $hq x : Q x$. Hence we have

$$\text{proj}_2 \text{taut}_1 (hp, hq) = \lambda x \rightarrow hp x, hq x$$

What happens if we replace $_ \wedge _$ by $_ \vee _$ in the previous statement. Can we prove

$$(\forall [x \in A] P x \vee Q x) \Leftrightarrow (\forall [x \in A] P x) \vee (\forall [x \in A] Q x)$$

A quick analysis using common sense shows that this doesn't hold. It maybe that all students have yellow laces or black shirts but this doesn't mean that all students have yellow laces or all students have black shirts. I.e. the left to right implication doesn't hold. However, the right to left one is actually valid and can be proven:

$$\text{taut}_2 : (\forall [x \in A] P x) \vee (\forall [x \in A] Q x) \Rightarrow (\forall [x \in A] P x \vee Q x)$$

$$\text{taut}_2 (\text{inj}_1 p) = \lambda x \rightarrow \text{inj}_1 (p x)$$

$$\text{taut}_2 (\text{inj}_2 q) = \lambda x \rightarrow \text{inj}_2 (q x)$$

Ok, what happens if we start again with the first tautology but this time we replace $\forall P$ with $\exists P$? Does exists commute with and?

$$(\exists [x \in A] P x \wedge Q x) \Leftrightarrow (\exists [x \in A] P x) \wedge (\exists [x \in A] Q x)$$

Again common sense shows that this doesn't hold, it may well be that there is a student who has yellow laces and another one with a black shirt but this doesn't mean that there is one student with yellow laces and a black shirt. This time the implication from right to left fails but we can prove the one from left to right:

$$\text{taut}_3 : (\exists [x \in A] P x \wedge Q x) \Rightarrow (\exists [x \in A] P x) \wedge (\exists [x \in A] Q x)$$

$$\text{taut}_3 (a, (p, q)) = (a, p), (a, q)$$

The code shows clearly why we can't turn it around. We use the same a twice but this won't work in the other direction when both witnesses may be different.

What happens if we do both replacements at the same time. Can we prove:

$$(\exists [x \in A] P x \vee Q x) \Leftrightarrow (\exists [x \in A] P x) \vee (\exists [x \in A] Q x)$$

And indeed we can:

$$\begin{aligned} \text{taut}_4 &: (\exists [x \in A] P x \vee Q x) \Leftrightarrow (\exists [x \in A] P x) \vee (\exists [x \in A] Q x) \\ \text{proj}_1 \text{taut}_4 (a, \text{inj}_1 p) &= \text{inj}_1 (a, p) \\ \text{proj}_1 \text{taut}_4 (a, \text{inj}_2 q) &= \text{inj}_2 (a, q) \\ \text{proj}_2 \text{taut}_4 (\text{inj}_1 (a, p)) &= a, \text{inj}_1 p \\ \text{proj}_2 \text{taut}_4 (\text{inj}_2 (a, q)) &= a, \text{inj}_2 q \end{aligned}$$

I leave it to you to go through the code line by line.

To summarize we can show:

$$\begin{aligned} (\forall [x \in A] P x \wedge Q x) &\Leftrightarrow (\forall [x \in A] P x) \wedge (\forall [x \in A] Q x) \\ (\exists [x \in A] P x \vee Q x) &\Leftrightarrow (\exists [x \in A] P x) \vee (\exists [x \in A] Q x) \end{aligned}$$

what is the pattern? $\forall$ is represented as a Π -type which is an infinite product and $\wedge$ is represented as $\times$ which is binary product. Now it should be no surprise that products commute with products. In the same vain $\exists$ was represented as a Σ -type which is an infinite sum and $\vee$ was represented using $\uplus$ which is a binary sum and again sums commute with sums.

Finally, here is a tautology which I find difficult to express in english:

$$\text{taut}_5 : ((\exists [x \in A] P x) \Rightarrow R) \Leftrightarrow (\forall [x \in A] P x \Rightarrow R)$$

This time I use $P x$ to mean that x is clever and R means that the lecturer is happy. Now is it equivalent to say *If there is a student who is clever then the lecturer is happy* and *For all students it holds that if the student is clever then the lecturer is happy*? I think we have reached the limits of natural language here and we better just prove it:

$$\begin{aligned} \text{proj}_1 \text{taut}_5 h &= \lambda a p \rightarrow h (a, p) \\ \text{proj}_2 \text{taut}_5 h &= \lambda g \rightarrow h (\text{proj}_1 g) (\text{proj}_2 g) \end{aligned}$$

Not only is the proof very simple, we have seen in before! Yes, this is **curry** and **uncurry** from section 2.5.2 And indeed what we do here is to curry a dependent pair and a function.

6.3 Equality

While we usually define relations that are specific to a set (e.g. $_ \leq _ : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{prop}$), there is one relation which is used for all sets: equality. We write

$$_ \equiv _ : A \rightarrow A \rightarrow \text{prop}$$

That is given a $b : A$ we write $a \equiv b : \text{prop}$ for the proposition that a is equal to b . So for example we write $1 + 2 \equiv 1 + 2 : \text{prop}$ which is actually inhabited, while $0 \equiv 1 : \text{prop}$ is empty.

Equality is defined as an inductive type with one constructor: reflexivity.

```
data  $\equiv$  :  $A \rightarrow A \rightarrow \text{prop}$  where
  refl :  $\{a : A\} \rightarrow a \equiv a$ 
```

Indeed, this is not fundamentally different to the dependent types we have already seen e.g. `Vec` and `Fin`. However, the special thing about equality is that the same index is repeated in the codomain of `refl`.

We can understand the equality type as saying that identical things are equal. This sounds trivial but before we weren't able to talk about identity. The equality type reflects the notion of identity as a type and hence we can talk about it, i.e. use it in a proposition.

Now that we have defined equality, we should verify that basic properties hold. What are the basic properties of equality? First of all it should be an equivalence relation. A relation $\sim : A \rightarrow A \rightarrow \text{prop}$ is an equivalence relation, if it is reflexive, symmetric and transitive. We can express this using a record:²

```
record isEqRel ( $\sim$  :  $A \rightarrow A \rightarrow \text{prop}$ ) : prop where
  field
    reflexive :  $\{a : A\} \rightarrow a \sim a$ 
    symmetric :  $\{a\ b : A\} \rightarrow a \sim b \rightarrow b \sim a$ 
    transitive :  $\{a\ b\ c : A\} \rightarrow a \sim b \rightarrow b \sim c \rightarrow a \sim c$ 
```

Ok let's show that $\equiv : A \rightarrow A \rightarrow \text{prop}$ is an equivalence relation. The first property, reflexivity, is obvious because it is exactly what the constructor `refl` says. What about symmetry?

```
sym :  $\{a\ b : A\} \rightarrow a \equiv b \rightarrow b \equiv a$ 
```

```
sym h = ?
```

We have assumed $a\ b : A$ and $h : a \equiv b$ and our goal is to prove $b \equiv a$. We are going to use the fact that the only proof of $a \equiv b$ is `refl` and pattern match on h . We end up with

```
sym refl = ?
```

And now we only have one assumption $a : A$ and our goal is to prove $a \equiv a$ because once we matched h with `refl` we also need to identify a and b . Now this is easy to complete:

²Ok, at this point I am giving up on using the special $\forall$ -syntax which translates into a dependent function type because it is more convenient to use $\{.. \} \rightarrow$ here because we don't always want to fill in these parameters when using `sym` or any of the other combinators for equality.

`sym refl = refl`

To summarise: we can show that equality is symmetric because the only way we can prove $a \equiv b$ is by `refl` and in this case a and b are identical and we can use `refl` to prove $b \equiv a$ which is just $a \equiv a$.

Transitivity is also straightforward using basically the same idea:

`trans : {a b c : A} → a ≡ b → b ≡ c → a ≡ c`

Ok we start with:

`trans p q = ?`

where $a b c : A$, $p : a \equiv b$ and $q : b \equiv c$ and we need to prove $a \equiv c$. An obvious move is to pattern match one of the equality proofs. But which one? It turns out it doesn't matter. Let's just go with p .

`trans refl q = ?`

Matching p with `refl` has identified a and b . Hence we now only have $a c : A$ and $q : a \equiv c$ and our goal is still $a \equiv c$ which just follow by assumption:

`trans refl q = q`

To summarise: we have shown that `_≡_ {A}` is an equivalence relation:

```
≡-is-eqRel : isEqRel (_≡_ {A})
≡-is-eqRel = record {
  reflexive  = refl;
  symmetric = sym;
  transitive = trans}
```

There are other properties of equality which are useful in proofs. Every function preserves equality, we say equality is a congruence.

`cong : (f : A → B) → {a b : A} → a ≡ b → f a ≡ f b`

Again this is easy to prove by pattern matching $a \equiv b$:

`cong f refl = refl`

At this point we may wonder why we can't prove the inverse of `cong`

`uncong : (f : A → B) → {a b : A} → f a ≡ f b → a ≡ b`

However, we cannot pattern match $f a \equiv f b$. Even though we know that it has to be proven by `refl` we don't know that the argument would have to be $f a$, and indeed f may be a constant function and there is no reason why we should assume that both sides need to be $f a$. Actually `uncong` states that every function is injective which clearly isn't true.

We can only pattern match an equality assumption if at least one side is a variable which can be identified with the other side — which implies it cannot contain this variable as a subterm.

Another important property of equality is that we can replace equals by equals. That is if we have a goal $P \ b$ and we know that $a \equiv b$ then we can prove $P \ a$ instead. This is expressed by the following function:

$$\text{subst} : (P : A \rightarrow \text{prop}) \rightarrow \{a \ b : A\} \rightarrow a \equiv b \rightarrow P \ a \rightarrow P \ b$$

which again is easy to prove using pattern matching:

$$\text{subst } P \ \text{refl } p = p$$

Indeed, all the principles we have proved before can be easily derived from `subst`. I'll leave this as an exercise.

6.3.1 Propositional vs definitional equality

Sometimes Type Theory is criticised because we have two equalities. And indeed, maybe you have noticed that I sometimes use `_ = _` to talk about equalities which follow directly from the definitions of functions and now we have introduced `_ ≡ _` as *propositional equality*. Isn't this rather confusing.

I don't think so. There is only one equality we can talk about *in* Type Theory and this is exactly propositional equality `_ ≡ _`, which captures precisely what we mean by equality in Mathematics. However, when we talk *about* Type Theory then Type Theory is indeed a bit more involved than conventional logic, in that we also have a static equality, i.e. definitional equality. And we cannot talk about this in Type Theory as we cannot talk about the inhabitation `_ : _`. Both are called *judgements* and not propositions. They are static properties of the language, so to say rules of the grammar but not the content we are talking about.

Once we accept that $a : A$ is a judgement we also need to be able to talk about a judgemental equality. For example let's say I define

$$\begin{aligned} n & : \mathbb{N} \\ n & = 3 \end{aligned}$$

and later I construct:

$$\begin{aligned} v & : \text{Vec } \mathbb{N} \ n \\ v & = 1 :: 2 :: 3 :: [] \end{aligned}$$

Then to see that the definition of v is well formed, i.e. is statically an acceptable text we need to know statically that $n = 3$. Which means that equality is a part of the judgement that a certain text is grammatically correct.

Now when I present Type Theory, i.e. in a book about Type Theory or in a lecture I am often switching between two modes: I am talking about Type Theory and then I am talking in Type Theory. Hence I will sometimes mention

that certain expressions are definitionally equal using `_=_` even though this is nothing I can talk about in Type Theory. And certainly if two expressions are definitionally equal then the values they denote are indeed propositionally equal just using `refl`.

6.4 Proving properties of addition

Ok, now we have the tools, let's do something with them. Actually let's build more tools. When reasoning about numbers we need to use algebra and what are the most basic laws of algebra? Here are two basic laws about addition on natural numbers: adding zero doesn't change anything. Why two laws? Since we haven't yet proven that $x + y = y + x$ we can add zero on the left $0 + n \equiv n$ or on the right $n + 0 \equiv n$.

Before we look at them let's recall the definition of addition from section 4.1.2:

```
_+_ : ℕ → ℕ → ℕ
zero + n = n
suc m + n = suc (m + n)
```

Now, let's prove the laws. Starting with adding zero on the left:

```
lneutr+ : {n : ℕ} → 0 + n ≡ n
```

It turns out that this law is trivial, because this follows directly from our definition of `_+_`. Hence we just need to say

```
lneutr+ = refl
```

and we are done.

The other direction is less straightforward but more interesting.

```
rneutr+ : {n : ℕ} → n + 0 ≡ n
```

We cannot say `rneutr+ = refl` because the two sides are not identical or compute to the same expression. However, we can still prove it by using recursion. For this purpose first of all let's make the parameter visible

```
rneutr+ {n} = ?
```

Ok, we have assumed `n : ℕ` what can we do with it? Exactly, we can pattern match on it:

```
rneutr+ {zero} = ?
rneutr+ {suc n} = ?
```

The first goal is `zero + zero ≡ zero` and this is easy to prove again, because we know from the definition of `_+_` that `zero + n = n`, hence

$$\text{rneutr}+ \{ \text{zero} \} = \text{refl}$$

In the $\text{suc } n$ case our goal is $\text{suc } n + \text{zero} \equiv \text{suc } n$. While here we can't reduce to identical expressions we can compute a bit exploiting the second line in the definition of $_+_$ which tells us that $\text{suc } m + n \equiv \text{suc } (m + n)$. Applying this to the left side of our goal it simplifies to $\text{suc } (n + \text{zero}) \equiv \text{suc } n$. This suggests we could recursively prove $n + \text{zero} \equiv n$ but we need to get rid of the pesky suc . But we can, we just need to use cong suc :

$$\text{rneutr}+ \{ \text{suc } n \} = \text{cong suc } ?$$

And now indeed the goal is $n + \text{zero} \equiv n$ and we can use a recursive call:

$$\text{rneutr}+ \{ \text{suc } n \} = \text{cong suc } (\text{rneutr}+ \{ n \})$$

What has actually happened? We just used recursion in a proof! And it meant that to prove that some properties holds for all natural numbers it is enough to prove it for zero and prove it for $\text{suc } n$ using that it holds for n . This proof principle has a name, it is called *the principle of induction*. This is a nice consequence of propositions as types: induction is recursion.

Let's move on. Another very useful property of addition is that it doesn't matter how we bracket several additions, that is:

$$\text{assoc}+ : \{ l \ m \ n : \mathbb{N} \} \rightarrow (l + m) + n \equiv l + (m + n)$$

This property is called *associativity*. Again our strategy is to use pattern matching and recursion but this time we have three numbers to choose from: l , m and n . It turns out that the first choice l is the right one, but why? Looking at the definition of addition we notice that an expression of the form $e_1 + e_2$ computes if e_1 starts with a constructor. Hence it is a good idea to turn expressions on the left into constructors and l is best positioned to achieve this.

$$\begin{aligned} \text{assoc}+ \{ \text{zero} \} &= ? \\ \text{assoc}+ \{ \text{suc } l \} &= ? \end{aligned}$$

The first goal is to show $(\text{zero} + m) + n \equiv \text{zero} + (m + n)$ and using the definition of $_+_$ we can see

$$\begin{aligned} (\text{zero} + m) + n &= m + n \\ \text{zero} + (m + n) &= m + n \end{aligned}$$

Hence we just need to say:

$$\text{assoc}+ \{ \text{zero} \} = \text{refl}$$

Ok, what about the $\text{suc } l$ case? We need to prove $(\text{suc } l + m) + n \equiv \text{suc } l + (m + n)$. Again we can use the definition of $_+_$:

$$\begin{aligned} (\text{suc } l + m) + n &= \text{suc } (l + m) + n \\ &= \text{suc } ((l + m) + n) \\ (\text{suc } l) + (m + n) &= \text{suc } (l + (m + n)) \end{aligned}$$

Hence all we need to do is to use `cong suc` and then we can use recursion:

$$\text{assoc+ } \{\text{suc } l\} = \text{cong suc } (\text{assoc+ } \{l\})$$

In algebra there is a name for an operation (like $_ + _$) and an element which behaves like 0: it is called *a monoid*:

```
record IsMonoid
  (e : A) (•_ : A → A → A) : prop where
  field
    lneutr : {x : A} → e • x ≡ x
    rneutr : {x : A} → x • e ≡ x
    assoc  : {x y z : A} → (x • y) • z ≡ x • (y • z)
```

and we can summarize our results so far:

```
+isMonoid : IsMonoid zero _+_
+isMonoid = record {
  lneutr = lneutr+;
  rneutr = rneutr+;
  assoc  = λ {x} {y} {z} → assoc+ {x} {y} {z}}
```

However, addition is one more useful property: it is commutative $m + n \equiv n + m$. How can we prove this?

$$\text{comm+} : \{m\ n : \mathbb{N}\} \rightarrow m + n \equiv n + m$$

Again the question is over which argument to do the induction (aka recursion). However, due to the symmetry of the equation it doesn't matter - hence let's use the first.

$$\begin{aligned} \text{comm+ } \{\text{zero}\} \{n\} &= ? \\ \text{comm+ } \{\text{suc } m\} \{n\} &= ? \end{aligned}$$

We get two goals:

$$\begin{aligned} \text{zero} + n &\equiv n + \text{zero} \\ \text{suc } m + n &\equiv n + \text{suc } m \end{aligned}$$

In both cases we can simplify the left hand side, but there is no simplification possible for the righthand sides, because they are stuck on n .

$$\begin{aligned} n &\equiv n + \text{zero} \\ \text{suc } (m + n) &\equiv n + \text{suc } m \end{aligned}$$

The first goal can be handled with `rneutr+` but it is not clear how to make progress with the 2nd. We need a way to move the successor from the 2nd argument to the front - while the definition only provides a way to move the

successor from inside the first argument. Hence we suspend this proof and first prove a lemma:

$$\text{suc+} : \{m\ n : \mathbb{N}\} \rightarrow \text{suc}\ (n + m) \equiv n + \text{suc}\ m$$

In this case it is clear that the induction should use n . The rest is straight forward:

$$\begin{aligned} \text{suc+}\ \{m\}\ \{\text{zero}\} &= \text{refl} \\ \text{suc+}\ \{m\}\ \{\text{suc}\ n\} &= \text{cong}\ \text{suc}\ (\text{suc+}\ \{m\}\ \{n\}) \end{aligned}$$

Back to our original problem:

$$\text{comm+} : \{m\ n : \mathbb{N}\} \rightarrow m + n \equiv n + m$$

While as remarked the **zero** case can be handled with **renutr+** there is a slight inconvenience that the equation is the wrong way around: **rneutr** proves $n + \text{zero} \equiv n$ but we need $n \equiv n + \text{zero}$. This can be easily dealt with using **sym**:

$$\text{comm+}\ \{\text{zero}\}\ \{n\} = \text{sym}\ \text{rneutr+}$$

For the second case we have to combine the lemma **suc+** we have shown and the recursive call. That is to show

$$\text{suc}\ m + n \equiv n + \text{suc}\ m$$

We use

$$\begin{aligned} \text{suc}\ m + n &= \text{suc}\ (m + n) && \text{The definition of } + \\ &= \text{suc}\ (n + m) && \text{The induction hypothesis for } m \\ &= n + \text{suc}\ m && \text{The lemma } \text{suc+} \end{aligned}$$

Using **trans** and **cong** we can translate this into agda code

$$\text{comm+}\ \{\text{suc}\ m\}\ \{n\} = \text{trans}\ (\text{cong}\ \text{suc}\ (\text{comm+}\ \{m\}\ \{n\}))\ \text{suc+}$$

It is clear that this presentation of equational proofs will become very unreadable very quickly. The standard agda library defines some syntactic sugar which provides a nicer presentation which shows the intermediate goals and enables us to layout the derivation nicely but which actually reduces to the same proof.

$$\begin{aligned} \text{comm+}\ \{\text{suc}\ m\}\ \{n\} &= \text{begin} \\ &\text{suc}\ m + n \\ &\equiv \langle \text{refl} \rangle \\ &\text{suc}\ (m + n) \\ &\equiv \langle \text{cong}\ \text{suc}\ (\text{comm+}\ \{m\}\ \{n\}) \rangle \\ &\text{suc}\ (n + m) \\ &\equiv \langle \text{suc+} \rangle \end{aligned}$$

$n + \text{succ } m$

■

However, we still have to spend a lot of space for mathematically trivial steps. This can be addressed using a solver which automatically computes solutions.

To summarise we have shown that $_ + _$ forms a *commutative monoid*:

```

record IsCommMonoid
  (e : A) (_ • _ : A → A → A) : prop where
  field
    isMonoid : IsMonoid e _ • _
    isComm : {x y : A} → x • y ≡ y • x

+isCommMonoid : IsCommMonoid zero _ + _
+isCommMonoid = record {
  isMonoid = +isMonoid;
  isComm = λ {x} {y} → comm+ {x} {y}}

```

6.5 Extending the negative translation

We have introduced the negative translation in section 3.4 as an alternative to the boolean interpretation of classical logic. Now moving to predicate logic we cannot actually use the boolean interpretation because given $P : A \rightarrow \text{Bool}$ how can we determine a boolean to $\forall [x : A] P x$. In particular it may be that A is infinite, e.g. $A = \mathbb{N}$.

However, the negative translation which translates any proposition P so that $\text{RAA } P = \neg (\neg P) \rightarrow P$ holds still works as we will see. First of all we need to show that RAA holds for $\forall [x \in A] P x$ if for all $x : A$ we have that $\text{RAA } (P x)$ holds.

```

all-neg : ((x : A) → RAA (P x)) → RAA (∀ [x ∈ A] P x)
all-neg raa h x = raa x (λ g → h (λ k → g (k x)))

```

What about $\exists [x \in A] P x$? Like $P \vee Q$ this is a positive statement, i.e. it contains information, namely the choice of witness. Hence as for $_ \vee _$ we translate $\exists$ using the classical equivalence that $\exists [x \in A] P x$ is equivalent to saying that it is not the case the property isn't false for all $x : A$.

```

Ex-c : (A : Set) (P : A → prop) → prop
Ex-c A P = ¬ (∀ [x ∈ A] ¬ (P x))

```

I am going to write $\exists^c [x \in A] P$ for classical existence. As for $_ \vee^c _$. It should be clear that we have:

```

neg-ex-c : RAA (∃c [x ∈ A] P)

```

because we have already shown that all negated propositions satisfy RAA.

We need to show that the basic principles for existentials are provable, that is we need to show:

$$\begin{aligned} \text{ex}^c\text{-i} &: (a : A) \rightarrow P\ a \rightarrow \exists c[x \in A] P\ x \\ \text{ex}^c\text{-e} &: \text{RAA}\ R \rightarrow ((x : A) \rightarrow P\ x \rightarrow R) \rightarrow \exists^c[x \in A] P\ x \rightarrow R \end{aligned}$$

What about equality? It turns out that most equalities we encounter are already negative, for example $_ \equiv _ \{ \mathbb{N} \}$ satisfies RAA because it is decidable as we are going to show in the next chapter. However, we don't need to assume this but we can define classical equality as the double negation of the usual equality:

$$\begin{aligned} _ \equiv^c _ &: A \rightarrow A \rightarrow \text{prop} \\ a \equiv^c b &= \neg (\neg (a \equiv b)) \end{aligned}$$

and as before we have:

$$\text{ne-eq-c} : \{a\ b : A\} \rightarrow \text{RAA}\ (a \equiv^c b)$$

We can derive some of the basic principles associated with equality for $_ \equiv^c _$:

$$\begin{aligned} \text{refl}^c &: \{a : A\} \rightarrow a \equiv^c a \\ \text{subst}^c &: (P : A \rightarrow \text{prop}) \rightarrow (\{x : A\} \rightarrow \text{RAA}\ (P\ x)) \\ &\rightarrow \{a\ b : A\} \rightarrow a \equiv^c b \rightarrow P\ a \rightarrow P\ b \end{aligned}$$

I am leaving the verification of these properties as an exercise.

Hence it seems that classical logic is just the logic of the negative fragment of intuitionistic logic where we use negative, i.e. classical versions, of disjunction and existence. Almost, but there is another important aspect of classical Mathematics: the axiom of choice which we are going to discuss later. The negative translation is not going to help us here because the negative translation of the axiom of choice is not provable intuitionistically.

6.6 History

An early form of predicate logic was introduced by Frege in the late 19th century [Fre79]. His basic laws allowed quantification over all sets and hence was inconsistent as Russell pointed out. However, his attempt to make the rules of reasoning precise led to the formal system of predicate logic. This is also often called first order logic, because we don't allow quantification over propositions themselves.

While Frege tried to capture set theory and logic at the same time, later it became clear that it is a good idea to separate the system of reasoning and the particular basic assumptions, i.e. axioms. Hence for example Peano Arithmetic and Zermelo-Fraenkel set theory can be formulated in the same framework.

It was Hilbert and Ackermann who clearly formulated first order logic in the 1930ies, (there is a recent english translation: [HA99]).

Gödel proved a completeness theorem for classical predicate logic which shows that the formal system is complete for the classical notion of a model, i.e. if a proposition holds in all models it is also provable [Kur29]. However, this is done for classical logic in a classical metatheory and in this form it is not relevant in our context.

Unlike set theory type theory doesn't use the framework of predicate logic it stands on its own. However, many of the idioms of predicate logic are used and we can understand them via their translation into predicate logic as we have shown. However, the interpretation of disjunction $\vee$ and existence $\exists$ we have represented are debatable since the types don't really behave like propositions. We will return to this issue later and present an alternative view what is a proposition.

6.7 Exercises

1. We are playing logic poker again. But this time with predicate logic!

Consider the following propositions:

$$\begin{aligned}
P01 &= \{A \ B : \text{Set}\} \{R : A \rightarrow B \rightarrow \text{prop}\} \rightarrow \\
&\quad ((\forall [x \in A] \ \exists [y \in B] \ R \ x \ y)) \rightarrow \\
&\quad (\exists [y \in B] \ \forall [x \in A] \ R \ x \ y) \\
P02 &= \{A \ B : \text{Set}\} \{R : A \rightarrow B \rightarrow \text{prop}\} \rightarrow \\
&\quad (\exists [y \in B] \ (\forall [x \in A] \ R \ x \ y)) \\
&\quad \rightarrow (\forall [x \in A] \ (\exists [y \in B] \ R \ x \ y)) \\
P03 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad \neg (\exists [x \in A] \ P \ x) \rightarrow \forall [x \in A] \ \neg (P \ x) \\
P04 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\forall [x \in A] \ \neg (P \ x)) \rightarrow \neg (\exists [x \in A] \ P \ x) \\
P05 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\neg (\forall [x \in A] \ P \ x)) \rightarrow \exists [x \in A] \ \neg (P \ x) \\
P06 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\exists [x \in A] \ \neg (P \ x)) \rightarrow (\neg (\forall [x \in A] \ P \ x)) \\
P07 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\neg (\neg (\forall [x \in A] \ P \ x))) \rightarrow \forall [x \in A] \ \neg (\neg (P \ x)) \\
P08 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\forall [x \in A] \ \neg (\neg (P \ x))) \rightarrow (\neg (\neg (\forall [x \in A] \ P \ x))) \\
P09 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\exists [x \in A] \ \top) \rightarrow (\exists [x \in A] \ P \ x) \rightarrow \forall [x \in A] \ P \ x \\
P10 &= \{A : \text{Set}\} \{P : A \rightarrow \text{prop}\} \rightarrow \\
&\quad (\exists [x \in A] \ \top) \rightarrow (\exists [y \in A] \ (P \ y \rightarrow \forall [x \in A] \ P \ x))
\end{aligned}$$

Here is a quick summary of the rules of logic poker:

- (a) You can prove it directly (intuitionistically) - just provide $p_i : P_i$.

- (b) You cannot prove it with intuitionistic logic but you can prove it with classical logic. In this case prove $\text{pi} : \text{CLASS} \rightarrow \text{Pi}$ where

$$\text{CLASS} = \{P : \text{prop}\} \rightarrow \text{RAA } P$$

- (c) It isn't even true in classical logic. In this case prove $\text{pi} : \text{Pi} \rightarrow \perp$. Note that you will have to come up with predicates and relations to refute the proposition.

2. Show that all the properties of equality we have shown are derivable from `subst`. That is prove

$$\begin{aligned} \text{sym-s} &: \{a \ b : A\} \rightarrow a \equiv b \rightarrow b \equiv a \\ \text{trans-s} &: \{a \ b \ c : A\} \rightarrow a \equiv b \rightarrow b \equiv c \rightarrow a \equiv c \\ \text{cong-s} &: (f : A \rightarrow B) \rightarrow \{a \ b : A\} \rightarrow a \equiv b \rightarrow f \ a \equiv f \ b \end{aligned}$$

using only:

$$\text{subst} : (P : A \rightarrow \text{prop}) \rightarrow \{a \ b : A\} \rightarrow a \equiv b \rightarrow P \ a \rightarrow P \ b$$

instead of pattern matching.

3. Verify the laws of boolean algebra we discussed in section 3.1.

$$\begin{aligned} \text{com-}\& : \{x \ y : \text{Bool}\} \rightarrow x \& y \equiv y \& x \\ \text{distr-}\&| &: \{x \ y \ z : \text{Bool}\} \rightarrow x \& (y | z) \equiv x \& y | x \& z \\ \text{dm1} &: \{x \ y : \text{Bool}\} \rightarrow !(x | y) \equiv !x \& !y \\ \text{dm2} &: \{x \ y : \text{Bool}\} \rightarrow !(x \& y) \equiv !x | !y \end{aligned}$$

4. Show that the natural numbers $\mathbb{N}$ with addition and multiplication form a commutative semiring. We have already shown that addition forms a commutative monoid, hence it is left to show:

$$\begin{aligned} \text{*isCommMonoid} &: \text{IsCommMonoid } 1 \ _ \ * _ \\ \text{null-l} &: \{n : \mathbb{N}\} \rightarrow 0 * n \equiv 0 \\ \text{distr-l} &: \{l \ m \ n : \mathbb{N}\} \rightarrow (l + m) * n \equiv l * n + m * n \end{aligned}$$

Note that I have omitted the symmetric forms of the distributivity laws:

$$\begin{aligned} \text{null-r} &: \{n : \mathbb{N}\} \rightarrow n * 0 \equiv 0 \\ \text{distr-r} &: \{l \ m \ n : \mathbb{N}\} \rightarrow l * (m + n) \equiv l * m + l * n \end{aligned}$$

since they follow from commutativity but it may be useful to prove them before establishing commutativity.

5. Continuing from the previous question: show that the natural numbers form an exponential semi-ring, that is prove the laws of high school algebra:

$$\begin{aligned} \text{exp-plus} &: \{l \ m \ n : \mathbb{N}\} \rightarrow l \uparrow (m + n) \equiv l \uparrow m * l \uparrow n \\ \text{exp-times} &: \{l \ m \ n : \mathbb{N}\} \rightarrow l \uparrow (m * n) \equiv (l \uparrow m) \uparrow n \end{aligned}$$

6. Prove the missing lemmas about the negative translation from section 6.5.

$$\begin{aligned}
 \text{all-neg} & : ((x : A) \rightarrow \text{RAA } (P \ x)) \rightarrow \text{RAA } (\forall [x \in A] P \ x) \\
 \text{ex}^c\text{-i} & : (a : A) \rightarrow P \ a \rightarrow \exists^c [x \in A] P \ x \\
 \text{ex}^c\text{-e} & : \text{RAA } R \rightarrow ((x : A) \rightarrow P \ x \rightarrow R) \rightarrow \exists^c [x \in A] P \ x \rightarrow R \\
 \text{refl}^c & : \{a : A\} \rightarrow a \equiv^c a \\
 \text{subst}^c & : (P : A \rightarrow \text{prop}) \rightarrow (\{x : A\} \rightarrow \text{RAA } (P \ x)) \\
 & \rightarrow \{a \ b : A\} \rightarrow a \equiv^c b \rightarrow P \ a \rightarrow P \ b
 \end{aligned}$$

Chapter 7

Programs and proofs

While in the last chapter I have spent some effort to introduce a nice syntax for predicate logic resembling the commonly used symbols actually in type theory we usually stick to Π -types and Σ -types where the latter are usually replaced by more generic record types. One of the reasons for this is that the proofs and programs or logic and types are not always clearly separable.

Let's for example consider a program that sorts natural numbers. It should have the type

$$\text{sort} : \text{List } \mathbb{N} \rightarrow \text{List } \mathbb{N}$$

Let's say we want to be more specific and express in the type that the program does actually sorts its input. Let's assume we have defined a predicated $\text{Sorted} : \text{List } \mathbb{N} \rightarrow \text{prop}$ which expressed that a list is sorted and a relation $\sim : \text{List } \mathbb{N} \rightarrow \text{List } \mathbb{N} \rightarrow \text{prop}$ where $l \sim l'$ means that both lists have the same elements but in a different order (we say one is a permutation of the other). Now we can define a more refined output type as a record:

```
record SortSpec (inp : List ℕ) : Set where
  out : List ℕ
  sorted : Sorted out
  perm : inp ~ out
```

and our sorting program now has the type

$$\text{sort} : (\text{inp} : \text{List } \mathbb{N}) \rightarrow \text{SortSpec inp}$$

By the way this shows how we can also use Σ -types or record types to express what is called *comprehension* in set theory, that is:

$$\text{SortSpec out} = \{ \text{out} : \text{List } \mathbb{N} \mid \text{Sorted out} \wedge \text{inp} \sim \text{out} \}$$

However, the point I want to make is that we think of the list `out` as data and the other two components `sorted` and `perm` as proofs of propositions. However, if

we want to use the `sort`-function to externally sort a sequence maybe using some robot then it is `perm` which tells us how the items should be rearranged! That is what has been a proof becomes data and we should have said: $_ \sim _ : \text{List } \mathbb{N} \rightarrow \text{List } \mathbb{N} \rightarrow \text{Set}$. Hence the distinction between `Set` and `prop` we have made was a bit artificial and maybe we should have stuck to `Set` anyway.

7.1 Decidability of equality

We are going to look at another program that combines logic and types. We can define a function that decides whether two natural numbers are equal:

```
eqb : ℕ → ℕ → Bool
eqb zero    zero    = true
eqb zero    (suc n) = false
eqb (suc m) zero    = false
eqb (suc m) (suc n) = eqb m n
```

While this is a perfectly fine program we have to read its code to see that it is doing the right thing. Ok, this is maybe easy in this case but wouldn't it be nicer if the type says what the program is doing (as in our improved version of `sort`)? And not only nicer but also potentially more useful because this sort of information may come in handy when we want to prove something about some program that uses `eqb`. That is we would like to express that `eqb` decides $_ \equiv _ \{ \mathbb{N} \}$.

Here we can exploit an advantage of working in a logic where $P \vee \neg P$ is not a tautology because we can use $P \vee \neg P$ to express that P has been decided, i.e. we know whether it is true or false. Since this construction is used frequently, there is an explicit definition as a sum:

```
data Dec (P : Set) : Set where
  yes : P → Dec P
  no  : ¬ P → Dec P
```

That is `Dec P` is equivalent to $P \vee \neg P$ and expresses that P has been decided. Examples for propositions which can be decided easily are `True` and `False`:

```
decTrue : Dec True
decTrue = yes tt
decFalse : Dec False
decFalse = no case⊥
```

More interesting cases of decided propositions include *Fermat's last theorem* but not for example $P=NP$, i.e. whether the class of problems solvable in deterministic and non-deterministic polynomial time are the same. While most people think that the answer is no, nobody has got a proof so we cannot prove $\text{Dec } P=NP$.

There is some hope that one day $P=NP$ will be decided some day and we may wonder whether there are intrinsically undecidable propositions? The answer is no, because we proved

$$\text{nntnd} : \{P : \text{prop}\} \rightarrow \neg (\neg (P \vee \neg P))$$

in section 3.3.

We can use the notion of decided to express what it means for a predicate $P : A \rightarrow \text{prop}$ to be *decidable*, it means that we can decide each instance:

$$(x : A) \rightarrow \text{Dec } (P x)$$

It is not worth to define a predicate *decidable* because it would only work for predicates but not for relations and our first application is exactly the decidability of $_ \equiv _ \{ \mathbb{N} \}$:

$$_ \equiv \mathbb{N} ? _ : (m \ n : \mathbb{N}) \rightarrow \text{Dec } (m \equiv n)$$

The first case is easy:

$$\text{zero} \equiv \mathbb{N} ? \text{zero} = \text{yes refl}$$

We say *yes* and we give the reason. In the next case we want to say no:

$$\text{zero} \equiv \mathbb{N} ? \text{suc } n = \text{no } ?$$

We need to show $\neg (\text{zero} \equiv \text{suc } n)$. It turns out that this can be done just by local pattern matching because *agda knows* that different constructors cannot be equal:

$$\text{zero} \equiv \mathbb{N} ? \text{suc } n = \text{no } \lambda ()$$

The next case is symmetric and we use the same trick:

$$\text{suc } m \equiv \mathbb{N} ? \text{zero} = \text{no } \lambda ()$$

Finally if both inputs start with a successor we have to investigate the result of the recursive call:

$$\begin{aligned} & \text{suc } m \equiv \mathbb{N} ? \text{suc } n \text{ **with** } m \equiv \mathbb{N} ? n \\ & (\text{suc } m \equiv \mathbb{N} ? \text{suc } n) \mid \text{yes } p = ? \\ & (\text{suc } m \equiv \mathbb{N} ? \text{suc } n) \mid \text{no } np = ? \end{aligned}$$

In the *yes* case we can just match x :

$$\begin{aligned} & \text{suc } m \equiv \mathbb{N} ? \text{suc } n \text{ **with** } m \equiv \mathbb{N} ? n \\ & (\text{suc } m \equiv \mathbb{N} ? \text{suc } .m) \mid \text{yes refl} = \text{yes refl} \\ & (\text{suc } m \equiv \mathbb{N} ? \text{suc } n) \mid \text{no } np = ? \end{aligned}$$

The negative case is a bit more interesting. Certainly if $\neg (m \equiv n)$ then also $\neg (\text{suc } m \equiv \text{suc } n)$:

```

suc m ≡N? suc n with m ≡N? n
(suc m ≡N? suc .m) | yes refl = yes refl
(suc m ≡N? suc n)   | no np   = no (λ p → np ?)

```

At this point we need to show $m \equiv n$ and we know $p : \text{suc } m \equiv \text{suc } n$ hence we need to know that suc is injective. This is easy to prove using (local) pattern matching:

```

suc m ≡N? suc n with m ≡N? n
(suc m ≡N? suc .m) | yes refl = yes refl
(suc m ≡N? suc n)   | no np   = no (λ { refl → np refl })

```

If we look at the whole function definition again:

```

_≡N?_ : (m n : ℕ) → Dec (m ≡ n)
zero ≡N? zero   = yes refl
zero ≡N? suc n   = no λ ()
suc m ≡N? zero   = no λ ()
suc m ≡N? suc n with m ≡N? n
(suc m ≡N? suc .m) | yes refl = yes refl
(suc m ≡N? suc n)   | no np   = no (λ { refl → np refl })

```

we see that it has exactly the same structure as `eqb` but instead of returning a boolean we return an element of $\text{Dec } (m \equiv n)$ which is basically a boolean with the reason that we are allowed to return it.

Hence decidability is an example where we can refine a simply typed program with a non very expressive type into a type which tells us exactly what the function is doing. Again we see that the border between reasoning and programming is rather fluid.

7.1.1 Church's thesis

While we cannot have a proposition which cannot be decided, it is possible to have a predicate or a relation which is not decidable. Indeed the Halting problem provides an example, which we can view as a predicate

$\text{Halts} : \mathbb{N} \rightarrow \text{prop}$

assuming that we use some encoding of programs (e.g. Turing machines) as natural numbers. Can we show:

$\text{undecHalt} : ((n : \mathbb{N}) \rightarrow \text{Dec } (\text{Halts } n)) \rightarrow \perp$

The answer is no: Type Theory still allows us to assume the excluded middle for all propositions

$\text{tnd} : \{P : \text{prop}\} \rightarrow \text{Dec } P$

and using this there is a trivial proof of $(n : \mathbb{N}) \rightarrow \text{Dec} (\text{Halts } n)$ and hence it is inconsistent to have both `undecHalt` and `tnd`. We can assume a principle which says that all functions on the natural numbers are computable (this is called *Church's thesis*) and using this we can prove `undecHalt`. Hence, clearly Church's thesis is inconsistent with classical logic, it is anti-classical.

7.1.2 Uniqueness of equality proofs

In Type Theory we can ask questions that don't make sense in conventional reasoning. For example we may wonder whether there is at most one way to prove that two numbers are equal

$$\text{uip}\mathbb{N} : \{m\ n : \mathbb{N}\} (p\ q : m \equiv n) \rightarrow p \equiv q$$

This is interesting because it shows that $\equiv_{\mathbb{N}}$ is *propositional*. Ok, previously we have said that `prop` = `Set` and I have argued that the distinction is not very precise. However, we may say that a type is a proposition if it contains no other information but that it is inhabited. That is all elements are equal.

It seems that we can prove `uipN` in a generic way using pattern matching. We match the first element:

$$\text{uip}\mathbb{N} \text{ refl } q = ?$$

Now we have $q : m \equiv m$. Can we pattern match this? Before we were instantiating at least one of the arguments, but this time they are already equal. One may think this doesn't matter and:

$$\text{uip}\mathbb{N} \text{ refl refl} = ?$$

Now we just have to show `refl` $\equiv$ `ref` which is easy:

$$\text{uip}\mathbb{N} \text{ refl refl} = \text{refl}$$

However, the last pattern matching isn't completely uncontroversial and in particular it is inconsistent with Homotopy Type Theory which we will be considering later. We can rule out this proof by enabling the option `--without-K` which will reject the pattern matching on `q`.

However, it turns out that in this particular case we can still prove `uipN`. This is an instance of Hedberg's theorem which tells us that `uip` holds for any type which has a decidable equality.

7.2 Inductively defined relations: less or equal

The equality relation has been defined as an inductive type in a particular simple way: we only needed one constructor. Inductively defined relations and predicates are quite useful in general and we shall use the less or equal relation on natural numbers $\leq : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Set}$ as an example.

There are various ways to define $_ \leq _$, e.g. we can use addition $m \leq n = \exists [k \in \mathbb{N}] k + m \equiv n$. However, here we shall go another way and define the relation as given by some deduction rules:

$$\frac{}{0 \leq n} \text{le0} \quad \frac{m \leq n}{\text{suc } m \leq \text{suc } n} \text{leS}$$

We can use these rules to derive for example that $1 \leq 3$:

$$\frac{\frac{}{0 \leq 2} \text{le0}}{1 \leq 3} \text{leS}$$

On the other hand there cannot be a derivation of $3 \leq 1$. The only rule directly applicable is leS and after this we are stuck because no rule applies:

$$\frac{\text{impossible}}{\frac{2 \leq 0}{3 \leq 1} \text{leS}}$$

We can turn this idea into an inductive definition with le0 and leS as constructors:

data $_ \leq _ : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Set}$ **where**
 $\text{le0} : \{n : \mathbb{N}\} \rightarrow \text{zero} \leq n$
 $\text{leS} : \{m n : \mathbb{N}\} \rightarrow m \leq n \rightarrow \text{suc } m \leq \text{suc } n$

The derivation of $1 \leq 3$ can be expressed:

$$\begin{aligned} 1 \leq 3 &: 1 \leq 3 \\ 1 \leq 3 &= \text{leS le0} \end{aligned}$$

Using pattern matching we can show $\neg (3 \leq 1)$:

$$\begin{aligned} \neg 3 \leq 1 &: \neg (3 \leq 1) \\ \neg 3 \leq 1 \ x &= ? \end{aligned}$$

We pattern match $x : 3 \leq 1$, only leS is applicable:

$$\begin{aligned} \neg 3 \leq 1 &: \neg (3 \leq 1) \\ \neg 3 \leq 1 \ (\text{leS } x) &= ? \end{aligned}$$

Now $x : 2 \leq 0$ and no constructor is applicable, hence we get the empty pattern:

$$\begin{aligned} \neg 3 \leq 1 &: \neg (3 \leq 1) \\ \neg 3 \leq 1 \ (\text{leS } ()) & \end{aligned}$$

and we are done.

Let's establish some basic properties of $_ \leq _$, it is reflexive, transitive and antisymmetric. A relation with these properties is called a *partial order*. We

have already seen reflexivity and transitivity from equality which was an equivalence relation. Antisymmetry is not exactly the opposite of symmetry but it says that $m \leq n$ and $n \leq m$ can only both hold, if $m \equiv n$.

We start with reflexivity which we can prove by induction over natural numbers:

$$\begin{aligned} \text{refl} \leq & : \{n : \mathbb{N}\} \rightarrow n \leq n \\ \text{refl} \leq \{\text{zero}\} & = \text{le0} \\ \text{refl} \leq \{\text{suc } n\} & = \text{leS } (\text{refl} \leq \{n\}) \end{aligned}$$

Transitivity is maybe more interesting.

$$\text{trans} \leq : \{l \ m \ n : \mathbb{N}\} \rightarrow l \leq m \rightarrow m \leq n \rightarrow l \leq n$$

$$\text{trans} \leq p \ q = ?$$

Here we need to do an induction over derivations, which just corresponds to pattern matching

$$\begin{aligned} \text{trans} \leq \text{le0 } q & = ? \\ \text{trans} \leq (\text{leS } p) \ q & = ? \end{aligned}$$

The first case $\text{zero} \leq n$ is easily dealt with:

$$\text{trans} \leq \text{le0 } q = \text{le0}$$

While the 2nd needs a further induction / pattern matching over the 2nd derivation $q : \text{suc } m \leq n$ but luckily only one rule applies:

$$\text{trans} \leq (\text{leS } p) (\text{leS } q) = ?$$

And now to show $\text{suc } l \leq \text{suc } n$ we only need to use leS and recursion, aka appeal to the induction hypothesis:

$$\text{trans} \leq (\text{leS } p) (\text{leS } q) = \text{leS } (\text{trans} \leq p \ q)$$

The proof of antisymmetry:

$$\text{antisym} : \{m \ n : \mathbb{N}\} \rightarrow m \leq n \rightarrow n \leq m \rightarrow m \equiv n$$

works in a similar way: we do induction over both derivations and nicely our first choice limits the 2nd:

$$\begin{aligned} \text{antisym } \text{le0 } \text{le0} & = \text{refl} \\ \text{antisym } (\text{leS } p) (\text{leS } q) & = \text{cong suc } (\text{antisym } p \ q) \end{aligned}$$

In the 2nd line we reduce showing $\text{suc } m \equiv \text{suc } n$ to $m \equiv n$ using cong suc and this we can prove recursively.

Indeed, $_ \leq _$ is decidable, which is a refinement of the boolean function:

```

leb : ℕ → ℕ → Bool
leb zero n = true
leb (suc m) zero = false
leb (suc m) (suc n) = leb m n

```

This is similar to our construction of $_ \equiv? _$:

```

_≤?_ : (m n : ℕ) → Dec (m ≤ n)
zero ≤? n = yes le0
suc m ≤? zero = no (λ ())
suc m ≤? suc n with m ≤? n
(suc m ≤? suc n) | yes x = yes (leS x)
(suc m ≤? suc n) | no x = no (λ {(leS p) → x p})

```

In the last case we are exploiting that leS is invertible, i.e. that $\text{suc } m \leq \text{suc } n$ implies $m \leq n$. This follows by pattern matching since leS is the only constructor that can prove $\text{suc } m \leq \text{suc } n$.

Finally, we may ask whether $m \leq n$ is a proposition in the sense explained in the last section. Indeed, it should be clear that it is, since we never had a choice between derivations. However, this time there are no issues and the construction is permitted even without K .

```

isProp≤ : {m n : ℕ} (p q : m ≤ n) → p ≡ q
isProp≤ le0 le0 = refl
isProp≤ (leS p) (leS q) = cong leS (isProp≤ p q)

```

In the last line we actually use cong on a proof rule: we reduce showing $\text{leS } p \equiv \text{leS } q$ to $p \equiv q$ which can be done by recursion.

7.3 An alternative definition of less or equal

The definition of $_ \leq _$ in the previous section is not the only possibility. Here is an alternative:

```

data _≤'_ : ℕ → ℕ → Set where
  le'refl : {m : ℕ} → m ≤' m
  le'S : {m n : ℕ} → m ≤' n → m ≤' suc n

```

We use le'refl to show that $n \leq n$ and then we can raise the second argument using le'S . So for example:

```

1 ≤' 3 : 1 ≤' 3
1 ≤' 3 = le'S (le'S le'refl)

```

And using pattern matching we can still show negative results:

```

¬3 ≤' 1 : ¬ (3 ≤' 1)
¬3 ≤' 1 (le'S ())

```

I hope it is intuitively clear that $_ \leq _$ and $_ \leq' _$ define the same relation. Let's show this in agda:

$$\leq \Leftrightarrow \leq' : \{m\ n : \mathbb{N}\} \rightarrow m \leq n \Leftrightarrow m \leq' n$$

We have to define functions in both directions, mapping one sort of derivations into the other and vice versa.

$$\begin{aligned} \text{proj}_1 \leq \Leftrightarrow \leq' p &= ? \\ \text{proj}_2 \leq \Leftrightarrow \leq' p &= ? \end{aligned}$$

Let's look at the first line. We analyse the derivation of $m \leq n$

$$\begin{aligned} \text{proj}_1 \leq \Leftrightarrow \leq' \text{le0} &= ? \\ \text{proj}_1 \leq \Leftrightarrow \leq' (\text{leS } p) &= ? \end{aligned}$$

We observe that we need to prove that $_ \leq' _$ is closed under the rules defining $_ \leq _$. That is we need to establish

$$\begin{aligned} \text{le0} \leq' : \{n : \mathbb{N}\} &\rightarrow \text{zero} \leq' n \\ \text{suc}' \leq' : \{m\ n : \mathbb{N}\} &\rightarrow m \leq' n \rightarrow \text{suc } m \leq' \text{suc } n \end{aligned}$$

For the first one we use recursion over $n : \mathbb{N}$:

$$\begin{aligned} \text{le0} \leq' \{\text{zero}\} &= \text{le'refl} \\ \text{le0} \leq' \{\text{suc } n\} &= \text{le'S } (\text{le0} \leq' \{n\}) \end{aligned}$$

and for the second we do an induction over the derivation of $m \leq' n$:

$$\begin{aligned} \text{suc}' \leq' \text{le'refl} &= \text{le'refl} \\ \text{suc}' \leq' (\text{le'S } p) &= \text{le'S } (\text{suc}' \leq' p) \end{aligned}$$

Using these we can recursively substitute the rules used to derive $m \leq n$ to prove $m \leq' n$.

$$\begin{aligned} \text{proj}_1 \leq \Leftrightarrow \leq' \text{le0} &= \text{le0} \leq' \\ \text{proj}_1 \leq \Leftrightarrow \leq' (\text{leS } p) &= \text{suc}' \leq' (\text{proj}_1 \leq \Leftrightarrow \leq' p) \end{aligned}$$

We apply the same strategy for the other direction. We now need to show that $_ \leq _$ is closed under the rules defining $_ \leq' _$. The first one we have already done, when we showed that $_ \leq _$ is reflexive, but we still need to show the 2nd which we derive again by recursion over derivations:

$$\begin{aligned} \text{leS} \leq : \{m\ n : \mathbb{N}\} &\rightarrow m \leq n \rightarrow m \leq \text{suc } n \\ \text{leS} \leq \text{le0} &= \text{le0} \\ \text{leS} \leq (\text{leS } p) &= \text{leS } (\text{leS} \leq p) \end{aligned}$$

Here is the complete proof:

$$\begin{aligned}
\leq \Leftrightarrow \leq' &: \{m\ n : \mathbb{N}\} \rightarrow m \leq n \Leftrightarrow m \leq' n \\
\text{proj}_1 \leq \Leftrightarrow \leq' \text{le0} &= \text{le0} \leq' \\
\text{proj}_1 \leq \Leftrightarrow \leq' (\text{leS } p) &= \text{suc}' \leq' (\text{proj}_1 \leq \Leftrightarrow \leq' p) \\
\text{proj}_2 \leq \Leftrightarrow \leq' \text{le'refl} &= \text{refl} \leq \\
\text{proj}_2 \leq \Leftrightarrow \leq' (\text{le'S } p) &= \text{leS} \leq (\text{proj}_2 \leq \Leftrightarrow \leq' p)
\end{aligned}$$

In the usual jargon we could say that $_ \leq _$ is the least relation closed under the rules defining it and once we show that $_ \leq' _$ is closed under the same rules we know that $m \leq n$ implies $m \leq' n$. However, this seems a bit ambitious, do we really need to think about the least relation among all relations? The reasoning we are using here is more concrete, we just mapping trees to trees. There is no need to think about the *least relation*

7.4 History

The idea that we can use dependent types to support a programming style where programs are intrinsically correct is quite recent. It goes along with the evolution of implementations such as the ALF system [AGNvS94] in Gothenburg, Conor McBride's OLEG [McB00a] which grew out of the LEGO system [LP92].¹ OLEG later evolved to Epigram [McB04] which together with ALF provided much input into Agda.

I made the case for *progification* as a synthesis of programming and verification in an unpublished paper [Alt94]. This was also the base of my course at the ESSL summer school in Prague in 1996 [Alt96] on *Integrated verification in Type Theory* which also featured an introduction to ALF.

Conor McBride² is the most influential contributor to this style of writing correct programs using dependent types (see his PhD thesis [McB00a]): he has explained this in many of his *gigs* which involved animations made with plastic slides in a time when we still used overhead projectors. Most famous is his talk *Winging It* [McB09] where he explains dependent types by an analogy with a proletarian revolution: the terms are the workers on the left of the colon whereas the types are the bourgeoisie on the right hand side of the colon who have no idea about work.

¹Conor said that OLEGO was a rearrangement of LEGO which referred to the treatment of holes. However, it was also necessary to rearrange the name due to threats by lawyers of the LEGO company now that the internet was growing beyond academia.

²For full disclosure: Conor was like me supervised by Rod Burstall in Edinburgh.



However, the established social order is overcome and the terms march through the colon. Are the types now running away in panic?



No, they are actually happy about their new expressivity:



Jointly with James McKinna many of the ideas are explained in *The view*

from the left [MM04] and later in our joint and also our unpublished paper [AMM05] on *why dependent types matter?*.

7.5 Exercises

1. Show that equality for the type `Expr` as defined in section 4.3.2 is decidable.

```
data Expr : Set where
  const : ℕ → Expr
  _[+]_ : Expr → Expr → Expr
  _[*]_ : Expr → Expr → Expr
```

That is construct a program of the following type:

$$_ \equiv \text{Expr?} _ : (e\ f : \text{Expr}) \rightarrow \text{Dec } (e \equiv f)$$

You may use `_≡ℕ?_`.

2. Prove uniqueness of equality proofs for `_≡_ {ℕ}` without using `K`, that is with the flag `--without-K` enabled.

$$\text{uip}\mathbb{N} : \{m\ n : \mathbb{N}\} (p\ q : m \equiv n) \rightarrow p \equiv q$$

Remark: While this is a consequence of Hedberg’s theorem which we will introduce later — it is also possible to prove it directly. For this purpose it may be helpful to define equality for the natural numbers inductively:

```
data _~ℕ_ : ℕ → ℕ → Set where
  zero : zero ~ℕ zero
  suc : {m n : ℕ} → m ~ℕ n → suc m ~ℕ suc n
```

and then showing that the `_≡_ {ℕ}` and `_~ℕ_` are equivalent (Actually it is enough to establish a retraction, that is a function and with an inverse). It should be straightforward to show uniqueness for `_~ℕ_`.

3. A common definition of $\leq$ avoiding the use of an inductively defined family is

$$\begin{aligned} _ \leq + _ &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{prop} \\ m \leq + n &= \exists [k \in \mathbb{N}] \ k + m \equiv n \end{aligned}$$

or equivalently one could also use:

$$\begin{aligned} _ + \leq _ &: \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{prop} \\ m + \leq n &= \exists [k \in \mathbb{N}] \ m + k \equiv n \end{aligned}$$

Show that they are logically equivalent to $_ \leq _$ or $_ \leq' _$ without having to use commutativity of $_ + _$. That is show

$$R \Leftrightarrow S : \{m, n : \mathbb{N}\} \rightarrow m R n \Leftrightarrow m S n$$

for R being $\leq +$ or $+ \leq$ and depending on the choice for R , S is either $\leq$ or $\leq'$.

Chapter 8

Deduction

So far I have introduced the rules of Type Theory informally and have explained logical principles by the proposition as types explanation. But in the end we want to make the rules of reasoning precise. This can be done by presenting a formal system defining the logic or the type theory.

In this chapter we will look at a very modest subset, basically the logic corresponding to propositional logic using only implication, which corresponds to section 2.1. We are not going to model β -reduction but only present a notion of provability which is called *natural deduction*. We will also study the relationship to combinatory logic and give a formal account of the translation presented in section 2.4.

Another important aspect of a logical system is its semantics. We will use the semantics to prove non-derivability results, i.e. that something is not provable. First we use the naive semantics which allows us to disprove statements by giving a counterexample but then we will explore a more precise semantics, Kripke models, which allows us to disprove also classically valid statements like Pierce's formula $((P \rightarrow Q) \rightarrow P) \rightarrow P$. We prove completeness of Kripke models (i.e. everything which holds semantically is also derivable) and then use a modified form of the completeness result to prove normalisation — this is inspired by a technique called *normalisation by evaluation*.

This chapter also serves a different purpose in giving more examples how the technology we have introduced in the previous chapters can be used, in particular how we can use inductive families to present deductive systems.

8.1 The rules of natural deduction

The first step is to formally define the syntax of propositions, i.e. formulas: no surprises here they will be trees. The trees are built from implications which we will write as $_[\Rightarrow]_$ but what is at the leaves of the trees? OK we need to introduce propositional variables which I am going to represent as strings.

```

data Form : Set where
  Var : String → Form
  _[⇒]_ : Form → Form → Form

```

So for example we can represent the proposition

$$(Q \rightarrow R) \rightarrow (P \rightarrow Q) \rightarrow P \rightarrow R : \text{prop}$$

as

```

((Var "Q") [⇒] (Var "R")) [⇒] ((Var "P") [⇒] (Var "Q"))
[⇒] (Var "P") [⇒] (Var "R")

```

Using strings to represent propositional variables is not the best choice but we will go for it in the moment. However, we will not use strings for the variables used in terms or derivations as we will see in a moment.

We want to define a predicate what propositional formulas are derivable, that is $\vdash : \text{Form} \rightarrow \text{Set}$ but this is not sufficient because we need to take care of assumptions as well. That is we are going to introduce a type $\text{Con} : \text{Set}$ of contexts and then define derivations as $_ \vdash _ : \text{Con} \rightarrow \text{Form} \rightarrow \text{Set}$.

How to represent contexts? As I said I am not going to use names to represent variables but only pointers, or actually a form of *de Bruijn indices*. Hence a context is just a sequence of assumptions, i.e. a list of formulas:

$\text{Con} = \text{List Form}$

We are going to use $\Gamma \Delta \Theta : \text{Con}$ to quantify over contexts and $A B C : \text{Form}$ for formulas.

Ok, what are the rules of natural deduction. There are two rules to deal with implication: the first one corresponds to λ -abstraction and it conveys the idea: to prove $A \Rightarrow B$ we assume A and prove B :

$$\frac{(A :: \Gamma) \vdash B}{\Gamma \vdash A \Rightarrow B} \text{ lam}$$

The other rule corresponds to application: If we know $A \Rightarrow B$ and also A then we can derive B . The context stays the same, i.e. the number of assumptions doesn't change.

$$\frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \text{ app}$$

This rule is also called *modus ponens*.

We still need some structural rules to access the context of assumptions. In the usual syntax of λ -calculus we use named variables but really these variables are just pointers into the context. Hence we can replace variables by numbers representing the offset within the context. This was introduced by Niklas de Bruijn and hence we call this representation of variables *de Bruijn indices*. E.g. the λ -term $\lambda f \rightarrow \lambda x \rightarrow f x x$ would be written as $\lambda \lambda 1 0 0$, because x is

represented by 0 since it was the last variable introduced and f is represented by the index 1 because it is one step down in our list of assumptions.

Hence we introduce the two rules:

$$\frac{}{(A :: \Gamma) \vdash A} \text{ zero} \quad \frac{\Gamma \vdash A}{(B :: \Gamma) \vdash A} \text{ suc}$$

zero allows us to access the last item in the context, and **suc** discards the last item and allows us to access previous items. So for example in the context

$$(\text{Var "P"}) :: (\text{Var "Q"}) :: []$$

We can access

$$\frac{}{(\text{Var "P"}) :: (\text{Var "Q"}) :: [] \vdash \text{Var "P"}} \text{ zero}$$

and

$$\frac{\frac{}{(\text{Var "Q"}) :: [] \vdash \text{Var "Q"}} \text{ zero}}{(\text{Var "P"}) :: (\text{Var "Q"}) :: [] \vdash \text{Var "Q"}} \text{ suc}$$

Actually since we don't have a special type of *assumptions* (corresponding to variables) we can apply **suc** to any derivation. This rule is usually called weakening. In terms of de Bruijn indices it means that we can replace 1 2 by **suc** (0 1) that is we *weaken* all free variables, i.e. add a successor.

As before for $_ \leq _$, derivations are an indexed inductive definition with constructors corresponding to rules:

```
data  $\_ \vdash \_ : \text{Con} \rightarrow \text{Form} \rightarrow \text{Set}$  where
  zero :  $(A :: \Gamma) \vdash A$ 
  suc  :  $\Gamma \vdash A \rightarrow (B :: \Gamma) \vdash A$ 
  lam  :  $(A :: \Gamma) \vdash B \rightarrow \Gamma \vdash (A [\Rightarrow] B)$ 
  app  :  $\Gamma \vdash (A [\Rightarrow] B) \rightarrow \Gamma \vdash A \rightarrow \Gamma \vdash B$ 
```

As an example we derive $(Q \rightarrow R) \rightarrow (P \rightarrow Q) \rightarrow P \rightarrow R$. That is we translate the meta level Agda construction

$$\lambda f g x \rightarrow f (g x) : (Q \rightarrow R) \rightarrow (P \rightarrow Q) \rightarrow P \rightarrow R$$

into

$$\begin{aligned} \text{d-C} &: [] \vdash ((\text{Var "Q"}) [\Rightarrow] (\text{Var "R"})) [\Rightarrow] ((\text{Var "P"}) [\Rightarrow] (\text{Var "Q"})) \\ &\quad [\Rightarrow] (\text{Var "P"}) [\Rightarrow] (\text{Var "R"}) \\ \text{d-C} &= \text{lam} (\text{lam} (\text{lam} (\text{app} (\text{suc} (\text{suc zero})) (\text{app} (\text{suc zero} zero)))) \end{aligned}$$

8.2 Semantics

Ok, we can translate λ -terms into derivations in our calculus, using de Bruijn indices to represent variables. However, certain formulas should not be derivable, for example we should not be able to derive

$$[] \vdash ((\text{Var "P"}) \Rightarrow (\text{Var "Q"})) \Rightarrow (\text{Var "P"}).$$

Why not. Because we can actually show that this proposition is false for instances of the variables:

$$\begin{aligned} \text{cex} &: (\{P \ Q : \text{prop}\} \rightarrow (P \Rightarrow Q) \Rightarrow P) \rightarrow \perp \\ \text{cex } h &= h \{\perp\} \{\top\} (\lambda x \rightarrow \text{tt}) \end{aligned}$$

Hence if our system is reasonable, the technical term is *sound*, we should expect that this formula is not derivable.

When we needed to show that certain instances of $_ \leq _$ are not derivable, e.g. $\neg (3 \leq 2)$ we could just use pattern matching which ended with impossible branches. Can we do the same here to prove:

$$(\Box \vdash (((\text{Var "P"}) [\Rightarrow] (\text{Var "Q"})) [\Rightarrow] (\text{Var "P"}))) \rightarrow \perp$$

We quickly notice that this doesn't lead anywhere. In particular the **app**-rule allows us to introduce more and more complicated formulas whose unprovability doesn't lead to a situation where no rules are applicable.

However, the solution is to actually prove soundness, that is to show that any formula we can derive is actually true for any instances of the propositional variables. This provides more than only just a tool to prove non-derivability it also shows us that our system is sensible.

We define semantics of formulas, contexts and derivations. The semantics of a formula is a proposition, but we need to know an interpretation of the propositional variables. I call this an *environment*:

$$\text{Env} = \text{String} \rightarrow \text{prop}$$

Now we can define the semantics of formulas:

$$\begin{aligned} \llbracket _ \rrbracket F &: \text{Form} \rightarrow \text{Env} \rightarrow \text{prop} \\ \llbracket \text{Var } x \rrbracket F \rho &= \rho x \\ \llbracket A [\Rightarrow] B \rrbracket F \rho &= \llbracket A \rrbracket F \rho \Rightarrow \llbracket B \rrbracket F \rho \end{aligned}$$

I am using here Scott brackets $\llbracket \dots \rrbracket$, named after Dana Scott, to signal the demarcation between syntax and semantics.

The definition is very straightforward, variables we look up in the context and $_ [\Rightarrow] _$ is translated as $_ \Rightarrow _$. Contexts are just the conjunction of all the assumptions, hence:

$$\begin{aligned} \llbracket _ \rrbracket C &: \text{Con} \rightarrow \text{Env} \rightarrow \text{prop} \\ \llbracket \Box \rrbracket C \rho &= \top \\ \llbracket A :: \Gamma \rrbracket C \rho &= \llbracket A \rrbracket F \rho \wedge \llbracket \Gamma \rrbracket C \rho \end{aligned}$$

Finally, a derivation is translated into *the assumptions imply the conclusions*:

$$\begin{aligned} \llbracket _ \rrbracket d &: \Gamma \vdash A \rightarrow \{\rho : \text{Env}\} \rightarrow \llbracket \Gamma \rrbracket C \rho \rightarrow \llbracket A \rrbracket F \rho \\ \llbracket \text{zero} \rrbracket d (p, _) &= p \end{aligned}$$

$$\begin{aligned}
\llbracket \text{suc } d \rrbracket d \, (_, \gamma) &= \llbracket d \rrbracket d \, \gamma \\
\llbracket \text{lam } d \rrbracket d \, \gamma &= \lambda p \rightarrow \llbracket d \rrbracket d \, (p, \gamma) \\
\llbracket \text{app } d \, e \rrbracket d \, \gamma &= \llbracket d \rrbracket d \, \gamma \, (\llbracket e \rrbracket d \, \gamma)
\end{aligned}$$

The rules for **zero** and **suc** translate indices into the access to the context (similar to our definition of $_!!_$ earlier), and **lam** is translated using λ -abstraction and **app** using application.

We can now reap the fruits of our efforts and apart from having the happy feeling that our calculus actually makes sense we can prove the non-derivability example. First we need to cook up an environment:

$$\begin{aligned}
\rho_0 &: \text{Env} \\
\rho_0 \text{ "P" } &= \perp \\
\rho_0 \text{ "Q" } &= \top \\
\rho_0 _ &= \perp
\end{aligned}$$

Here we also see why I don't like to use strings for variables: we need to assign a meaning to every string and not just to the ones we used in our formula. Here our solution is to interpret all other variables with $\perp$ as a default.

We are now ready to show the non-derivability result:

$$\begin{aligned}
\text{cex-d} &: (\llbracket _ \rrbracket \vdash (((\text{Var "P"}) [\Rightarrow] (\text{Var "Q"})) [\Rightarrow] (\text{Var "P"}))) \rightarrow \perp \\
\text{cex-d } d &= \llbracket d \rrbracket d \, \{\rho_0\} \text{ tt } (\lambda ())
\end{aligned}$$

The method we have used exploits that we could refute the proposition semantically whose non-derivability we wanted to show. However, for a classically true formula we cannot do this, as we cannot refute excluded middle as we have seen previously. Hence the naive semantics we have shown here isn't good enough. While it is *sound*, it is not *complete*.

8.3 Equivalence with combinatory logic

In section 2.4 we informally showed that pure functions in simply typed λ -calculus can be represented using just the combinators **S** and **K**. We will do this formally now and show that provability in combinatory logic, i.e. a logic which uses only **S**, **K** and application but no **lam** is equivalent to provability in natural deduction.

One of the main advantages of combinatory logic is that we don't need to use variables, aka assumptions. That means we don't have to use contexts but can use a one sided version of $\vdash$:

$$\begin{aligned}
\text{data } \vdash_{\text{sk}} &: \text{Form} \rightarrow \text{Set} \text{ where} \\
\text{SS} &: \vdash_{\text{sk}} ((A [\Rightarrow] B [\Rightarrow] C) \\
&\quad [\Rightarrow] (A [\Rightarrow] B) [\Rightarrow] A [\Rightarrow] C) \\
\text{KK} &: \vdash_{\text{sk}} (A [\Rightarrow] B [\Rightarrow] A) \\
\text{app} &: \vdash_{\text{sk}} (A [\Rightarrow] B) \rightarrow \vdash_{\text{sk}} A \rightarrow \vdash_{\text{sk}} B
\end{aligned}$$

Our goal is to show that propositions provable from no assumptions are equivalent to propositions provable in propositional logic.

$$\text{thm} : (\Box \vdash A) \Leftrightarrow \vdash\text{-sk } A$$

The direction from right to left is the easy one. All we need to show is that the combinators S and K are provable in natural deduction:

$$\begin{aligned} \text{K-d} &: \Box \vdash (A \Rightarrow B \Rightarrow A) \\ \text{K-d} &= \text{lam } (\text{lam } (\text{suc zero})) \\ \text{S-d} &: \Box \vdash ((A \Rightarrow B \Rightarrow C) \\ &\quad [\Rightarrow] (A \Rightarrow B) [\Rightarrow] A [\Rightarrow] C) \\ \text{S-d} &= \\ &\quad \text{lam } (\text{lam } (\text{lam } (\text{app } (\text{app } (\text{suc } (\text{suc zero})) \text{ zero}) \\ &\quad (\text{app } (\text{suc zero}) \text{ zero})))) \end{aligned}$$

We only need them in the empty context here but we could actually prove them in any context. We can now show one implication $\vdash\text{-sk } A \Rightarrow \Box \vdash A$ by induction over the derivation of $\vdash\text{-sk}$:

$$\begin{aligned} \text{proj}_2 \text{ thm SS} &= \text{S-d} \\ \text{proj}_2 \text{ thm KK} &= \text{K-d} \\ \text{proj}_2 \text{ thm (app d e)} &= \text{app } (\text{proj}_2 \text{ thm d}) (\text{proj}_2 \text{ thm e}) \end{aligned}$$

The other direction is a bit more difficult. We cannot hope to prove anything by induction over $\Box \vdash A$ because as soon as we encounter lam we need to talk about non-empty contexts.

To be able to compare natural deduction with combinatory logic we introduce a two-sided version of natural deduction with assumptions. Here we are using the same rules as for natural deduction:

$$\begin{aligned} \text{data } _ \vdash\text{-skC } _ : \text{Con} \rightarrow \text{Form} \rightarrow \text{Set where} \\ \text{zero} &: (A :: \Gamma) \vdash\text{-skC } A \\ \text{suc} &: \Gamma \vdash\text{-skC } A \rightarrow (B :: \Gamma) \vdash\text{-skC } A \\ \text{SS} &: \Gamma \vdash\text{-skC } ((A \Rightarrow B \Rightarrow C) \\ &\quad [\Rightarrow] (A \Rightarrow B) [\Rightarrow] A [\Rightarrow] C) \\ \text{KK} &: \Gamma \vdash\text{-skC } (A \Rightarrow B \Rightarrow A) \\ \text{app} &: \Gamma \vdash\text{-skC } (A \Rightarrow B) \rightarrow \Gamma \vdash\text{-skC } A \rightarrow \Gamma \vdash\text{-skC } B \end{aligned}$$

We can show that derivability in the empty context for $\vdash\text{-skC}$ coincides with provability in $\vdash\text{-sk}$. Actually for our proof we only need one direction:

$$\begin{aligned} \text{skC} \rightarrow \text{sk} &: \Box \vdash\text{-skC } A \rightarrow \vdash\text{-sk } A \\ \text{skC} \rightarrow \text{sk SS} &= \text{SS} \\ \text{skC} \rightarrow \text{sk KK} &= \text{KK} \\ \text{skC} \rightarrow \text{sk (app d e)} &= \text{app } (\text{skC} \rightarrow \text{sk d}) (\text{skC} \rightarrow \text{sk e}) \end{aligned}$$

Unlike for natural deduction we can prove this by derivation over $\Box \vdash\text{-skC } A$ for the empty context because there is no lam and the variable rules do not apply in the empty context.

We can now try to show a more general implication relating natural deduction ($_ \vdash _$) and the two-sided combinatory logic: ($_ \vdash_{\text{skC}} _$):

$$\vdash \rightarrow \vdash_{\text{skC}} : \Gamma \vdash A \rightarrow \Gamma \vdash_{\text{skC}} A$$

It is clear that we are going to prove this by induction over $\Gamma \vdash A$, i.e. we pattern match on the derivation:

$$\begin{aligned} \vdash \rightarrow \vdash_{\text{skC}} \text{zero} &= ? \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{suc } d) &= ? \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{lam } d) &= ? \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{app } d \ e) &= ? \end{aligned}$$

The cases for `zero`, `suc` and `app` are straightforward because we have corresponding constructors for $\vdash_{\text{skC}}$:

$$\begin{aligned} \vdash \rightarrow \vdash_{\text{skC}} \text{zero} &= \text{zero} \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{suc } d) &= \text{suc } (\vdash \rightarrow \vdash_{\text{skC}} d) \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{app } d \ e) &= \text{app } (\vdash \rightarrow \vdash_{\text{skC}} d) (\vdash \rightarrow \vdash_{\text{skC}} e) \end{aligned}$$

It remains the case for `lam` which is the core of the proof, namely that we can simulate `lam` using just `S` and `K`. Hence we need to prove a lemma, that $_ \vdash_{\text{skC}} _$ is closed under `lam`:

$$\text{lam-sk} : (A :: \Gamma) \vdash_{\text{skC}} B \rightarrow \Gamma \vdash_{\text{skC}} (A [\Rightarrow] B)$$

We will do this by induction over derivations again:

$$\begin{aligned} \text{lam-sk zero} &= ? \\ \text{lam-sk (suc } d) &= ? \\ \text{lam-sk SS} &= ? \\ \text{lam-sk KK} &= ? \\ \text{lam-sk (app } d \ e) &= ? \end{aligned}$$

The first line, the case for `zero` we used `zero` to prove $(A :: \Gamma) \vdash_{\text{skC}} A$ hence we need to show $\Gamma \vdash_{\text{skC}} (A [\Rightarrow] A)$. This is exactly the point where we needed `I`. Hence we better derive it:

$$\begin{aligned} \text{II} &: \Gamma \vdash_{\text{skC}} (A [\Rightarrow] A) \\ \text{II } \{A = A\} &= \text{app } (\text{app SS KK}) (\text{KK } \{B = A\}) \end{aligned}$$

Here we encounter the same problem as before, we need to instantiate one of the propositional variables explicitly. Using `II` we can cover the `zero`-case:

$$\text{lam-sk zero} = \text{II}$$

The next case is `suc`, given a derivation $d : \Gamma \vdash_{\text{skC}} A$ we get $\text{suc } d : (B :: \Gamma) \vdash_{\text{skC}} A$. Hence we need to derive $\Gamma \vdash_{\text{skC}} B \Rightarrow A$. In this case we

used `suc` aka weakening to ignore the last variable and this is clearly the time for `K`:

$$\text{lam-sk (suc d)} = \text{app KK d}$$

The cases where we used `SS` or `KK` to derive $(A :: \Gamma) \vdash_{\text{skC}} B$ are similar, because we certainly ignored the last variable. Hence we are also using `KK` for those cases together with the same combinator:

$$\begin{aligned} \text{lam-sk SS} &= \text{app KK SS} \\ \text{lam-sk KK} &= \text{app KK KK} \end{aligned}$$

The final and interesting case is `app`, we are abstracting from an application. That is exactly the point which justifies `S` which distributes abstractions over an application. This is the only case where we actually need to refer to the recursive result in the proof:

$$\text{lam-sk (app d e)} = \text{app (app SS (lam-sk d)) (lam-sk e)}$$

We have now covered all the cases:

$$\begin{aligned} \text{lam-sk} &: (A :: \Gamma) \vdash_{\text{skC}} B \rightarrow \Gamma \vdash_{\text{skC}} (A [\Rightarrow] B) \\ \text{lam-sk zero} &= \text{II} \\ \text{lam-sk (suc d)} &= \text{app KK d} \\ \text{lam-sk SS} &= \text{app KK SS} \\ \text{lam-sk KK} &= \text{app KK KK} \\ \text{lam-sk (app d e)} &= \text{app (app SS (lam-sk d)) (lam-sk e)} \end{aligned}$$

Now we can complete the proof of $\vdash \rightarrow \vdash_{\text{skC}}$ by filling in the case for `lam`:

$$\begin{aligned} \vdash \rightarrow \vdash_{\text{skC}} &: \Gamma \vdash A \rightarrow \Gamma \vdash_{\text{skC}} A \\ \vdash \rightarrow \vdash_{\text{skC}} \text{zero} &= \text{zero} \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{suc d}) &= \text{suc } (\vdash \rightarrow \vdash_{\text{skC}} d) \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{lam d}) &= \text{lam-sk } (\vdash \rightarrow \vdash_{\text{skC}} d) \\ \vdash \rightarrow \vdash_{\text{skC}} (\text{app d e}) &= \text{app } (\vdash \rightarrow \vdash_{\text{skC}} d) (\vdash \rightarrow \vdash_{\text{skC}} e) \end{aligned}$$

We can now prove the other direction of `thm`, namely that $\Box \vdash A \Rightarrow \vdash_{\text{sk}} A$ by using $\vdash \rightarrow \vdash_{\text{skC}}$ in the special case of the empty context and combining it with `skC  $\rightarrow$  sk`:

$$\begin{aligned} \text{thm} &: (\Box \vdash A) \Leftrightarrow \vdash_{\text{sk}} A \\ \text{proj}_1 \text{ thm d} &= \text{skC} \rightarrow \text{sk } (\vdash \rightarrow \vdash_{\text{skC}} d) \\ \text{proj}_2 \text{ thm SS} &= \text{S-d} \\ \text{proj}_2 \text{ thm KK} &= \text{K-d} \\ \text{proj}_2 \text{ thm (app d e)} &= \text{app (proj}_2 \text{ thm d) (proj}_2 \text{ thm e)} \end{aligned}$$

We can apply the left to right direction to our example to derive a combinatory proof of the example derivation:

$$\begin{aligned} \text{d-C-sk} &: \vdash \text{sk } (((\text{Var "Q"}) [\Rightarrow] (\text{Var "R"})) [\Rightarrow] ((\text{Var "P"}) [\Rightarrow] (\text{Var "Q"})) \\ &\quad [\Rightarrow] (\text{Var "P"}) [\Rightarrow] (\text{Var "R"})) \\ \text{d-C-sk} &= \text{proj}_1 \text{ thm d-C} \end{aligned}$$

The derivation we obtain is truly huge:

[illegible]

Indeed, we have implemented only the inefficient algorithm which is unsuitable when translating by hand. In section 2.4 we informally described a more efficient version which is more useful when doing the translation by hand. I leave it as an exercise to implement a modified version of `lam-sk` which implements the more efficient translation.

8.4 Kripke models

The naive semantics we introduced in section 8.2 isn't good enough to prove the non-derivability of propositions which are classically true such as the Pierce formula $((P \rightarrow Q) \rightarrow P) \rightarrow P$, this can be easily seen by checking the truth table.

It is less obvious that we cannot derive it. We can show that it entails the excluded middle

$$\text{Pierce} = \{P \rightarrow Q : \text{prop}\} \rightarrow ((P \rightarrow Q) \rightarrow P) \rightarrow P$$

$\text{pierce} \rightarrow \text{tnd} : \text{Pierce} \rightarrow \{P : \text{prop}\} \rightarrow \text{TND } P$
 $\text{pierce} \rightarrow \text{tnd } \text{pierce } \{P\} = \text{pierce } \{\text{TND } P\} \{\perp\} (\lambda p \rightarrow \text{case} \perp (\text{nntnd } p))$

which strongly suggests that we cannot prove it but this doesn't amount to a proof.

To address this problem we need to come up with a better semantics, one which is actually complete that is for any underivable proposition we can construct a semantic counterexample. Or more positively: if a formula is true in the semantics then it is also derivable. This is the inverse of soundness which just stated that derivable propositions are true in the semantics.

A semantics which is complete for the fragment of intuitionistic propositional logic we have presented are Kripke models. The basic idea is that we have different worlds which correspond to stages of knowledge. In each world we know that certain atomic propositions hold and if we expand our knowledge we may learn that additional propositions are true but one which was true will remain so. The interesting point is how to interpret implication. We say that $P \rightarrow Q$ holds in a world if whenever in the future I will find out P then at this point Q will also hold.

Using this idea we can present a countermodel using just two worlds: lets call them w_0 and w_1 . w_1 is a future of w_0 , we write $w_1 \sqsubseteq w_0$ and while we know nothing in w_0 , we know P in w_1 , we write $w_1 \Vdash P$. Now we observe that $(P \rightarrow Q) \rightarrow P$ holds in w_0 , we write $w_0 \Vdash (P \rightarrow Q) \rightarrow P$ because in w_0 Q doesn't hold hence the implication holds trivially and in the future w_1 we actually know P hence the formula holds. However, since w_0 doesn't force P , the Pierce formula doesn't hold in w_0 . Hence we have found a countermodel.

Ok, lets do this a bit more formally: A Kripke model is given by

record Kripke : Set₁ **where**
field
 Worlds : Set
 $_ \sqsubseteq _ : \text{Worlds} \rightarrow \text{Worlds} \rightarrow \text{prop}$
 $\text{refl} \sqsubseteq : \{w : \text{Worlds}\} \rightarrow w \sqsubseteq w$
 $\text{trans} \sqsubseteq : \{w_0 w_1 w_2 : \text{Worlds}\} \rightarrow w_0 \sqsubseteq w_1 \rightarrow w_1 \sqsubseteq w_2 \rightarrow w_0 \sqsubseteq w_2$

Here $_ \sqsubseteq _$ is the future relation, we write $w_1 \sqsubseteq w_0$ to mean that w_1 is a future of w_0 . We assume that the future relation is reflexive and transitive, i.e. it is a preorder.

$_ \Vdash _ : \text{Worlds} \rightarrow \text{String} \rightarrow \text{prop}$
 $\text{mon} \Vdash : \{w_0 w_1 : \text{Worlds}\} \rightarrow w_1 \sqsubseteq w_0 \rightarrow \{x : \text{String}\} \rightarrow w_0 \Vdash x \rightarrow w_1 \Vdash x$

The relation $w \Vdash x$ (often called *forcing*) expresses that the propositional variable x presented by a string holds in world w . This should be monotone, any variable we know holds now will still be valid in the future.

This completes the definition of a Kripke model. Now we assume as given a Kripke model $K : \text{Kripke}$ and we are going to extend $_ \Vdash _$ to formulas and

context and uses this to express what it means that a sequent $\Gamma \vdash A$ is valid in K . We define $_ \Vdash F _$ using the interpretation of $_ [\Rightarrow] _$ suggested earlier:

$$\begin{aligned} _ \Vdash F _ &: \text{Worlds} \rightarrow \text{Form} \rightarrow \text{prop} \\ w \Vdash F \text{Var } x &= w \Vdash x \\ w \Vdash F A [\Rightarrow] B &= \{w' : \text{Worlds}\} \rightarrow w' \sqsubseteq w \rightarrow w' \Vdash F A \rightarrow w' \Vdash F B \end{aligned}$$

We extend $_ \Vdash _$ to contexts, which simply says that it should hold for all formulas in the context. As in the naive semantics we are interpreting a context as the conjunction of all the formulas it contains.

$$\begin{aligned} _ \Vdash C _ &: \text{Worlds} \rightarrow \text{Con} \rightarrow \text{prop} \\ w \Vdash C [] &= \text{True} \\ w \Vdash C (A :: \Gamma) &= w \Vdash F A \wedge w \Vdash C \Gamma \end{aligned}$$

We write $\Gamma \models A$ to express that semantically Γ entails A , i.e. every world that forces Γ also forces A :

$$\begin{aligned} _ \models _ &: \text{Con} \rightarrow \text{Form} \rightarrow \text{prop} \\ \Gamma \models A &= \{w : \text{Worlds}\} \rightarrow w \Vdash C \Gamma \rightarrow w \Vdash F A \end{aligned}$$

We can now show soundness by induction over $\Gamma \vdash A$:

$$\llbracket _ \rrbracket : \Gamma \vdash A \rightarrow \Gamma \models A$$

I leave the details as an exercise. To show soundness you first need to show that $_ \Vdash F _$ and $_ \Vdash C _$ are monotone.

Now to complete the proof of the non-derivability of the Pierce formula we have to formally define the Kripke model we sketched earlier.

```

module Pierce where
  data W : Set where
    w0 w1 : W
  data  $\_ \sqsubseteq P \_$  : W  $\rightarrow$  W  $\rightarrow$  prop where
    refl : {w : W}  $\rightarrow$  w  $\sqsubseteq P$  w
    w10 : w1  $\sqsubseteq P$  w0
  data  $\_ \Vdash P \_$  : W  $\rightarrow$  String  $\rightarrow$  prop where
    w1p : w1  $\Vdash P$  "P"

```

I leave it as an exercise to complete the definition of the Kripke model. We can then show that the Pierce formula is not forced in w_0 :

$$\text{not} \Vdash \text{Pierce} : \neg (w_0 \Vdash F (((\text{Var } "P" [\Rightarrow] \text{Var } "Q") [\Rightarrow] \text{Var } "P") [\Rightarrow] \text{Var } "P"))$$

and hence using soundness it is not provable:

$$\text{no-Pierce-deriv} : \neg ([\] \vdash (((\text{Var } "P" [\Rightarrow] \text{Var } "Q") [\Rightarrow] \text{Var } "P") [\Rightarrow] \text{Var } "P"))$$

8.5 Completeness

Ok, Kripke models are good enough to refute the Pierce formula but are they good enough to refute any formula that is not derivable? This means are they complete? Indeed we can construct one Kripke model, the universal model, which models exactly derivability. Hence if a formula holds in all Kripke models then it will also hold in this one and hence it is derivable.

We construct this model from our derivation relation, that is we turn derivability into a Kripke model. The future relation we obtain by extending derivability on contexts, we define $\Gamma \vdash_C \Delta$, if all the formulas in Δ are derivable from Γ . Formally this can be defined as follows:

```
data  $\vdash_C$  : Con  $\rightarrow$  Con  $\rightarrow$  Set where
  [] :  $\Gamma \vdash_C$  []
   $\_::\_ :$   $\Gamma \vdash A \rightarrow \Gamma \vdash_C \Delta \rightarrow \Gamma \vdash_C (A :: \Delta)$ 
```

The idea is that $\vdash_C$ corresponds to parallel substitutions. That is an element $\Gamma \vdash_C A_0, A_1, \dots, A_n$ is a sequence of derivations $a_0 :: a_1 :: \dots :: a_n$ with $\Gamma \vdash a_i : A_i$. We have an identity substitution:

```
idC :  $\Gamma \vdash_C \Gamma$ 
```

and we can *cut* a term and a substitution:

```
cut :  $\Gamma \vdash_C \Delta \rightarrow \Delta \vdash A \rightarrow \Gamma \vdash A$ 
```

Iterating this operation provides us with a cut for $\vdash_C$ corresponding to the transitivity of the future relation:

```
cutC :  $\Gamma \vdash_C \Delta \rightarrow \Delta \vdash_C \Theta \rightarrow \Gamma \vdash_C \Theta$ 
```

The forcing relation for our universal model is just derivability applied to variables and monotonicity is just an instance of cut. This completes the construction of the universal Kripke model - the details are again left as an exercise:

```
UM : Kripke
UM = record
  { Worlds = Con
  ;  $\sqsubseteq$  =  $\vdash_C$ 
  ; refl $\sqsubseteq$  = idC
  ; trans $\sqsubseteq$  = cutC
  ;  $\Vdash$  =  $\lambda \Gamma x \rightarrow \Gamma \vdash \text{Var } x$ 
  ; mon $\Vdash$  =  $\lambda \gamma a \rightarrow \text{cut } \gamma a$ 
  }
```

The derived forcing relation $\Vdash_F$ for this model has the same type as the derivation relation. Hence the key lemma is to establish that forcing and derivability coincide. This can be shown by mutual induction over the formula A :

$$\begin{aligned} q &: \Gamma \Vdash F A \rightarrow \Gamma \vdash A \\ u &: \Gamma \vdash A \rightarrow \Gamma \Vdash F A \end{aligned}$$

I use q for *quote* since it goes from the semantics into the syntax, and u for *unquote*. Using these lemmas it is not hard to derive completeness:

$$\text{compl} : \Gamma \vdash A \rightarrow \Gamma \models A$$

8.6 Normalisation

There is an alternative way to establish non-provability which corresponds more to our naive reasoning. We only consider derivations which are normal, which means that they avoid derivations which can be β -reduced, i.e. that correspond to λ -terms of the form $(\lambda x \rightarrow M) N$ and which always use λ to prove an implication. The latter corresponds to always η -expanding and is called the long η normal form.

Now when we try to prove the Pierce formula

$$\begin{aligned} p &: ((P \rightarrow Q) \rightarrow P) \rightarrow P \\ p &= ? \end{aligned}$$

the first step has to be a λ :

$$p = \lambda x \rightarrow ?$$

with $x : (P \rightarrow Q) \rightarrow P$. Now our goal is P and the only assumption that produces something in P is x . Hence we have to refine by x :

$$p = \lambda x \rightarrow x ?$$

Ok our goal is now to prove $P \rightarrow Q$. Since this is an implication we need to use a λ :

$$p = \lambda x \rightarrow x (\lambda y \rightarrow ?)$$

We now have a new assumption $y : P$ and our goal is Q . But now we are stuck because none of our assumptions produces Q s. Hence there is no normal proof.

It is clear that to use this idea to show that there is no derivation of $((P \rightarrow Q) \rightarrow P) \rightarrow P$ in the original system we need to prove that for any derivation $\Gamma \vdash A$ there is a normal derivation $\Gamma \vdash_{\text{nf}} A$.

Let's start with defining normal derivations inductively. First of all we need to recover variables, which correspond to using an assumption from the context.

$$\begin{aligned} \text{data } _ \vdash_v _ &: \text{Con} \rightarrow \text{Form} \rightarrow \text{Set} \text{ where} \\ \text{zero} &: (A :: \Gamma) \vdash_v A \\ \text{suc} &: \Gamma \vdash_v A \rightarrow (B :: \Gamma) \vdash_v A \end{aligned}$$

This derivation correspond to numbers, indeed they are de Bruijn indices which we have discussed earlier.

We now mutually define three kinds of derivations:

```
data _ $\vdash$ nf_ : Con  $\rightarrow$  Form  $\rightarrow$  Set
data _ $\vdash$ ne_ : Con  $\rightarrow$  Form  $\rightarrow$  Set
data _ $\vdash$ sp_,_ : Con  $\rightarrow$  Form  $\rightarrow$  Form  $\rightarrow$  Set
```

Here $_ \vdash \text{nf} _$ are the normal derivations, $_ \vdash \text{ne} _$ are *neutral derivations* this is they are derivations which are a variable applied to a sequence of normal terms, and $_ \vdash \text{sp} _, _$ is the *spine* of a neutral derivation, i.e. the sequence of normal terms to which the variable is applies. The spine derivation has two formula parameters: the first one is the assumption we start with and the 2nd one is the goal.

So let's look at $_ \vdash \text{sp} _, _$ first:

```
data _ $\vdash$ sp_,_ where
  [] :  $\Gamma \vdash \text{sp} \ C, C$ 
  _::_ :  $\Gamma \vdash \text{nf} \ A \rightarrow \Gamma \vdash \text{sp} \ B, C \rightarrow \Gamma \vdash \text{sp} \ (A [\Rightarrow] B), C$ 
```

Once we have reached the end of the sequence the goal has to agree with the premise. The $_ :: _$ constructor shortens the distance from the premise $A [\Rightarrow] B$ to B by providing $\Gamma \vdash \text{nf} \ A$.

The neutral derivations are simply given by a variable applied to a spine:

```
data _ $\vdash$ ne_ where
  ne :  $\Gamma \vdash v \ A \rightarrow \Gamma \vdash \text{sp} \ A, B \rightarrow \Gamma \vdash \text{ne} \ B$ 
```

Finally we can define normal derivations:

```
data _ $\vdash$ nf_ where
  ne $\rightarrow$ nf :  $\Gamma \vdash \text{ne} \ \text{Var } x \rightarrow \Gamma \vdash \text{nf} \ (\text{Var } x)$ 
  lam :  $(A :: \Gamma) \vdash \text{nf} \ B \rightarrow \Gamma \vdash \text{nf} \ (A [\Rightarrow] B)$ 
```

A normal derivation of an implication is always given by `lam` but when our goal is a variable we have to resort to neutral derivations.

We can now show that there is no normal derivation of the Pierce formula just by using pattern matching:

```
no-Pierce-nf-deriv :
   $\neg ([\ ] \vdash \text{nf} \ (((\text{Var } "P" [\Rightarrow] \text{Var } "Q") [\Rightarrow] \text{Var } "P") [\Rightarrow] \text{Var } "P")))$ 
```

But as usual I leave this as an exercise.

Now let's prove normalisation. Actually we have already done it more or less in the previous section when we proved completeness. The basic idea is to combine soundness and completeness. That is using soundness we go from a derivation to the semantics in the universal Kripke model. Using completeness we can go back to a derivation but this wouldn't tell us anything new. However,

we can observe that the completeness prove always returns a normal derivation! Hence by combining soundness and completeness we can prove normalisation.

The Kripke model we are using is just a variation of the universal model from the last section. The future relation just uses variables since normal derivations are not closed under general substitutions but only under renamings

```
data  $\vdash_v C$  : Con  $\rightarrow$  Con  $\rightarrow$  Set where
  [] :  $\Gamma \vdash_v C$  []
  _::_ :  $\Gamma \vdash_v A \rightarrow \Gamma \vdash_v C \Delta \rightarrow \Gamma \vdash_v C (A :: \Delta)$ 
```

We need to show that we can establish identity and cut operations for variable substitutions:

```
idvC :  $\Gamma \vdash_v C \Gamma$ 
cutvC :  $\Gamma \vdash_v C \Delta \rightarrow \Delta \vdash_v C \Theta \rightarrow \Gamma \vdash_v C \Theta$ 
cutne :  $\Gamma \vdash_v C \Delta \rightarrow \Delta \vdash_{ne} A \rightarrow \Gamma \vdash_{ne} A$ 
```

the proofs are the same as before but now applied to the refined derivation forms. Hence we get another Kripke model:

```
NM : Kripke
NM = record
  { Worlds = Con
  ;  $\sqsubseteq$  =  $\vdash_v C$ 
  ; refl $\sqsubseteq$  = idvC
  ; trans $\sqsubseteq$  = cutvC
  ;  $\Vdash$  =  $\lambda \Gamma x \rightarrow \Gamma \vdash_{ne} \text{Var } x$ 
  ; mon $\Vdash$  =  $\lambda \gamma a \rightarrow \text{cutne } \gamma a$ 
  }
```

We also need to refine the types of q and u :

```
q :  $\Gamma \Vdash_F A \rightarrow \Gamma \vdash_{nf} A$ 
u :  $\Gamma \vdash_{ne} A \rightarrow \Gamma \Vdash_F A$ 
```

And using q and soundness ($\llbracket _ \rrbracket$) we can prove normalisation:

```
norm :  $\Gamma \vdash A \rightarrow \Gamma \vdash_{nf} A$ 
```

Putting norm and no-Pierce-nf-deriv together we derive an alternative proof of the non-derivability of the Pierce formula”

```
no-Pierce-deriv :  $\neg (\llbracket \_ \rrbracket \vdash (((\text{Var "P"} \Rightarrow \text{Var "Q"}) \Rightarrow \text{Var "P"}) \Rightarrow \text{Var "P"}))$ 
no-Pierce-deriv p = no-Pierce-nf-deriv (norm p)
```

So in a way normalisation is a canned completeness proof where the semantic component is replaced by a purely syntactic definition of normal forms.

8.7 History

Natural deduction was introduced by Gentzen in the 1930ies [Gen35, Gen35]. In his presentation he didn't use explicit contexts as we did but marked assumptions in derivations and linked them to the position where they are discharged. He also presented the system for predicate logic and covered all logical connectives. Gentzen also introduced a different calculus, the sequent calculus, as a tool to prove his *Hauptsatz* which corresponds to the normalisation theorem we have discussed.

We have already discussed the history of combinatory logic in section 2.6. The completeness proof was first sketched by Schönfinkel [Sch24] it can be found in [Cur30] and in the form we have presented it was first given by Rosenbloom [Ros50] (according to [CH06]).

Kripke models were first introduced for modal logic [Kri59]. The application to intuitionistic logic is discussed by Schütte in [Sch68]. The completeness theorem in the form we have presented is only provable for the fragment including only $\rightarrow$, $\wedge$, True and $\forall$. To include the remaining connectives, i.e. False , $\vee$, $\exists$ one has to use Beth models [Bet48] (as described in [TvU99]) or resort to a classical metatheory. In general one finds a good overview over this kind of material in the books by Troelstra and van Dalen [TvD88b, TvD88a].

My presentation of normalisation is inspired by *Normalisation by Evaluation* which was discovered by Schwichtenberg and Berger [BS91]. The relation between this algorithm and the completeness proof for Kripke models was noted in [AHS95].

8.8 Exercises

1. Implement the efficient algorithm for translating λ -terms into combinatory logic which was described in section 2.4. That is
 - If the variable x does not appear in M then $\lambda x \rightarrow M$ is just $K M$.
 - $\lambda x \rightarrow g x$ is just g .

You need to reimplement:

$$\text{lam-sk-x} : (A :: \Gamma) \vdash_{\text{skC}} B \rightarrow \Gamma \vdash_{\text{skC}} (A [\Rightarrow] B)$$

The rest stays basically the same — we only replace one direction of thm :

$$\begin{aligned} \vdash \rightarrow \vdash_{\text{skC-x}} & : \Gamma \vdash A \rightarrow \Gamma \vdash_{\text{skC}} A \\ \vdash \rightarrow \vdash_{\text{skC-x}} \text{zero} & = \text{zero} \\ \vdash \rightarrow \vdash_{\text{skC-x}} (\text{suc } d) & = \text{suc } (\vdash \rightarrow \vdash_{\text{skC-x}} d) \\ \vdash \rightarrow \vdash_{\text{skC-x}} (\text{lam } d) & = \text{lam-sk-x } (\vdash \rightarrow \vdash_{\text{skC-x}} d) \\ \vdash \rightarrow \vdash_{\text{skC-x}} (\text{app } d e) & = \text{app } (\vdash \rightarrow \vdash_{\text{skC-x}} d) (\vdash \rightarrow \vdash_{\text{skC-x}} e) \\ \text{thm-x} : (\Box \vdash A) & \rightarrow \vdash_{\text{sk}} A \\ \text{thm-x } d & = \text{skC} \rightarrow \text{sk } (\vdash \rightarrow \vdash_{\text{skC-x}} d) \end{aligned}$$

Try the improved version on

```
d-C-sk-x : ⊢sk (((Var "Q") [⇒] (Var "R")) [⇒] ((Var "P") [⇒] (Var "Q")))
           [⇒] ((Var "P") [⇒] (Var "R")))
d-C-sk-x = thm-x d-C
```

the result should be:

```
app (app SS (app KK SS)) KK
```

2. Complete the soundness proof for Kripke models from section 8.4. That is prove the following propositions:

```
mon-F : w1 ⊆ w0 → {A : Form} → w0 ⊨F A → w1 ⊨F A
mon-C : w1 ⊆ w0 → {Γ : Con} → w0 ⊨C Γ → w1 ⊨C Γ
[[_]] : Γ ⊢ A → Γ ⊨ A
```

3. Complete the proof that the Pierce formula is not derivable using the soundness result and by completing the definition of the Kripke counter-model.

Complete the definition of the Kripke model KP:

```
trans⊆P : {w0 w1 w2 : W} → w0 ⊆P w1 → w1 ⊆P w2 → w0 ⊆P w2
mon⊨P : {w0 w1 : W}
       → w1 ⊆P w0 → {x : String} → w0 ⊨P x → w1 ⊨P x
KP : Kripke
KP = record
  { Worlds = W
  ; _⊆_ = _⊆P_
  ; refl⊆ = refl
  ; trans⊆ = trans⊆P
  ; _⊨_ = _⊨P_
  ; mon⊨ = mon⊨P
  }
```

And then using KP show that Pierce is not derivable:

```
not⊨Pierce : ¬ (w0 ⊨F (((Var "P" [⇒] Var "Q") [⇒] Var "P") [⇒] Var "P")))
no-Pierce-deriv : ¬ ([] ⊢ (((Var "P" [⇒] Var "Q") [⇒] Var "P") [⇒] Var "P"))
```

4. Complete the completeness proof from section 8.5, that is complete the definition of the universal Kripke model:

```
idC : Γ ⊢C Γ
cut : Γ ⊢C Δ → Δ ⊢ A → Γ ⊢ A
```

```

cutC :  $\Gamma \vdash C \Delta \rightarrow \Delta \vdash C \Theta \rightarrow \Gamma \vdash C \Theta$ 
UM : Kripke
UM = record
  { Worlds = Con
  ;  $\sqsubseteq$  =  $\vdash C$ 
  ; refl $\sqsubseteq$  = idC
  ; trans $\sqsubseteq$  = cutC
  ;  $\Vdash$  =  $\lambda \Gamma x \rightarrow \Gamma \vdash \text{Var } x$ 
  ; mon $\Vdash$  =  $\lambda \gamma a \rightarrow \text{cut } \gamma a$ 
  }

```

Hint: It is useful to establish the following lemmas generalising weakening:

```

sucC :  $\Gamma \vdash C \Delta \rightarrow (B :: \Gamma) \vdash C \Delta$ 
extC :  $\Gamma \vdash C \Delta \rightarrow (A :: \Gamma) \vdash C (A :: \Delta)$ 

```

```

q :  $\Gamma \Vdash F A \rightarrow \Gamma \vdash A$ 
u :  $\Gamma \vdash A \rightarrow \Gamma \Vdash F A$ 
qC :  $\Gamma \Vdash C \Delta \rightarrow \Gamma \vdash C \Delta$ 
compl :  $\Gamma \vdash A \rightarrow \Gamma \models A$ 

```

5. Show that there is no normal derivation of the Pierce formula:

```

no-Pierce-nf-deriv :  $\neg ([\ ] \vdash \text{nf } (((\text{Var "P"} [\Rightarrow] \text{Var "Q"}) [\Rightarrow] \text{Var "P"}) [\Rightarrow] \text{Var "P"}))$ 

```

6. Complete the construction of the Kripke model using normal forms as described in section 8.6. That is complete the following definitions by adapting the construction of the previous question:

```

idvC :  $\Gamma \vdash vC \Gamma$ 
cutvC :  $\Gamma \vdash vC \Delta \rightarrow \Delta \vdash vC \Theta \rightarrow \Gamma \vdash vC \Theta$ 
cutne :  $\Gamma \vdash vC \Delta \rightarrow \Delta \vdash ne A \rightarrow \Gamma \vdash ne A$ 
NM : Kripke
NM = record
  { Worlds = Con
  ;  $\sqsubseteq$  =  $\vdash vC$ 
  ; refl $\sqsubseteq$  = idvC
  ; trans $\sqsubseteq$  = cutvC
  ;  $\Vdash$  =  $\lambda \Gamma x \rightarrow \Gamma \vdash ne \text{Var } x$ 
  ; mon $\Vdash$  =  $\lambda \gamma a \rightarrow \text{cutne } \gamma a$ 
  }

```

Once this is done we can derive the variations of q and u :

$$\begin{aligned} q &: \Gamma \Vdash F A \rightarrow \Gamma \vdash_{nf} A \\ u &: \Gamma \vdash_{ne} A \rightarrow \Gamma \Vdash F A \end{aligned}$$

Finally use this to derive the normalisation function:

$$\text{norm} : \Gamma \vdash A \rightarrow \Gamma \vdash_{nf} A$$

which can be use to provide an alternative proof of the non derivability of the Pierce formula:

open NormPierce

$$\begin{aligned} \text{no-Pierce-deriv} &: \neg (\Box \vdash (((\text{Var "P"} [\Rightarrow] \text{Var "Q"}) [\Rightarrow] \text{Var "P"}) [\Rightarrow] \text{Var "P"})) \\ \text{no-Pierce-deriv } p &= \text{no-Pierce-nf-deriv } (\text{norm } p) \end{aligned}$$

Chapter 9

Eliminators and coinduction

Definitions by pattern matching and recursion are a convenient way to define functions out of inductive types but it is hard to make precise exactly what is allowed and what isn't. Also to define pattern matching precisely is not so easy and this makes it hard to use this for metatheoretic study along the lines of what we have done in the last chapter.

Previously, in the simply typed setting we have introduced the iterator and the recursor as combinators which cover recursive definitions (see section 4.2). In this chapter we will look at the extension to dependent functions using a more general combinator called the *eliminator*. We will also look at iterators and eliminators for inductive families, especially for equality.

Finally we will look at coinductive definitions but notice that the mirror in the type theory we have introduced so far is rather imperfect. To prove properties of coinductive definition we need *coinduction* which doesn't generalize coiteration but worse which cannot be given a computational semantics in the type theory we are using, which is *intensional type theory*. We will fix this later when we are introducing cubical type theory.

9.1 The eliminator for natural numbers

Ok, the point of the eliminator is to explain how we can define functions out of an inductive type, which is called elimination in contrast to introduction which is achieved by the constructors. Let's take the natural numbers as an example. We construct them using `zero` and `suc`.

When defining a dependent function out of the natural numbers we assume as given a family $M : \mathbb{N} \rightarrow \text{Set}$ which we call the motive. We want to define

$$f : (n : \mathbb{N}) \rightarrow M\ n$$

We do this by pattern matching and recursion:

$$\begin{aligned} f\ \text{zero} &= z \\ f\ (\text{suc}\ n) &= s\ n\ (f\ n) \end{aligned}$$

where

$$\begin{aligned} z &: M \text{ zero} \\ s &: (n : \mathbb{N}) \rightarrow M n \rightarrow M (\text{suc } n) \end{aligned}$$

Note that s needs to see n because it is the index to M . Hence the eliminator behaves like the recursor in that we have to supply the natural number and not only the recursively computed result.

We can put this together into one constant, the eliminator:

$$\begin{aligned} \text{EN} &: (M : \mathbb{N} \rightarrow \text{Set}) \\ &\rightarrow M 0 \\ &\rightarrow ((n : \mathbb{N}) \rightarrow M n \rightarrow M (\text{suc } n)) \\ &\rightarrow (n : \mathbb{N}) \rightarrow M n \\ \text{EN } M \text{ z } s \text{ zero} &= z \\ \text{EN } M \text{ z } s (\text{suc } n) &= s n (\text{EN } M \text{ z } s n) \end{aligned}$$

You may notice that while the type is different the equations defining the eliminator are exactly the same as the one for the recursor.

Let's revisit some examples and translate them from pattern matching to using EN , e.g. there is rneutr+

$$\begin{aligned} \text{rneutr+} &: (n : \mathbb{N}) \rightarrow n + 0 \equiv n \\ \text{rneutr+ zero} &= \text{refl} \\ \text{rneutr+ (suc } n) &= \text{cong suc (rneutr+ } n) \end{aligned}$$

It is straightforward to turn this into an application of the eliminator:

$$\begin{aligned} \text{rneutr+-e} &: (n : \mathbb{N}) \rightarrow n + 0 \equiv n \\ \text{rneutr+-e} &= \text{EN } (\lambda n \rightarrow n + 0 \equiv n) \text{ refl } (\lambda n \text{ ih} \rightarrow \text{cong suc ih}) \end{aligned}$$

I have called the recursive result ih short for *induction hypothesis* and indeed the type of EN indeed corresponds to the induction principle for natural numbers.

However, not all uses of EN are of a logical nature, there are completely computational uses of it. One example we already looked it was:

$$\begin{aligned} \text{zeroes-v} &: (n : \mathbb{N}) \rightarrow \text{Vec } \mathbb{N} n \\ \text{zeroes-v zero} &= [] \\ \text{zeroes-v (suc } n) &= 0 :: (\text{zeroes-v } n) \end{aligned}$$

And again we can read of the parameters for the eliminator from the recursive definition:

$$\begin{aligned} \text{zeroes-v-e} &: (n : \mathbb{N}) \rightarrow \text{Vec } \mathbb{N} n \\ \text{zeroes-v-e} &= \text{EN } (\text{Vec } \mathbb{N}) [] (\lambda n \text{ v} \rightarrow 0 :: v) \end{aligned}$$

The recursor can be derived from the eliminator, all we need to do is to use a constant family as the motive:

$$\begin{aligned} \text{RN-E} &: \{M : \text{Set}\} \rightarrow M \rightarrow (\mathbb{N} \rightarrow M \rightarrow M) \rightarrow \mathbb{N} \rightarrow M \\ \text{RN-E } \{M\} &= \text{EN } (\lambda _ \rightarrow M) \end{aligned}$$

and again since the iterator is a special case of the recursor we get this as well:

$$\begin{aligned} \text{ltN-E} &: \{M : \text{Set}\} \rightarrow M \rightarrow (M \rightarrow M) \rightarrow \mathbb{N} \rightarrow M \\ \text{ltN-E } z \ s &= \text{RN-E } z \ (\lambda _ \rightarrow s) \end{aligned}$$

9.2 Iterator and eliminators for dependent types

OK, we have seen a dependent eliminator for a non-dependent type like the natural numbers. I'll leave it as an exercise to derive eliminators for the inductive types we discussed in 4.3, e.g. `List`, `Expr`, `RoseTree` and `Ord`. To have a bit more excitement we shall look at a dependent inductive type like `Vec`. As a reminder `Vec` was defined as follows:

```
data Vec : ℕ → Set where
  [] : Vec A 0
  _::_ : {n : ℕ} → A → Vec A n → Vec A (suc n)
```

However, before looking at the eliminator for `Vec`, maybe we should consider the iterator? But hang on, since `Vec` is a dependent type how can it have a non-dependent iterator? The answer is that the iterator will depend on the indices for vectors but not on the vector itself. That is we have a motive $M : \mathbb{N} \rightarrow \text{Set}$ and we want to define a function:

$$f : \{n : \mathbb{N}\} \rightarrow \text{Vec } A \ n \rightarrow M \ n$$

That is the function does depend on n but not on the vector itself. When we define this function by pattern matching we end up in the following situation:

$$\begin{aligned} f \{n\} [] &= m\text{-nil} \\ f \{suc\ n\} (a :: v) &= m\text{-cons } \{n\} \ a \ (f \{n\} \ v) \end{aligned}$$

with

```
m-nil   : M 0
m-cons : {n : ℕ} → A → M n → M (suc n)
```

That is as before we obtain the types of the methods by replacing the inductive type `Vec` by the motive M . And also as we are now used to we can turn this into the definition of the iterator:

$$\begin{aligned} \text{ltVec} &: (M : \mathbb{N} \rightarrow \text{Set}) \rightarrow M \ 0 \\ &\rightarrow (\{n : \mathbb{N}\} \rightarrow A \rightarrow M \ n \rightarrow M \ (suc \ n)) \\ &\rightarrow \{n : \mathbb{N}\} \rightarrow \text{Vec } A \ n \rightarrow M \ n \end{aligned}$$

$$\begin{aligned} \text{ltVec } M \text{ m-nil m-cons } [] &= \text{m-nil} \\ \text{ltVec } M \text{ m-nil m-cons } (a :: v) &= \text{m-cons } a (\text{ltVec } M \text{ m-nil m-cons } v) \end{aligned}$$

An example of a non-dependent function over vectors is the $_++v_$.

$$\begin{aligned} _++v_ : \{i j : \mathbb{N}\} &\rightarrow \text{Vec } A \ i \rightarrow \text{Vec } A \ j \rightarrow \text{Vec } A \ (i + j) \\ [] ++v w &= w \\ (x :: v) ++v w &= x :: (v ++v w) \end{aligned}$$

Ok, $_++v_$ is a dependent function but it doesn't depend on vectors but only on the indices. An indeed, we can define $_++v_$ using only ltVec :

$$\begin{aligned} _++v\text{-it}_ : \{i j : \mathbb{N}\} &\rightarrow \text{Vec } A \ i \rightarrow \text{Vec } A \ j \rightarrow \text{Vec } A \ (i + j) \\ _++v\text{-it}_ \{A = A\} \{i\} \{j\} v w &= \text{ltVec } (\lambda i \rightarrow \text{Vec } A \ (i + j)) w \\ &\quad (\lambda a \ v ++w \rightarrow a :: v ++w) v \end{aligned}$$

Ok, but when do we actually need dependent functions over vectors? I haven't yet given an example for such a thing but I can easily make one up. We may want to show that Vec with $[]$ and $_++v_$ form an indexed monoid.

In particular we may want to show that $[]$ is a right inverse:

$$\{n : \mathbb{N}\} (v : \text{Vec } A \ n) \rightarrow v ++v [] \equiv v$$

He we already encounter a problem: this statement doesn't type check! The left hand side has type $\text{Vec } A \ (n + 1)$ and the right hand side has type $\text{Vec } A \ n$. However, we have shown that they are equal $\text{rneutr}+ : (n : \mathbb{N}) \rightarrow n + 0 \equiv n$ (in section 6.4) and we can use subst to fix this issue:

$$\begin{aligned} \text{rneutr}++ : \{n : \mathbb{N}\} (v : \text{Vec } A \ n) \\ \rightarrow (\text{subst } (\text{Vec } A) (\text{rneutr}+ \ n) (v ++v [])) \equiv v \end{aligned}$$

and then just use pattern matching:

$$\begin{aligned} \text{rneutr}++ \{n\} \quad [] &= \text{refl} \\ \text{rneutr}++ \{\text{suc } n\} (x :: v) &= ? \end{aligned}$$

The second case is not completely straightforward because now the subst is on our way. There are more principled ways to deal with this situation but for the sake of brevity, I am just using a very special lemma that allows us to commute $_::_$ and subst :

$$\begin{aligned} \text{lem} : \{m n : \mathbb{N}\} (p : m \equiv n) (x : A) (v : \text{Vec } A \ m) \\ \rightarrow \text{subst } (\text{Vec } A) (\text{cong suc } p) (x :: v) \equiv x :: (\text{subst } (\text{Vec } A) p \ v) \\ \text{lem refl } x \ v = \text{refl} \end{aligned}$$

This allows us to complete the proof:

$$\begin{aligned} \text{rneutr}++ [] &= \text{refl} \\ \text{rneutr}++ \{A = A\} \{n = \text{suc } n\} (x :: v) &= \end{aligned}$$

```

begin
  subst (Vec A) (cong suc (rneutr+ n)) (x :: (v ++v []))
    ≡⟨ lem (rneutr+ n) x (v ++v []) ⟩
  x :: subst (Vec A) (rneutr+ n) (v ++v [])
    ≡⟨ cong (λ w → x :: w) (rneutr++ v) ⟩
  x :: v
  ■

```

We got a bit distracted on the way: the point was to derive an eliminator for `Vec`. In general our problem is that we want to derive a function that depends on vectors:

$$f : \{n : \mathbb{N}\} (v : \text{Vec } A \ n) \rightarrow M \{n\} \ v$$

where

$$M : \{n : \mathbb{N}\} (v : \text{Vec } A \ n) \rightarrow \text{Set}$$

We can achieve this by pattern matching:

$$\begin{aligned}
f \{0\} [] &= m\text{-nil} \\
f \{suc \ n\} (a :: v) &= m\text{-cons } \{n\} \ a \ v \ (f \{n\} \ v)
\end{aligned}$$

where

$$\begin{aligned}
m\text{-nil} &: M \{0\} [] \\
m\text{-cons} &: \{n : \mathbb{N}\} (a : A) (v : \text{Vec } A \ n) \rightarrow M \{n\} \ v \rightarrow M \{suc \ n\} (a :: v)
\end{aligned}$$

We turn the handle and derive an eliminator for `Vec`:

$$\begin{aligned}
E\text{Vec} &: (M : \{n : \mathbb{N}\} \rightarrow \text{Vec } A \ n \rightarrow \text{Set}) \\
&\rightarrow M \{0\} [] \\
&\rightarrow (\{n : \mathbb{N}\} (a : A) (v : \text{Vec } A \ n) \\
&\quad \rightarrow M \{n\} \ v \rightarrow M \{suc \ n\} (a :: v)) \\
&\rightarrow \{n : \mathbb{N}\} (v : \text{Vec } A \ n) \rightarrow M \{n\} \ v \\
E\text{Vec } M \ m\text{-nil } m\text{-cons} [] &= m\text{-nil} \\
E\text{Vec } M \ m\text{-nil } m\text{-cons} (a :: v) &= m\text{-cons } a \ v \ (E\text{Vec } M \ m\text{-nil } m\text{-cons } v)
\end{aligned}$$

And now we can use our example to cook up a use case for `EVec`:

$$\begin{aligned}
rneutr++\text{-e} &: \{n : \mathbb{N}\} (v : \text{Vec } A \ n) \\
&\rightarrow (\text{subst } (\text{Vec } A) \ (\text{rneutr+ } n) \ (v ++v [])) \equiv v \\
rneutr++\text{-e } \{A = A\} &= \\
E\text{Vec } (\lambda \{n\} \ v \rightarrow (\text{subst } (\text{Vec } A) \ (\text{rneutr+ } n) \ (v ++v []))) &\equiv v \\
\text{refl} & \\
\lambda \{n\} \ x \ v \ h \rightarrow \text{trans } (\text{lem } (\text{rneutr+ } n) \ x \ (v ++v [])) & \\
&\quad (\text{cong } (\lambda w \rightarrow x :: w) \ (\text{rneutr++ } v))
\end{aligned}$$

9.3 Eliminators for equality

9.3.1 The recursor

The previous section was mainly a built-up for this one where we actually deal with a much simpler dependent inductive type : equality ($\equiv$). Here is how we defined equality as an inductive type in section 6.3:

```
data  $\equiv$  : A → A → Set where
  refl : {a : A} → a  $\equiv$  a
```

We have only one constructor `refl` and it isn't even recursive. Without further ado we can write down the iterator:

```
It $\equiv$  : (M : A → A → Set)
  → ({a : A} → M a a)
  → {a b : A} → a  $\equiv$  b → M a b
It $\equiv$  M r refl = r
```

We can verify the properties of equality we have shown in 6.3 using only the iterator. As an example here is the proof of symmetry using pattern matching:

```
sym : {a b : A} → a  $\equiv$  b → b  $\equiv$  a
sym refl = refl
```

And it is easy to translate this into a proof using `It $\equiv$` :

```
sym-it : {a b : A} → a  $\equiv$  b → b  $\equiv$  a
sym-it = It $\equiv$  ( $\lambda$  a b → b  $\equiv$  a) refl
```

We can also prove `trans`, `cong` and `subst` using only the iterator because they don't depend on equality proofs.

9.3.2 J and K

The eliminator for equality has a name, it was called J by Martin-Löf and hence we shall use J instead of $E\equiv$. Here we go:

```
J : (M : {a b : A} → a  $\equiv$  b → Set)
  → ({a : A} → M {a} {a} refl)
  → {a b : A} (p : a  $\equiv$  b) → M {a} {b} p
J M m-refl refl = m-refl
```

But what can we prove with J what we couldn't already prove with `It $\equiv$` ? We need some type which depends on equality proofs. So far we have only seen one, uniqueness of equality proofs ¹

¹Which is traditionally called *uip* for uniqueness of identity proofs.

$$\begin{aligned} \text{uip} &: \{x\ y : A\} (p\ q : x \equiv y) \rightarrow p \equiv q \\ \text{uip refl refl} &= \text{refl} \end{aligned}$$

Now we should hope that we can derive `uip` from `J`, actually two instances since we pattern match twice. Ok let's start with replacing matching `p`:

$$\begin{aligned} \text{uip-e} &: \{x\ y : A\} (p\ q : x \equiv y) \rightarrow p \equiv q \\ \text{uip-e} &= J (\lambda \{x\} \{y\} p \rightarrow (q : x \equiv y) \rightarrow p \equiv q) ? \end{aligned}$$

We are left with having to show $\{a : A\} (q : a \equiv a) \rightarrow \text{refl} \equiv q$ but this turns out tricky because it is not an instance of `J`. `J` is only applicable to define functions of the form $\{x\ y : A\} (p : x \equiv y) \rightarrow M \{x\} \{y\} p$ but not goals of the form $\{x : A\} (p : x \equiv x) \rightarrow M \{x\} p$. We may hope that there is a way to massage the goal so that we can use `J` to prove it. When I was doing my PhD we tried and failed and it was Martin Hofmann and Thomas Streicher who proved that it is indeed impossible. But before talking about this, let's notice that there is an obvious and cheap solution. We simply need another eliminator which handles the case I have just described, and this is called `K`. I hasten to add that this has nothing to do with the `K` from combinatory logic and it was just chosen because it is the next letter after `J`.

$$\begin{aligned} K &: (M : \{a : A\} \rightarrow a \equiv a \rightarrow \text{Set}) \\ &\rightarrow (\{a : A\} \rightarrow M \{a\} \text{refl}) \\ &\rightarrow \{a : A\} (p : a \equiv a) \rightarrow M \{a\} p \\ K\ M\ m\text{-refl}\ \text{refl} &= m\text{-refl} \end{aligned}$$

Using `K` it is easy to complete `uip-e`:

$$\begin{aligned} \text{uip-e} &: \{x\ y : A\} (p\ q : x \equiv y) \rightarrow p \equiv q \\ \text{uip-e} &= J (\lambda \{x\} \{y\} p \rightarrow (q : x \equiv y) \rightarrow p \equiv q) \\ &\quad (K (\lambda \{a\} q \rightarrow \text{refl} \equiv q) \text{refl}) \end{aligned}$$

On the other hand we can derive `K` from `uip` using only `subst` which is derivable from `J`:

$$\begin{aligned} K\text{-from-uip} &: (M : \{a : A\} \rightarrow a \equiv a \rightarrow \text{Set}) \\ &\rightarrow (\{a : A\} \rightarrow M \{a\} \text{refl}) \\ &\rightarrow \{a : A\} (p : a \equiv a) \rightarrow M \{a\} p \\ K\text{-from-uip}\ M\ m\text{-refl}\ \{a\}\ p &= \text{subst } (M \{a\}) (\text{uip refl } p) m\text{-refl} \end{aligned}$$

Indeed we need only an instance of `uip` namely `uip refl` which says that any equality proof showing $a = a$ is equal to `refl`: $\{x : A\} (q : x \equiv x) \rightarrow \text{refl} \equiv q$.

At the time I thought this was the deal: we just add `K` and everything is fine. Indeed Conor McBride showed in his PhD that all instances of pattern matching are derivable from the eliminators and `K`. Hence we have a combinatory way to say what we mean by pattern matching (ok we still need to address the question what is a good inductive type but this is another issue).

However, then came Voevodsky — actually this is not completely true this was already pointed out by Martin Hofmann much earlier. The univalence principle characterises equality of types and an important consequence is that equality of types is given by equivalence which for most types boils down to having a bijection or an isomorphism (an invertible function). However, there is usually more than one bijection between types, e.g. there are two bijections showing $\mathbf{Bool} \cong \mathbf{Bool}$, namely identity and negation. In general there are $n!$ bijections between a finite type with n elements and itself.

So here is a choice: we either can have $\mathbf{K}$ or equivalently $\mathbf{uip}$ or we can have univalence but we can't have both. We will discuss univalence in more detail later but let's see what are the consequences of not having $\mathbf{K}$ which also provides some insight how Hofmann and Streicher managed to show that it is not derivable.

Maybe we should ask the question: if we cannot prove $\mathbf{uip}$ what can we actually do with $\mathbf{J}$. Just using the recursor we can prove that equality is an equivalence relation, that is we have:

$$\begin{aligned} \text{refl} &: \{a : A\} \rightarrow a \equiv a \\ \text{sym} &: \{a\ b : A\} \rightarrow a \equiv b \rightarrow b \equiv a \\ \text{trans} &: \{a\ b\ c : A\} \rightarrow a \equiv b \rightarrow b \equiv c \rightarrow a \equiv c \end{aligned}$$

However, maybe we should use the word *relation* carefully because there is more to equality than just being inhabited, if we don't have $\mathbf{uip}$.

However, in this case we have more structure, we can show some algebraic laws. We have already seen monoids (e.g. the natural numbers with 0 and addition), and another important structure are groups, where we also have inverses (e.g. the integers with addition and unary negation). However, we don't have a group here because we have to take the indices into account but the structure we get is called a *groupoid*. Here are the laws of a groupoid in the specific case of equality:

$$\begin{aligned} \text{lneutr}\equiv &: \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans refl } p \equiv p \\ \text{rneutr}\equiv &: \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } p \text{ refl} \equiv p \\ \text{assoc}\equiv &: \{a\ b\ c\ d : A\} (p : a \equiv b) (q : b \equiv c) (r : c \equiv d) \\ &\rightarrow \text{trans } p (\text{trans } q\ r) \equiv \text{trans } (\text{trans } p\ q)\ r \\ \text{linv}\equiv &: \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } (\text{sym } p)\ p \equiv \text{refl} \\ \text{rinv}\equiv &: \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } p (\text{sym } p) \equiv \text{refl} \end{aligned}$$

Now $\text{lneutr}\equiv$ is just refl because we defined trans this way but the remaining ones are less trivial but provable using $\mathbf{J}$. For example $\text{rneutr}\equiv$:

$$\begin{aligned} \text{rneutr}\equiv &: \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } p \text{ refl} \equiv p \\ \text{rneutr}\equiv &= \mathbf{J} (\lambda \{a\} \{b\} p \rightarrow \text{trans } p \text{ refl} \equiv p) \text{ refl} \end{aligned}$$

I leave it as an exercise to prove the remaining ones. Actually it is easy to see since it is sufficient to replace p with refl which we can do using $\mathbf{J}$ since both ends of the equality p are variables, namely a and b .

Now Hofmann’s and Streicher’s main idea was that we can turn this around: not only does the equality type with only J gives rise to a groupoid but actually the laws of a groupoid are sufficient to derive J . That is we can model each type as an arbitrary groupoid and with this interpretation we can derive J . Naturally there are some technical details one needs to work out, in particular what is a model of type theory in terms of groupoids.

However, there is a further complication: the equality within the groupoid of equality types itself, that is equations like $\text{trans } p \text{ refl} \equiv p$ themselves do not in general satisfy uip . So indeed, we don’t have just a groupoid but a much more complicated beast called a ω -groupoid. Hence, Hofmann’s and Streicher’s groupoid model while sound is not complete in the sense that it entails uip for equalities of equality proofs and this is not provable in Type Theory.

9.3.3 Paulin-Mohring’s eliminator

The eliminator J can only be used to define functions of a type of the form

$$\{x : A\} (p : x \equiv y) \rightarrow C \{x\} \{y\} p$$

It doesn’t work when any side of the equality is fixed. We cannot do much if both sides are fixed because this would mean we can derive K but the situation is different if only one side is fixed as in

$$\{y : A\} (p : a \equiv y) \rightarrow C \{y\} p$$

for some fixed $a : A$.

Indeed, we can understand this as a different definition of equality: we fix $a : A$ and we define the predicate of being equal to a . We can write this down as follows

```
data  $\equiv'$  (a : A) : A → Set where
  refl : a  $\equiv'$  a
```

Here $a : A$ is a parameter we could as well have assumed it globally.

We note that $\equiv'$ looks very similar to $\equiv$: it has the same type $A \rightarrow A \rightarrow \text{Set}$ and the constructor refl has the same type $a \equiv a$ and $a \equiv' a$. However, the eliminator is different:

```
J' : (a : A)
      (M : {y : A} → a  $\equiv'$  y → Set)
      → (M {a} refl)
      → {y : A} (p : a  $\equiv'$  y) → M {y} p
J' a M m-refl refl = m-refl
```

If we want to compare the two types it makes sense to check whether their respective eliminators are derivable. One direction is easy, we can derive J for $\equiv'$ from J' .

$$\begin{aligned}
& \text{J-from-J}' : (M : \{a\ b : A\} \rightarrow a \equiv' b \rightarrow \text{Set}) \\
& \rightarrow (\{a : A\} \rightarrow M \{a\} \{a\} \text{refl}) \\
& \rightarrow \{a\ b : A\} (p : a \equiv' b) \rightarrow M \{a\} \{b\} p \\
& \text{J-from-J}' M \text{m-refl } \{a\} = \text{J}' a (M \{a\}) (\text{m-refl } \{a\})
\end{aligned}$$

All we needed to do is to specialise the motive M and the method m-refl . What about the other direction?

$$\begin{aligned}
& \text{J}'\text{-from-J} : (a : A) \\
& (M : \{y : A\} \rightarrow a \equiv y \rightarrow \text{Set}) \\
& \rightarrow (M \{a\} \text{refl}) \\
& \rightarrow \{y : A\} (p : a \equiv y) \rightarrow M \{y\} p
\end{aligned}$$

This is less obvious because we cannot just cook up motive for J' from a motive for M . But what about the iterator? Actually we have already seen the recursor for $_ \equiv' _$

$$\text{It}\equiv' : (a : A) (M : A \rightarrow \text{Set}) \rightarrow M a \rightarrow \{y : A\} \rightarrow (a \equiv y) \rightarrow M y$$

Indeed, if we just commute the parameters, we get a well known acquaintance

$$\text{subst-it} : (P : A \rightarrow \text{Set}) \rightarrow \{a\ b : A\} \rightarrow a \equiv b \rightarrow P a \rightarrow P b$$

which is indeed derivable from $\text{It}\equiv$

$$\text{subst-it } P = \text{It}\equiv (\lambda x\ y \rightarrow P\ x \rightarrow P\ y) (\lambda q \rightarrow q)$$

The other ingredient in the derivation of J' is an important principle which is called *contractibility of singletons*. Given $a : A$ the singleton represents the set of elements that is equal to a , we could write $\{y : A \mid a \equiv y\}$ that is using Σ : $\Sigma A (\lambda y \rightarrow a \equiv y)$. Now there is always at least one element in the singleton type, namely (a, refl) , but the important point is that this is the only one.

$$\begin{aligned}
& \text{contr-sgl} : (a : A) \rightarrow (p : \Sigma A (\lambda y \rightarrow a \equiv y)) \\
& \rightarrow (a, \text{refl}) \equiv p
\end{aligned}$$

That this is provable by J becomes obvious once we uncurry the Σ -type

$$\begin{aligned}
& \text{contr-sgl} : \{a\ y : A\} \rightarrow (q : a \equiv y) \\
& \rightarrow _ \equiv _ \{A = \Sigma A (\lambda y \rightarrow a \equiv y)\} (a, \text{refl}) (y, q) \\
& \text{contr-sgl} \{A\} = \\
& \text{J } (\lambda \{a\} \{y\} q \rightarrow _ \equiv _ \{A = \Sigma A (\lambda z \rightarrow a \equiv z)\} (a, \text{refl}) (y, q)) \\
& \text{refl}
\end{aligned}$$

How does this help us to derive J' ? Now if you look at the type of J' it says that once we know $M \{a\} \text{refl}$ we know $M \{y\} p$ for any $y : A$ and $p : a \equiv y$. Hell yes, all we need to do is to combine subst and contr-sgl .

$$\begin{aligned}
& \text{J}'\text{-from-J} \{A\} a M \text{m-refl } p = \\
& \text{subst } M' (\text{contr-sgl } p) \text{m-refl}
\end{aligned}$$

where $M' : (\Sigma A (\lambda y \rightarrow a \equiv y)) \rightarrow \text{Set}$
 $M'(y, q) = M \{y\} q$

Hence the two versions of equality are indeed equivalent, but the eliminator which comes from $\equiv'$ is more versatile because it allows one side of the equation to be fixed. This was noticed first by Christine Paulin-Mohring and is the standard definition of equality in the Coq system.

9.4 Coinduction for streams

We would hope that we could just hold up the categorical mirror and derive a dual version of the eliminator as we have done with coiterators and corecursors. However, the story is not as nice as we may have hoped but reveal a deep asymmetry in the kind of Type Theory we have studied so far.

Let's look at the streams again. Here is the definition from section 4.5.1:

```
record Stream (A : Set) : Set where
  constructor _::_
  coinductive
  field
    head : A
    tail : Stream A
```

The coiterator is defined as follows:

```
ColtStream : {W : Set} → (W → A) → (W → W)
           → W → Stream A
head (ColtStream h t x) = h x
tail (ColtStream h t x) = ColtStream h t (t x)
```

While the eliminator simply arose as a dependent version of the iterator / re-cursor this will not work in this case because the coinductive type is the output of the iterator.

The eliminator expressed the fact that all elements of an inductive type are created from the constructors. In the dual situation whatever we can do with a stream has to happen through the destructors, we cannot observe the actual definition of the stream using the coiterator. Hence we going to define identify two streams which are *bisimilar*, i.e. which behave the same wrt the destructors.

We use the notion of a *bisimulation* which is a relation, in this case on streams, that expresses that we can identify elements. A relation

$$\sim : \text{Stream } A \rightarrow \text{Stream } A \rightarrow \text{Set}$$

is a bisimulation if related streams $s \sim t$ have equal heads $\text{head } s \equiv \text{head } t$ and related tails $\text{tail } s \sim \text{tail } t$. The principle of coinduction says that elements that are bisimilar, that is there is a bisimulation that relates them are actually equal. We can make this precise as follows:

postulate

```

ColndStream : ( _ ~ _ : Stream A → Stream A → Set)
  → ({ s t : Stream A } → s ~ t → head s ≡ head t)
  → ({ s t : Stream A } → s ~ t → tail s ~ tail t)
  → { s t : Stream A } → s ~ t → s ≡ t

```

We can use `ColndStream` to prove equations about streams. We had previously defined

```

from : ℕ → Stream ℕ
head (from n) = n
tail (from n) = from (suc n)

```

and we now want to prove the following theorem: if we map `suc` to the stream of numbers starting with `n`, i.e. `from n` than this is the same stream as the stream starting with `suc n`, i.e. `from (suc n)`

To formally state the theorem, we need first that `Stream` is functorial, i.e. there is a map function. This is easy to define using copattern matching:

```

mapStream : (A → B) → Stream A → Stream B
head (mapStream f s) = f (head s)
tail (mapStream f s) = mapStream f (tail s)

```

I leave it as an exercise to derive `mapStream` from the coiterator. Now we can state the theorem formally:

```

thm : (n : ℕ) → from (suc n) ≡ mapStream suc (from n)

```

We are going to use coinduction to prove `thm`. In this instance we can read off the relation directly from the theorem:

```

data _ ~ _ : Stream ℕ → Stream ℕ → Set where
  ~-in : (n : ℕ) → from (suc n) ~ mapStream suc (from n)

```

We need to show that `_ ~ _` is indeed a bisimulation. This turns out to be very straightforward. First of all we note that the heads agree, that is

```

hd (from (suc n)) = hd (suc n :: from (suc (suc n)))
                  = suc n
hd (mapStream suc (from n)) = hd (mapStream suc (n :: from (suc n)))
                             = hd (suc n :: mapStream (from (suc n)))
                             = suc n

```

Hence the agda proof is just

```

hd~ : {s t : Stream ℕ} → s ~ t → head s ≡ head t
hd~ (~-in n) = refl

```

Next we need to show that if elements are related then their tails are also related. Ok, let's analyze the tails:

$$\begin{aligned}
 \text{tl } (\text{from } (\text{suc } n)) &= \text{tl } (\text{suc } n :: \text{from } (\text{suc } (\text{suc } n))) \\
 &= \text{from } (\text{suc } (\text{suc } n)) \\
 \text{tl } (\text{mapStream suc } (\text{from } n)) &= \text{tl } (\text{mapStream suc } (n :: \text{from } (\text{suc } n))) \\
 &= \text{tl } (\text{suc } n :: \text{mapStream } (\text{from } (\text{suc } n))) \\
 &= \text{mapStream } (\text{from } (\text{suc } n))
 \end{aligned}$$

Hence as we see the tails are an instance of the $\sim$ just by replacing n with $\text{suc } n$. Here is the agda code:

$$\begin{aligned}
 \text{tl}^\sim &: \{s\ t : \text{Stream } \mathbb{N}\} \rightarrow s \sim t \rightarrow \text{tail } s \sim \text{tail } t \\
 \text{tl}^\sim (\sim\text{-in } n) &= \sim\text{-in } (\text{suc } n)
 \end{aligned}$$

Hence putting everything together we can now prove thm:

$$\text{thm } n = \text{CoIndStream } \sim \text{hd}^\sim \text{tl}^\sim (\sim\text{-in } n)$$

I have added `CoIndStream` as a postulate because we cannot prove it. The reason is that we can only prove equalities for objects which are actually identical. However, the theorem entails that for example

$$\text{from } 1 \equiv \text{mapStream suc } (\text{from } 0)$$

and given the definition of $\equiv$ we should be able to prove this using `refl`. However, we cannot because `from 1` and `mapStream suc (from 0)` are not identical, they do not compute to the same thing — indeed they do not compute at all.

This is a defect of the Type Theory we have introduced so far, which is also called *Intensional Type Theory*. So while the streams given by the two expressions are *extensionally* equal (they are the same infinite sequences), they are not *intensionally* equal, i.e. they are given by different definitions.

Another instance of this phenomena is the lack of functional extensionality, which is the following principle:

postulate

$$\text{funext} : \{f\ g : A \rightarrow B\} \rightarrow (x : A) \rightarrow f\ x \equiv g\ x \rightarrow f \equiv g$$

`funext` is basically coinduction for functions and it suffers from the same problem: consider $f, g : \mathbb{N} \rightarrow \mathbb{N}$ with $f\ x = x + 0$ and $g\ x = 0 + x$. Using `rneutr+` we can show that $(x : \mathbb{N}) \rightarrow f\ x \equiv g\ x$ but we cannot prove $f \equiv g$ because we cannot show this using `refl` which is the only constructor for equality.

Yes, we can just postulate `Coind` and `funext` but this is not only unsatisfying it also destroys the computational behaviour of type theory. We are thinking of natural numbers as generated from 0 and `suc` but with postulates present we may have a number which cannot be reduced to a numeral because somewhere it contains an instance of one of the postulates.

We will fix this problem later, when we are going to use a different definition of equality in cubical type theory.

9.5 History

Eliminators were used by Per Martin-Löf, examples can be found in the book with Sambin [MLS84]. However, the equality eliminator is missing from this book because Martin-Löf used equality reflection at this point — this is referred to as extensional type theory. J is presented under the name `eqpeel` in [NPS90]. Early examples of eliminators can be found in [BCMS89] and a systematic account of eliminators for inductive families is in [Dyb94].

The combinator K as an extension of J was introduced by Thomas Streicher in a letter to Per Martin-Löf in 1996 raising the question whether K is derivable from J. This was answered to the negative by the construction of the groupoid model [HS98].

A systematic account of dependent pattern matching and the fact that it can be derived from the eliminators including K can be found in Conor McBride's PhD thesis [McB00b].

Coinduction and bisimulation are well known techniques using in the context of labelled transition systems [San11]. The coinduction principle for streams we have presented can be found in [JR97].

9.6 Exercises

1. Implement decidability of equality for natural numbers using only EN.

$$_ \equiv \mathbb{N} \text{?-e} _ : (m\ n : \mathbb{N}) \rightarrow \text{Dec } (m \equiv n)$$

You also need the recursor for Dec:

$$\begin{aligned} \text{RDec} &: (A \rightarrow M) \rightarrow (\neg A \rightarrow M) \rightarrow \text{Dec } A \rightarrow M \\ \text{RDec } m\text{-yes } m\text{-no } (\text{yes } x) &= m\text{-yes } x \\ \text{RDec } m\text{-yes } m\text{-no } (\text{no } x) &= m\text{-no } x \end{aligned}$$

2. Derive eliminators for the inductive datatypes from section 4.3: `List`, `Expr` and `Ord`. I am intentionally not including `RoseTree`.
3. Prove that equality satisfies all the laws of a groupoid using only J

$$\begin{aligned} \text{lneutr} &\equiv : \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans refl } p \equiv p \\ \text{rneutr} &\equiv : \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } p \text{ refl} \equiv p \\ \text{assoc} &\equiv : \{a\ b\ c\ d : A\} (p : a \equiv b) (q : b \equiv c) (r : c \equiv d) \\ &\quad \rightarrow \text{trans } p (\text{trans } q\ r) \equiv \text{trans } (\text{trans } p\ q)\ r \\ \text{linv} &\equiv : \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } (\text{sym } p)\ p \equiv \text{refl} \\ \text{rinv} &\equiv : \{a\ b : A\} (p : a \equiv b) \rightarrow \text{trans } p (\text{sym } p) \equiv \text{refl} \end{aligned}$$

4. Show that $\mathbb{N}_\infty$ with $_ +_\infty _$ and `zero $\infty$` is a commutative monad, i.e. show

$$\begin{aligned} \text{rneutr} +_\infty &: \{m : \mathbb{N}_\infty\} \rightarrow m +_\infty \text{zero}_\infty \equiv m \\ \text{lneutr} +_\infty &: \{m : \mathbb{N}_\infty\} \rightarrow \text{zero}_\infty +_\infty m \equiv m \end{aligned}$$

```

assoc+∞ : {l m n : ℕ∞} → (l +∞ m) +∞ n ≡ l +∞ (m +∞ n)
comm+∞ : {m n : ℕ∞} → m +∞ n ≡ n +∞ m

```

using coinduction for natural numbers. To formulate `Colndℕ∞` we need an operation to lift a relation over `Maybe`

```

data MaybeR (_~_ : A → A → prop) :
  Maybe A → Maybe A → prop where
  nothing : MaybeR _~_ nothing nothing
  just : {m' n' : A} → m' ~ n'
        → MaybeR _~_ (just m') (just n')

```

Now a bisimulation is a relation which is stable under the destructor using `MaybeR`. Hence we arrive at the following coinduction principle which has to be postulated as usual.

```

postulate
  Colndℕ∞ : (_~_ : ℕ∞ → ℕ∞ → Set)
    → ({m n : ℕ∞} → m ~ n
       → MaybeR _~_ (pred∞ m) (pred∞ n))
    → {m n : ℕ∞} → m ~ n → m ≡ n

```

To solve this exercise you need some agda tools we haven't yet introduced:

The inspect pattern When we pattern match over an expression is in

```

f ... with pred∞ n
f ... | nothing = ?
f ... | just n' = ?

```

we don't know that `pred∞ n ≡ nothing` respectively `pred∞ ≡ just n'` which we will need in the proof. To address this we can use the `inspect`-pattern which uses the following type:

```

data Singleton {A : Set} (x : A) : Set where
  _with≡_ : (y : A) → x ≡ y → Singleton x

```

and `inspect` is a function which produces elements of `Singleton`:

```

inspect : ∀ {A : Set} (x : A) → Singleton x
inspect x = x with≡ refl

```

Now we can use `inspect` in the **with**-clause above:

```

f ... with inspect (pred∞ n)
f ... | nothing with≡ nothing-eq = ?
f ... | just m' with≡ just-eq    = ?

```

Now `nothing-eq : pred∞ n ≡ nothing` and `just-eq : pred∞ ≡ just n'` provide the missing equations.

The rewrite clause To get expressions using with unstuck you need to use `rewrite`. An example is to derive the defining equations for `_+∞_`:

```
eq+∞-nothing : {x y : N∞} → pred∞ x ≡ nothing
               → pred∞ (x +∞ y) ≡ pred∞ y
eq+∞-nothing p rewrite p = refl
eq+∞-just : {x y x' : N∞} → pred∞ x ≡ just x'
            → pred∞ (x +∞ y) ≡ just (x' +∞ y)
eq+∞-just p rewrite p = refl
```

This is also handy in conjunction with the `inspect-pattern`, e.g.

```
f ... with inspect (pred∞ n)
f ... | nothing with ≡ nothing-eq rewrite nothing-eq = ?
f ... | just m' with ≡ just-eq      rewrite just-eq    = ?
```

Chapter 10

Universes and paradoxes

We have introduced the type of sets `Set` (should really be called `Type`), it's elements are types like $\mathbb{N} : \text{Set}$ but what is the type of `Set` itself? The obvious naive answer is `Set` that is `Set : Set` but this rings a bell. Wasn't there a problem with this?

Indeed, when Frege was developing the foundations of Mathematics based on naive set theory he also assumed that there is a set of all sets. The british mathematician and philosopher Russell saw this and sent him a letter explaining that this assumption is inconsistent. This is what we now call *Russell's paradox*.

Do we have Russell's paradox in Type Theory? One may first think maybe not since we cannot talk about elements in Type Theory and this seems to be an essential part of Russell's paradox. Maybe this was the reason that Per Martin-Löf included this assumption in an early version of Type Theory. In his case it was the French logician Jean-Yves Girard who pointed out that this is inconsistent because he had studied a type-theoretic system, System U, that was also inconsistent and which was actually weaker than `Type : Type`. This paradox is now known as Girard's paradox.

However, it turns out that using inductive types we can directly encoded a variant of Russell's paradox in Type Theory, this was called the paradox of trees by Thierry Coquand.

10.1 The paradox of trees

Sets in the sense of set theory are trees where the order and multiplicity of subtrees doesn't matter. It turns out that to implement Russell's paradox we can just use ordinary trees we don't need to quotient them.

Hence we come up with the following definition of trees:

```
data Tree : Set where
  node : (l : Set) → (l → Tree) → Tree
```

Now, agda rejects this definitions because it knows that we are doing something naughty. It's size checker says that we cannot define a `Set` whose constructor has a `Set` as an argument. As we will see it is perfectly correct but since our plan is to derive a paradox we override this by adding the pragma `NO_UNIVERSE_CHECK` indicating that we are intentionally naughty.

We can play set theory with `Tree`. First of all the empty tree (`[]`), corresponding to $\{\}$

```
[] : Tree
[] = node ⊥ λ ()
```

Now we can define the tree whose only subtree is the empty tree (`[] []`) corresponding to $\{\{\}\}$:

```
[] [] : Tree
>[] [] = node ⊤ (λ _ → [])
```

and the tree containing the two previous trees (`[] [], [] []`) corresponding to $\{\{\}, \{\{\}\}\}$:

```
[] [], [] [] : Tree
>[] [], [] [] = node Bool (λ x → if x then [] else [] [])
```

We can also encode von Neumann's definition of natural numbers, where $n = 0, 1, \dots, n - 1$:

```
ℕ-tree : Tree
ℕ-tree = node ℕ λ n → node (Fin n) (fin-trees {n})
  where fin-trees : {n : ℕ} → Fin n → Tree
        fin-trees {zero} ()
        fin-trees {suc n} zero = node (Fin n) (fin-trees {n})
        fin-trees {suc n} (suc j) = fin-trees {n} j
```

Indeed we can define the `_∈_` relation on trees, where $t \in u$ expresses that t is a direct subtree of u :

```
data _∈_ : Tree → Tree → Set where
  ∈-in : {l : Set} (f : l → Tree) (i : l) → f i ∈ node l f
```

The main idea of Russell's paradox is that we define a good tree to be a tree that doesn't contain itself and then show that the tree of all good trees is good iff it is not good:

```
Bad : Tree → Set
Bad t = t ∈ t

Good : Tree → Set
Good t = ¬ (Bad t)
```

We call the tree of all good trees **Russell**:

Russell : Tree
 Russell = node ($\Sigma[t \in \text{Tree}] \text{Good } t$) proj₁

Now we can show that Russell is good iff it is bad

strange : Good Russell $\Leftrightarrow$ Bad Russell
 proj₁ strange h = $\in\text{-in proj}_1 (\text{Russell}, h)$
 proj₂ strange ($\in\text{-in} \circ (\text{proj}_1) (. \text{Russell}, \text{good})$) = good

Because if Russell is good then it must contain itself hence it is bad but if it is bad, then it contains itself and hence it must be good, because Russell only contains good trees.

Now we have a situation that $P \Leftrightarrow \neg P$ and indeed this was an exercise of propositional logic to show that this is inconsistent:

ex : $\neg (\neg P \Leftrightarrow P)$
 ex {P} (npp , pnp) = pnp p p
where np : $\neg P$
 np p = pnp p p
 p : P
 p = npp np

Combining strange and ex we can prove $\perp$:

paradox : $\perp$
 paradox = ex strange

Clearly, this shows that our assumption that Set : Set is unsound.

Bibliography

- [AGNvS94] Thorsten Altenkirch, Veronica Gaspes, Bengt Nordström, and Björn von Sydow. A user's guide to alf. *Chalmers University of Technology, Sweden*, 1994.
- [AHS95] Thorsten Altenkirch, Martin Hofmann, and Thomas Streicher. Categorical reconstruction of a reduction free normalization proof. In David Pitt, David E. Rydeheard, and Peter Johnstone, editors, *Category Theory and Computer Science*, LNCS 953, pages 182–199, 1995.
- [Alt94] Thorsten Altenkirch. Programming + verification = progification. available from <http://www.cs.nott.ac.uk/~txa/publ/progify94.pdf>, 1994.
- [Alt96] Thorsten Altenkirch. Integrated verification in Type Theory. Lecture notes for a course at ESSLLI 96, Prague, 1996. available from <http://www.cs.nott.ac.uk/~txa/publ/esslli96.pdf>.
- [AMM05] Thorsten Altenkirch, Conor McBride, and James McKinna. Why dependent types matter. Manuscript, available online, April 2005.
- [AP13] Andreas M Abel and Brigitte Pientka. Wellfounded recursion with copatterns: A unified approach to termination and productivity. *ACM SIGPLAN Notices*, 48(9):185–196, 2013.
- [Aug98] Lennart Augustsson. Cayenne—a language with dependent types. In *International School on Advanced Functional Programming*, pages 240–267. Springer, 1998.
- [B⁺84] Hendrik P Barendregt et al. *The lambda calculus*, volume 3. North-Holland Amsterdam, 1984.
- [BCMS89] Roland Backhouse, Paul Chisholm, Grant Malcolm, and Erik Saa-man. Do-it-yourself type theory. *Formal Aspects of Computing*, 1(1):19–84, 1989.
- [Bet48] Evert W Beth. Semantical considerations on intuitionistic mathematics. 1948.

- [BMS80] Rod M Burstall, David B MacQueen, and Donald T Sannella. Hope: An experimental applicative language. In *Proceedings of the 1980 ACM conference on LISP and functional programming*, pages 136–143, 1980.
- [Bra17] Edwin Brady. *Type-driven development with Idris*. Manning Publications Company, 2017.
- [BS91] U Berger and H Schwichtenberg. An inverse of the evaluation functional for typed lambda-calculus. In *[1991] Proceedings Sixth Annual IEEE Symposium on Logic in Computer Science*, pages 203–211. IEEE, 1991.
- [CH06] Felice Cardone and J Roger Hindley. History of lambda-calculus and combinatory logic. *Handbook of the History of Logic*, 5:723–817, 2006.
- [Chu32] Alonzo Church. A set of postulates for the foundation of logic. *Annals of mathematics*, pages 346–366, 1932.
- [Chu40] Alonzo Church. A formulation of the simple theory of types. *The journal of symbolic logic*, 5(2):56–68, 1940.
- [Coq93] Thierry Coquand. Infinite objects in type theory. In *International Workshop on Types for Proofs and Programs*, pages 62–78. Springer, 1993.
- [Cur30] Haskell Brooks Curry. Grundlagen der kombinatorischen logik. *American journal of mathematics*, 52(4):789–834, 1930.
- [dt20] The Coq development team. *The Coq proof assistant reference manual*. LogiCal Project, 2020+.
- [Dyb94] Peter Dybjer. Inductive families. *Formal aspects of computing*, 6(4):440–465, 1994.
- [Fre79] Gottlob Frege. *Begriffsschrift, eine der arithmetischen nachgebildete Formelsprache des reinen Denkens*. Nebert, 1879.
- [G⁺95] PRL Group et al. Implementing mathematics with the nuprl proof development system. *Computer Science Department, Cornell University*, 1995.
- [Gen35] Gerhard Gentzen. Untersuchungen über das logische schließen. i. *Mathematische zeitschrift*, 39(1):176–210, 1935.
- [Gir72] Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures de l’arithmétique d’ordre supérieur*. PhD thesis, PhD thesis, Université Paris VII, 1972.

- [GMM⁺78] Mike Gordon, Robin Milner, Lockwood Morris, Malcolm Newey, and Christopher Wadsworth. A metalanguage for interactive proof in lcf. In *Proceedings of the 5th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 119–130, 1978.
- [Göd58] Von Kurt Gödel. Über eine bisher noch nicht benützte erweiterung des finiten standpunktes. *dialectica*, 12(3-4):280–287, 1958.
- [HA99] David Hilbert and Wilhelm Ackermann. *Principles of mathematical logic*, volume 69. American Mathematical Soc., 1999.
- [How80] William A Howard. The formulae-as-types notion of construction. *To HB Curry: essays on combinatory logic, lambda calculus and formalism*, 44:479–490, 1980.
- [HS98] Martin Hofmann and Thomas Streicher. The groupoid interpretation of type theory. *Twenty-five years of constructive type theory (Venice, 1995)*, 36:83–111, 1998.
- [Hut16] Graham Hutton. *Programming in Haskell*. Cambridge University Press, 2016. 2nd edition.
- [JR97] Bart Jacobs and Jan Rutten. A tutorial on (co) algebras and (co) induction. *Bulletin-European Association for Theoretical Computer Science*, 62:222–259, 1997.
- [Kri59] Saul A Kripke. A completeness theorem in modal logic. *The journal of symbolic logic*, 24(1):1–14, 1959.
- [Kur29] Gödel Kurt. Über die vollständigkeit des logikkalküls. *PhD thesis*, 1929.
- [LP92] Zhaohui Luo and Robert Pollack. *LEGO proof development system: User’s manual*. LFCS, Department of Computer Science, University of Edinburgh, 1992.
- [McB00a] Conor McBride. Dependently typed functional programs and their proofs. 2000.
- [McB00b] Conor McBride. *Dependently typed functional programs and their proofs*. PhD thesis, University of Edinburgh, UK, 2000.
- [McB04] Conor McBride. Epigram: Practical programming with dependent types. In *International School on Advanced Functional Programming*, pages 130–170. Springer, 2004.
- [McB09] Conor McBride. Winging it. slides from a talk at WG2.8 #26, Frauenchiemsee, 2009. the original slides are about 10 years older.

- [McC60] John McCarthy. Recursive functions of symbolic expressions and their computation by machine, part i. *Communications of the ACM*, 3(4):184–195, 1960.
- [ML75] Per Martin-Löf. An intuitionistic theory of types: Predicative part. In *Studies in Logic and the Foundations of Mathematics*, volume 80, pages 73–118. Elsevier, 1975.
- [ML78] S Mac Lane. Categories for the working mathematician. *Springer*, 54:55–81, 1978.
- [MLS84] Per Martin-Löf and Giovanni Sambin. *Intuitionistic type theory*, volume 9. Bibliopolis Naples, 1984.
- [MM04] Conor McBride and James McKinna. The view from the left. *Journal of functional programming*, 14(1):69–111, 2004.
- [Nor08] Ulf Norell. Dependently typed programming in agda. In *International school on advanced functional programming*, pages 230–266. Springer, 2008.
- [NPS90] Bengt Nordström, Kent Petersson, and Jan M Smith. *Programming in Martin-Löf’s type theory*, volume 200. Oxford University Press Oxford, 1990.
- [Ros50] PC Rosenbloom. The elements of mathematical logic, new york, 1950.
- [San11] Davide Sangiorgi. *Introduction to bisimulation and coinduction*. Cambridge University Press, 2011.
- [Sch24] Moses Schönfinkel. Über die bausteine der mathematischen logik. *Mathematische annalen*, 92(3-4):305–316, 1924.
- [Sch68] Kurt Schütte. *Vollständige Systeme modaler und intuitionistischer Logik*, volume 42. Springer-Verlag, 1968.
- [TvD88a] Anne Sjerp Troelstra and Dirk van Dalen. Constructivism in mathematics: An introduction. vol. ii, vol. 123 of. *Studies in Logic and the Foundations of Mathematics*, 1988.
- [TvD88b] Anne Sjerp Troelstra and Dirk van Dalen. Constructivism in mathematics. vol. i, volume 121 of. *Studies in Logic and the Foundations of Mathematics*, 26, 1988.
- [TvU99] AS Troelstra and P van Ulsen. The discovery of ew beth’s semantics for intuitionistic logic. *Essays dedicated to Johan van Benthem on the occasion of his 50th birthday*. University of Amsterdam, Amsterdam, 1999.
- [VD99] Dirk Van Dalen. *Mystic, geometer, and intuitionist: The life of LEJ Brouwer*. Oxford: Clarendon Press, 1999.